Bachelorarbeit

Validation von dynamischen Eigenschaften eines UML und OCL Modells: Eine Fallstudie am Beispiel Toll Collect

Marcel Dechert
geboren am: 17.10.1993
decherma@informatik.uni-bremen.de
Matrikel-Nr.: 2787290



Universität Bremen

Universität Bremen
Fachbereich: 3 (Mathematik, Informatik)

Erstgutachter: Prof. Dr. Martin Gogolla
Zweitgutachter: Dr. Karsten Sohr

eingereicht am 07.10.2016

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Kontext der Arbeit 1

1.2 Ziel der Arbeit 2

1.3 Kapitelüberblick 3

2 Grundlagen **4**

2.1 Definition von Modellen 4

2.2 UML 4

2.2.1 Klassendiagramme 5

2.2.2 Objektdiagramme 7

2.2.3 Sequenzdiagramme 7

2.3 OCL 9

2.4 USE 11

2.5 Modelvalidator 14

2.5.1 Einsatzmöglichkeiten des Modelvalidators 15

2.5.2 Benutzung des Modelvalidators 16

2.6 Filmstrip-Modell 17

3 Beispielmodell: Toll Collect **20**

4 Durchführung der Fallstudie **25**

4.1 Einführung von Frameconditions 25

4.1.1 Point-Operationen 26

4.1.2 Truck-Operationen 28

4.2 Transformation ins Filmstrip-Modell 32

4.3 Testfälle . . . 35
4.3.1 Initialisierungsoperationen . . . 37
4.3.2 Alle Operationen des Trucks . . . 44
4.3.3 Alle Operationen von Truck und Point . . . 47
4.3.4 Ausführungszeiten . . . 49
4.4 Traversieren über ein Streckennetz . . . 49
4.4.1 Streckennetze . . . 50
4.4.2 Bewegung des Trucks . . . 53
4.4.3 Ausführungszeiten . . . 65
4.5 Negativer Testfall . . . 66

5 Fazit und Ausblick **68**
5.1 Fazit . . . 68
5.2 Ausblick . . . 69

Literaturverzeichnis **72**

Anhang **73**
I Toll Collect . . . 73
I.1 Vollständiges Toll Collect-Modell . . . 73
I.2 Transformiertes Toll Collect-Modell . . . 80
II Konfigurationsdateien der Testfälle . . . 91
II.1 Initialisierung eines Trucks . . . 91
II.2 Initialisierung zweier Punkte . . . 92
II.3 Alle Initialisierung . . . 94
II.4 Alle Operationen des Trucks . . . 95
II.5 Alle Operationen von Truck und Point . . . 97
III Konfigurationsdateien für Streckennetze . . . 99
III.1 Skripte zur Erstellung der Streckennetze . . . 99
III.2 Konfigurationsdateien für Bewegungen . . . 100
IV Negativer Testfall . . . 115

V Kommandozeilenausgaben der Testfälle 117
V.1 Initialisierung eines Trucks 117
V.2 Initialisierung zweier Punkte 117
V.3 Alle Initialisierungsoperationen 118
V.4 Alle Operationen des Trucks 118
V.5 Alle Operationen von Truck und Point 119
VI Kommandozeilenausgaben für Streckennetze 120
VI.1 Zwei Städte, eine Bewegung 120
VI.2 Zwei Städte, zwei Bewegung 121
VI.3 Zwei Städte, drei Bewegung 121
VI.4 Drei Städte, eine Bewegung 122
VI.5 Drei Städte, zwei Bewegungen 122
VI.6 Drei Städte, drei Bewegungen 123
VI.7 Fünf Städte, eine Bewegung 123
VI.8 Fünf Städte, zwei Bewegungen 123
VI.9 Fünf Städte, drei Bewegungen 124
VII Kommandozeilenausgaben für negativen Testfall 124

Abbildungsverzeichnis

2.1 Klassendiagramm 5
2.2 Klassendiagamm mit Generalisierung 6
2.3 Objektdiagramm 7
2.4 Objektdiagramm 8
2.5 GUI von USE 12
2.6 Transformation in das Filmstrip-Modell 19

3.1 Ursprüngliches Klassendiagramm des Toll Collect-Modells 20
3.2 Zustandsmaschinendiagramm des Trucks 23
3.3 Zustandsmaschinendiagramm des Points 23

4.1 Klassendiagramm vor der Transformierung 33
4.2 Optionen des Transformationstools 33
4.3 Neues Klassendiagramm 34
4.4 Objektdiagramm mit Initialisierungsoperationen eines Trucks 41
4.5 Objektdiagramm mit Initialisierungsoperationen zweier Punkte 42
4.6 Objektdiagramm mit allen Initialisierungsoperationen 44
4.7 Objektdiagramm mit allen Operationen des Trucks 46
4.8 Objektdiagramm mit allen Operationen 48
4.9 Streckennetz mit zwei Städten und einer Verbindungen 52
4.10 Streckennetz mit drei Städten und drei Verbindungen 52
4.11 Streckennetz mit fünf Städten und sechs Verbindungen 52
4.12 Objektdiagramm mit zwei Städten und einer Bewegung 54
4.13 Objektdiagramm mit zwei Städten und zwei Bewegungen 55
4.14 Objektdiagramm mit zwei Städten und drei Bewegungen 56
4.15 Objektdiagramm mit drei Städten und einer Bewegung 58
4.16 Objektdiagramm mit drei Städten und zwei Bewegungen 59
4.17 Objektdiagramm mit drei Städten und drei Bewegungen 60
4.18 Objektdiagramm mit fünf Städten und einer Bewegung 62

4.19 Objektdiagramm mit fünf Städten und zwei Bewegungen 63
4.20 Objektdiagramm mit fünf Städten und drei Bewegungen 64

Tabellenverzeichnis

4.1 Übersicht über Frameconditions des Point 26
4.2 Übersicht über Framecondtions des Trucks 28
4.3 Übersicht über alle Konfigurationen der Fallstudie 35
4.4 Übersicht über Konfigurationen der Initialisierungstestfälle 37
4.5 Konfiguration für Testfall mit allen Truck-Operationen 45
4.6 Konfiguration für Testfall mit allen Operationen 47
4.7 Übersicht über die Lösungszeiten der Testfälle 49
4.8 Überblick über Streckennetze 51
4.9 Bewegungen über Streckennetz mit zwei Städten 53
4.10 Bewegungen über Streckennetz mit drei Städten 57
4.11 Bewegungen über Streckennetz mit fünf Städten 61
4.12 Lösungszeiten aller Partial Solution Completion-Testfälle 65
4.13 Konfiguration für negativen Testfall 67

Listings

2.1 Precondition *immartikulieren* 10
2.2 SOIL-Skript für Studenten und Universität 13
3.1 Klasse *Point* in USE 21
3.2 *Connection* in USE 21
3.3 *Truck* in USE 22
3.4 *Current* in USE 22
3.5 *freshPoint*-Precondition in USE 24
3.6 *targetReachable*-Precondition in USE 24
3.7 *debtIncreased*-Postcondition in USE 24
4.1 Frameconditions für Operation *init* des Points 27
4.2 Frameconditions für Operation *northConnect* des Points 27
4.3 Frameconditions für Operation *southConnect* des Points 28
4.4 Frameconditions für Operation *init* des Trucks 29
4.5 Frameconditions für Operation *enter* des Trucks 29
4.6 Frameconditions für Operation *move* des Trucks 30
4.7 Frameconditions für Operation *pay* des Trucks 31
4.8 Frameconditions für Operation *bye* des Trucks 31
4.9 Öffnen des Modells in USE 32
4.10 Einstellung des SatSolvers 38
4.11 Einstellung der Bitbreite 38
4.12 Speichern der Einstellungen 38
4.13 Konsolenausgabe für Truck-Initialisierungs-Testfall 40
4.14 automaticDiagramextraction Option aktivieren 49
4.15 Streckennetz in USE laden 50
4.16 Validierung starten 50
4.17 Invariante *debtNotBelowZero* 66
4.18 Konsolenausgabe für negativen Testfall 67

1 Einleitung

1.1 Kontext der Arbeit

In der heutigen Softwareentwicklungspraxis gewinnt die Modellierung, speziell mit UML, immer mehr an Bedeutung. Modelle ermöglichen es, trotz ihrer reduzierten Komplexität, die wichtigen Eigenschaften eines Systems zu erfassen und somit durch eine gute Modellierung einer fehlerbehafteten Implementierung vorzubeugen. So können durch ein gründliches Testen des Modells im Vorlauf der Implementierung viele Unstimmigkeiten frühzeitig entdeckt und behoben werden. Mit dieser gesteigerten Wichtigkeit der Modellierung einhergehend, die sich auch durch den Aufstieg des Softwareentwicklungsparadigma Model-driven engineering (MDE) äußert, entsteht auch ein Anspruch an das Modell und eine Notwendigkeit für dessen Richtigkeit. Ein Modell für eine Software ist in diesem Sinne vergleichbar mit der Blaupause für ein Gebäude in der Architektur. Es ist möglich, ein Haus zu bauen, ohne vorher einen Plan zu erstellen, genauso wie es möglich ist, eine Software zu schreiben, ohne sich vorher Gedanken in Form einer Modellierung zu machen, jedoch ist die Wahrscheinlichkeit höher, je größer das Projekt ist, dass nicht das Richtige programmiert wird.[1]

Um dieses Risiko zu minimieren, gibt es den Prozess der Validierung[2]. Die Validierung beschreibt den Abgleich der existierenden Eigenschaften des Modells mit den informellen Erwartungen an das Verhalten des Systems. Die Validierung soll gewährleisten, dass das System im Sinne der Praxistauglichkeit für den vorgesehenen Zweck geeignet ist. Komplementär dazu ist der Begriff Verifikation von Bedeutung. Die Verifikation soll zeigen, dass die formalen Eigenschaften des Systems seiner Spezifikation entspricht. Der Unterschied zwischen Validierung und Verifikation lässt sich mit zwei kurzen Fragen zusammenfassen: Verifikation ist die Suche nach der Antwort auf die Frage: Tun wir es richtig? Der Validierungsprozess sucht nach der Antwort auf die Frage: Tun wir das Richtige?[3]

[1] Vgl. Jacobson/Booch/Rumbaugh (1999).
[2] Der Begriff Validation wir synonym verwendet.
[3] Vgl. Sommerville (2012).

Allerdings ist es wichtig, nicht nur statische Modelle zu betrachten, sondern auch die Dynamik des Systems abzubilden. Dadurch lässt sich nicht nur überprüfen, ob es einen zulässigen Zustand für die Spezifikation gibt, sondern ob Zustandsübergänge funktionieren und ob der Zielzustand korrekt ist. Außerdem können die verschiedenen Objekte und Attribute über die verschiedenen Zustände hinweg betrachtet werden, um festzustellen, ob diese sich wie gewünscht verändern.

Um jedoch die Eigenschaften eines dynamischen Systems überprüfen zu können, bedarf es einer Transformation des Modells, sodass das Verhalten der Systembeschreibung im simulierten, laufenden System überprüfbar wird.

1.2 Ziel der Arbeit

In dieser Arbeit wird auf dem bestehenden Toll Collect-Modell aufgebaut, welches eine Abstraktion des LKW-Maut-Systems in Deutschland ist. Dazu wird das vorhandene Modell erweitert und letztendlich die dynamischen Eigenschaften des Systems untersucht. Ergebnis der Arbeit soll sein, in einem mit Invarianten beschriebenen Modell eines Informationssystemes automatisch Testfälle in Form von Objektdiagrammen zu erzeugen.

Um dies zu erreichen, müssen mehrere Schritte durchgeführt werden. Zunächst muss das Modell um sogenannte Frameconditions angereichert, dann soll die Toll Collect-Modellierung in das Filmstrip-Modell transformiert werden, um anschließend Testzustände des Systems mithilfe des Modelvalidators erzeugen zu können. Die Frameconditions benötigt man, um zu verhindern, dass das System im Filmstrip-Modell bei Operationsanwendung den Zustand ungewünscht verändert. Durch die Transformation ins Filmstrip-Modell wird das dynamische Verhalten des Systems in einem statischen Modell beschrieben. Über das Filmstrip-Modell, auch Snapshot-Modell genannt, wird in ähnlicher Form auch an anderen Universitäten geforscht.[4] Letztendlich wird dann vom Modelvalidator ein Zustand

[4] Yu/France/Ray (2008).

gesucht, der den vorgegebenen Bedingungen genügen muss. Es gilt dann zu beobachten, ob das veränderte Modell immer noch konsistent ist, d. h. ob überhaupt ein entsprechendes Objektdiagramm gefunden werden kann. Wenn dies der Fall ist, kann darüber hinaus geprüft werden, ob die Eigenschaften des Ausgangs-Modells, also die Semantik des Modells, auch nach der Transformation weiterhin vorhanden ist, indem die Belegungen der Objekte und Attribute näher betrachtet werden. Dies alles wird unter Einsatz des USE-Tools (UML-based Specification Environment) mitsamt der adäquaten Plugins realisiert. Dabei sollen Bedienbarkeit und Praxistauglichkeit der Modellierung und Validierung mit den genannten Hilfsmitteln evaluiert werden.

1.3 Kapitelüberblick

Im 2. Kapitel werden die nötigen Grundlagen erläutert. Dazu gehören eine generelle Definition des Begriffs „Modell“, die Modellierungssprache UML und deren Erweiterung OCL, sowie das verwendete Tool USE. Anschließend werden die nötigen Techniken, zuerst das Filmstrip-Modell zur Transformation des Modells und dann der Modelvalidator zum Validieren der Testfälle vorgestellt.

Im folgenden 3. Kapitel wird als Beispielmodell Toll Collect in seiner ursprünglichen Form, auf dem in der weiteren Arbeit aufgebaut wird, beschrieben.

Im 4. Kapitel Durchführung der Fallstudie werden zunächst durch die Einführung von Frameconditions die notwendigen Änderungen am Modell vorgenommen. Im nächsten Abschnitt folgt dann die Transformation ins Filmstrip-Modell. Anschließend werden einige Testfälle in Form von Objektdiagrammen betrachtet. Im nächsten Abschnitt steht das Traversieren über ein Streckennetz im Fokus. Im letzten Abschnitt des Kapitels wird ein negativer Testfall betrachtet.

Im 5. Kapitel Fazit und Ausblick wird eine abschließende Betrachtung vorgenommen.

2 Grundlagen

2.1 Definition von Modellen

Zunächst gilt es die Frage zu erörtern, was genau ein Modell ist. Ein Modell ist eine Abstraktion eines Systems einer realen Domäne, die aus einer bestimmten Sicht zu einem bestimmten Zweck zu einem bestimmten Zeitpunkt betrachtet wird. Diese Abstraktion von der realen Welt führt dazu, dass sich bei der Modellierung der Detaillierungsgrad und die Informationsdichte reduzieren.[5] Modelle können verschiedene Aufgaben erfüllen bzw. dabei hilfreich sein. Am naheliegendsten ist die Nutzung zur Spezifikation eines Systems. Modelle können aber auch auf einer grundlegenden Ebene dabei helfen das Problem und die entsprechenden Lösungen zu verstehen. Sie können des Weiteren der Kommunikation dienen und somit zur Verdeutlichung der Modellierungsabsicht beitragen.[6]

2.2 UML

Die UML (Unified Modeling Language) ist die am weitesten verbreitete Modellierungssprache zur Spezifikation und Dokumentation von Informationssystemen. Sie wurde von der Object Management Group (OMG) entwickelt und ist sowohl von ihr als auch von der ISO (ISO/IEC 19505) standardisiert. Die UML ist der Versuch, eine einzige standardisierte Sprache zur objektorientierten Modellierung zu etablieren.[7]

Im Folgenden werden die für diese Arbeit relevanten Bestandteile der UML genauer erläutert. Naheliegenderweise wird das Klassendiagramm behandelt, da in ihm die grundlegende Struktur des Systems festgelegt wird. Da letztendlich Testzustände des Systems gebaut werden sollen, werden Objektdiagramme genauer erklärt. Außerdem werden Sequenzdiagramme vorgestellt.

[5] Vgl. Stachowiak (1973).
[6] Vgl. Rumbaugh/Jacobson/Booch (2004).
[7] Vgl. Object Management Group (OMG) (2013).

2.2.1 Klassendiagramme

In einem UML Klassendiagramm ist der grundsätzlichen Aufbau des Systems festgelegt, indem die Menge der möglichen Zustände und das Verhalten der abstrahierten Objekte festgelegt wird. In der UML besteht eine Klasse neben ihrem Namen aus zwei Teilen, nämlich einem Abschnitt mit Attributen und einem Abschnitt mit Operationen.

Ein Beispiel wäre ein Klassendiagramm, das *Studenten* und *Universitaeten* modelliert. Wie in der untenstehenden Abbildung 2.1 zu sehen ist, sind die Klassen *Student* und *Universitaet* durch die Assoziation *besucht* verbunden. Die Klasse *Student* hat die Attribute *name*, *alter* und *semester* und die Operation *immatrikulieren*. In Objekten der Klasse *Universitaet* sind *name* und *ort* gespeichert. Die Mulitplizitäten * auf der Seite des *Studenten* und 0..1 bei der *Universitaet* sagen aus, dass eine Universität beliebig viele Studenten haben kann, während ein Student keine oder eine Universität besuchen kann.

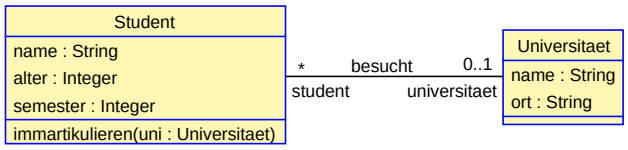


Abbildung 2.1: Klassendiagramm

Zu jedem Attribut wird ein Datentyp angegeben. Dieser kann unter anderem ein numerischer Wert (z. B. Integer oder Double), ein boolscher Wert oder schlicht ein String sein. Die Klassen können auf verschiedene Arten und Weisen miteinander in Beziehung stehen, wobei die grundlegende Art die Assoziation ist, welche die Interaktion zwischen den Objekten der Klassen beschreibt. Eine Assoziation, deren Name auf einem Strich zwischen den zwei Klassen abgebildet wird, hat zwei Assoziationsenden, dies sind die Klassen, die durch die Assoziation verbunden sind. Außerdem wird eine Multiplizität bzw. Kardinalität angegeben, die festlegt, wie viele Objekte dieser Klasse in Beziehung mit wie vielen Objekten der anderen Klasse stehen dürfen. Beispielsweise kann ein Objekt

einer Klasse in Beziehung zu vielen Objekten einer anderen Klasse stehen (1-n) oder eine Viele-zu-Viele Beziehung (n-m) bestehen. Assoziationen können auch reflexiv sein, wenn an beiden Enden dieselbe Klasse ist. Ein gutes Beispiel für einen Zusammenhang, in dem eine solche Assoziation sinnvoll ist, ist ein Modell, das durch eine Klasse Person, die reflexiv mit sich selbst verbunden ist, Eltern-Kind-Beziehungen darstellt. Eine andere Form ist die Generalisierung. Eine Generalisierung ist eine „Teil von“-Beziehung zwischen einer Oberklasse (generellere Klasse) und einer Unterklasse (speziellere Klasse). Auf diese Weise kann Vererbung in UML dargestellt werden kann, die Unterklasse hat also implizit alle Attribute der Oberklasse. Sie wird durch eine gerichtete Assoziation mit einem nicht ausgefüllten Pfeil am Assoziationsende der Oberklasse gekennzeichnet. Als Beispiel zeigt Abbildung 2.2 die generelle Klasse Person und zwei spezielle Klassen Student und Professor.

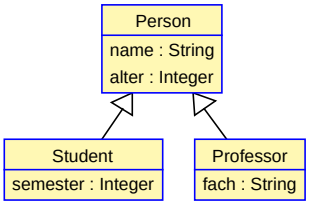


Abbildung 2.2: Klassendiagamm mit Generalisierung

Daneben gibt es spezielle Ausprägungen der Assoziation, nämlich Aggregation und Komposition. Eine Aggregation beschreibt eine Menge von Objekten, die zu einem Aggregat zusammengesetzt werden. Die Objekte haben eine Lebensdauer unabhängig von dem Aggregat und werden durch eine nicht ausgefüllte Raute am Assoziationsende des Aggregats dargestellt. Bei einer Komposition, grafisch zu unterscheiden durch die ausgefüllte Raute, besteht eine existenzielle Beziehung, ein Teil kann nicht ohne das Ganze existieren. Beispielsweise ergibt es keinen Sinn, ein Kapitel ohne dazugehöriges Buch zu speichern. Eine an eine Assoziation angeheftete Klasse nennt man Assoziationsklasse, die die Beziehung zwischen den Klassen der Assoziation genauer definiert und zusätzliche Informationen

gekapselt speichern kann. Ein gutes Beispiel hierfür ist eine Bibliothek, in der eine Assoziation zwischen Leser und Buch durch die Assoziationsklasse Ausleihe erweitert wird.

2.2.2 Objektdiagramme

In einem Objektdiagramm wird die Instanziierung des Klassendiagramms dargestellt. Somit kann der aktuelle Zustand eines Systems mit den Belegungen der Attribute aus einem Objektdiagramm abgelesen werden. Um das Objektdiagramm auch terminologisch vom Klassendiagramm abzuheben, gibt es entsprechende Bezeichnungen. Die konkrete Ausprägung einer Klasse wird Objekt genannt, die einer Assoziation heißt Link.[8] Die Abbildung 2.3 zeigt eine mögliche Ausprägung des im vorherigen Kapitel vorgestellten Klassendiagramms. In diesem Fall besuchen die Studenten *hans* und *gretel* die *Universitaet Bremen*.

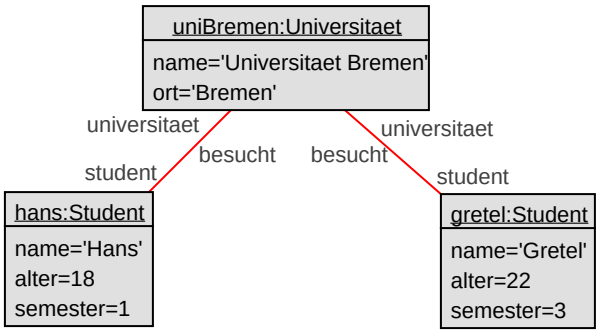


Abbildung 2.3: Objektdiagramm

2.2.3 Sequenzdiagramme

Das Sequenzdiagramm gehört im Kontext der UML zu den Interaktionsdiagrammen. Es stellt, allgemein gesagt, einen Austausch von Nachrichten zwischen verschiedenen Instan-

[8] Object Management Group (OMG) (2013).

zen dar. Dabei hat jede Ausprägung einer Klasse eine Lebenslinie, die vertikal von oben nach unten verläuft und die Lebensdauer des Objekts symbolisiert, wenn beispielsweise ein Objekt durch einen Operationsaufruf erzeugt wird, dann beginnt die Lebenslinie des neuen Objektes an dem Punkt des Operationsaufrufes bzw. falls spezifiziert, nach der Zeit, die es dauert, bis die Operation abschlossen ist.[9] Zwischen den Lebenslinien werden Pfeile gezogen, die den Zeitpunkt, den Startpunkt und das Ziel der Nachricht festlegen. Dabei gibt es verschiedene Arten von Nachrichten: Bei einer synchronen Nachricht (ausgefüllte Pfeilspitze) muss der Sender der Nachricht auf eine Antwort des Empfänger warten, um fortfahren zu können. Eine asynchrone Nachricht hingegen, erlaubt es dem Sender unabhängig von einer Antwort, ohne zu Warten, fortzufahren. Nachrichten könne auch an sich selbst gesendet werden. Die Antwort auf die Nachricht wird mit einer gestrichelten Linie dargestellt. Wenn ein Objekt zerstört wird, endet die Lebenslinie mit einem Stopp-Symbol.[10]

Die Abbildung 2.4 zeigt ein Sequenzdiagramm, in dem die Studenten *hans* und *gretel* jeweils die Operation *immatrikulieren* mit dem Parameter *uniBremen* aufrufen. Durch den Aufruf von *destroy* wird das Objekt gelöscht, womit auch die Lebenslinie endet.

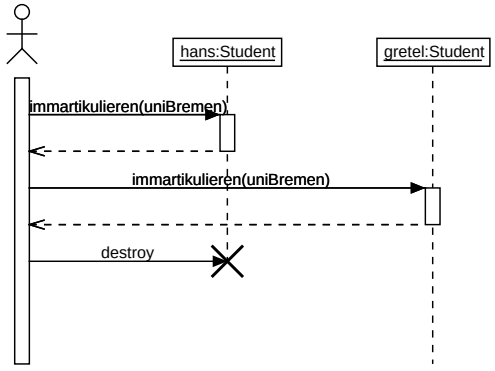


Abbildung 2.4: Objektdiagramm

[9] Object Management Group (OMG) (2013).
[10] Salvanos (2015).

2.3 OCL

OCL, die Object Constraint Lanugage, ist eine Erweiterung der UML. Sie wurde bei IBM, basierend auf IBEL und Syntropy, entwickelt und gehört seit Version 1.1 zur UML.[11] Zuvor waren Constraints in UML nur als informelle Annotationen möglich. Durch OCL kann das in UML modellierte System nun genauer spezifiziert und präzisiert werden. Eine reine UML-Spezifikation kann nicht präzise genug sein, da die grafische Notation der UML bei der Modellierung von spezifischen komplexen Constraints inhärent limitiert ist. Dies kann zu Mehrdeutigkeiten oder sogar Widersprüchen führen, die in der Praxis schwerwiegende Folgen haben können. Daher findet die genauere Spezifizierung mit Hilfe von OCL in textueller und nicht grafischer Form als Constraints statt. Dabei wird jedoch keine Aussage darüber getroffen, wie die Einhaltung der Constraints garantiert werden soll, sondern lediglich, welche Einschränkungen einzuhalten sind.[12]

In OCL gibt es drei verscheidende Arten von Constraints: Klasseninvarianten, Preconditions von Operationen und Postconditions von Operationen. Eine Invariante wird in OCL durch *inv* gekennzeichnet und gibt im Kontext einer Klasse vor, welche Eigenschaften zu jeder Zeit, in jedem Zustand zutreffen sollen. Durch Pre- und Postconditions wird das dynamische Systemverhalten beschrieben. In OCL werden sie durch das Anfügen der Stereotypen *precondition* bzw. *postcondition* identifiziert.

Ein OCL-Ausdruck besteht aus Context, Stereotyp und dem eigentlichen Ausdruck. Der Context gibt den Gültigkeitsbereich, also die entsprechende Klasse an. Es wird zwischen drei verschiedene Stereotypen von Constraints unterschieden, den Invarianten Pre- und Postconditions. Durch Pre- und Postconditions kann das Verhalten des Systems vor und nach der Ausführung von Operationen eingeschränkt werden. Das heißt, es werden Bedingungen formuliert, die sowohl vor (pre) als auch nach (post) der Operationsausführung

[11] Kahlbrandt (2001).
[12] Vgl. Object Management Group (OMG) (2014).

erfüllt sein müssen, um einen zulässigen Systemzustand zu erreichen. Die Auswertung der Constraints liefert somit einen boolschen Wert, der true oder false sein kann. Sogenannte Invarianten beschreiben Bedingungen, die in allen Systemzuständen erfüllt sein müssen. Sie bilden den Rahmen, um die Integrität und Sinnhaftigkeit des Systems stets zu gewährleisten. Der boolschen Ausdruck kann Konjunktionen, Disjunktionen, Negation, Gleichheit, Ungleichheit, Implikation und weitere logische Operatoren enthalten. Die Basisdatentypen, die verwendet werden können, sind Boolean, Integer, Real und String. Außerdem bietet OCL eine Vielzahl an Operationen, auf die zurückgegriffen werden kann.

Das Klassendiagramm über Studenten und Universitäten von vorher aufgreifend, könnte ein dazugehöriges OCL-Constraint (Listing 2.1) folgendermaßen aussehen: Ein Student darf bei keiner Universität eingeschrieben sein, wenn er sich immatrikulieren will.

Listing 2.1: Precondition *immartikulieren*

```
1 context Student::immartikulieren(uni:Universitaet)
2 pre notImarticulated:
3   self.universitaet->isEmpty
```

Der Kontext, in dem das OCL Constraint gelten soll, ist die Operation *immatrikulieren* des Studenten. Danach folgt das *pre*, das angibt, dass es sich um eine Precondition handelt und dann folgt *notImarticulated*, der Name der Precondition. In der nächsten Zeile befindet sich der OCL-Ausdruck, der die Precondition inhaltlich definiert.

2.4 USE

Die UML-based Specification Environment (USE)[13] ist ein im Kontext der AG Datenbanksysteme an der Universität Bremen entwickeltes Tool zur Modellierung, Verifikation und Validierung von Informationssystemen. Die Arbeit an USE begann 1998 mit der Doktorarbeit von Mark Richters.[14] Seitdem wird das Tool stetig weiterentwickelt und um neue Funktionen ergänzt.[15]

Es existieren viele weitere UML-Tools, die die Modellierung und grafische Visualisierung von UML unterstützen, jedoch nur wenige, häufig akademische Tools, die OCL adäquat mit einbeziehen. Neben USE gibt es einige weitere Tools, die dieses Problem angehen. Beispielsweise zu nennen ist Dresden OCL[16], das sowohl als eigenständiges Tool funktioniert, als auch als Plugin für das etablierte CASE-Tool ArgoUML.[17]

USE verfügt über zwei Benutzerschnittstellen. Zum einen die GUI, die unter anderem die grafische Darstellung der Diagramme beinhaltet und die Interaktion über die Konsole. Die Spezifikationen, d. h. die textuelle Beschreibung der Klassen, Assoziationen, Operationen und Constraints, wird vorwiegend in Textform vorgenommen, welche in einer *.use* Datei gespeichert werden.

Die Abbildung 2.5 zeigt das geöffnete USE-Tool mit verschiedenen geöffneten Fenstern. USE bietet diverse Ansichten, um das Modell zu betrachten, im untenstehenden Bild sind z. B. die *Command list*, welche alle ausgeführten Befehle auflistet und das Fenster *Class invariants*, das alle Invarianten des Systems und deren Auswertung anzeigt, sowie das Fenster *Class extend*, in dem alle Objekte einer Klasse und deren Attribute zu erkennen sind.

[13] https://sourceforge.net/projects/useocl/
[14] Vgl. Richters (2002).
[15] Vgl. Gogolla/Büttner/Richters (2007).
[16] http://www.dresden-ocl.org/index.php/DresdenOCL
[17] Demuth (2004).

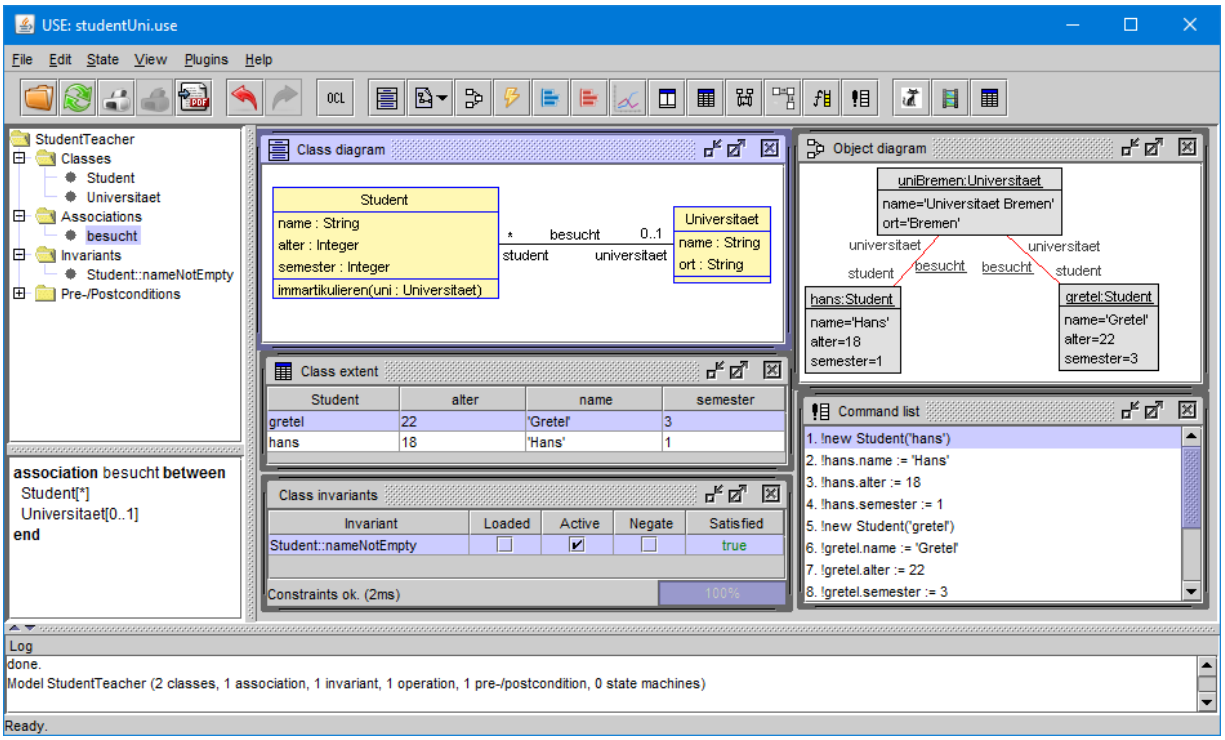


Abbildung 2.5: GUI von USE

Das Tool bietet viele Möglichkeiten mit dem geladenen Modell zu interagieren, z. B. können OCL-Anfragen gestellt oder der Zustand manipuliert werden, indem einzelne Objekte oder Links erzeugt oder gelöscht oder Attribute geändert werden. Des Weiteren kann man Objekt-, Sequenz- und Kommunikationsdiagramme erstellen. Die Betrachtung des dynamisches Verhalten gelingt mit Hilfe von state machines, indem eine Abfolge von Operationsaufrufen eingegeben wird.

USE protokolliert alle wichtigen Eingaben und Ausgaben in einem Log und bietet darüber hinaus die Möglichkeit durch den Evaluation Browser nachzuvollziehen, an welcher Stelle bzw. aus welchem Grund ein Constraint nicht erfüllt ist.

Daneben besteht die Möglichkeit, ein *.SOIL*-Skript (Simple OCL-like Imperative programming Language) zu schreiben bzw. aus den aktuell im System vorhandenen Daten zu exportieren. Dieses Skript enthält alle Befehle, die zur Erzeugung des aktuellen Zustands notwendig sind. Damit werden Objektdiagramme indirekt abgespeichert, um sie zu einem späteren Zeitpunkt weiter bearbeiten zu können. Ein solches Skript kann man natürlich auch manuell erstellen.

Im Beispiel vom Klassendiagramm über Studenten und Universitäten sieht das Skript für das Objektdiagramm folgendermaßen aus:

Listing 2.2: SOIL-Skript für Studenten und Universität

```
1 !create hans:Student
2 !hans.name:='Hans'
3 !hans.alter:=18
4 !hans.semester:= 1
5 !create gretel:Student
6 !gretel.name:='Gretel'
7 !gretel.alter:=22
8 !gretel.semester:= 3
9 !new Universitaet('uniBremen')
10 !uniBremen.name:='Universitaet Bremen'
11 !uniBremen.ort:='Bremen'
12 !insert (hans,uniBremen) into besucht
13 !insert (gretel,uniBremen) into besucht
```

Die Befehle entsprechen denen, die über die Konsolenschnittstelle von USE eingegeben werden können. So können Objekte mit *!create* erstellt werden und ihre Attribute nach dem Schema *Objekt.Attribut:=Wert* gesetzt werden. Assoziationen lassen sich mit Befehlen nach dem Muster *!insert (Objekt1, Objekt2) into Assoziationsname* befüllen.

2.5 Modelvalidator

Der Modelvalidator(MV) ist ein von Hendrik Reitmann 2012 in seiner Masterarbeit[18] entwickeltes Plugin für USE, das es u. a. ermöglicht, automatisch Objektdiagramme zu einem Klassendiagramm generieren zu lassen. Somit lässt sich mit Hilfe des Modelvalidators feststellen, ob es überhaupt ein Objektdiagramm gibt, das den aktuellen Vorgaben des Klassendiagramms und den dazugehörigen OCL-Constraints entspricht. Damit wird die Konsistenz des Modelles bezüglich Invarianten, die sich widersprechen und dadurch keine Instanziierung des Modells möglich ist, überprüft. Dabei ist es möglich, den Suchraum, die Eigenschaften des Objektdiagrammes, einzuschränken, um aussagekräftige Ergebnisse zu generieren. Danach kann man dann nach bestimmten Objektdiagrammen suchen.[19]

Der Modelvalidator bricht das Problem, ein Objektdiagramm zu finden, auf das Erfüllbarkeitsproblem der Aussagenlogik (SAT) herunter. Dieses Problem ist NP-vollständig, es ist also kein Algorithmus bekannt, der das Problem in Polynomialzeit lösen kann. Somit gibt es unterschiedlich performante Implementierungen. Der Modelvalidator kann auf verschiedene Implementierungen zurückgreifen. Mehr dazu im Kapitel 4.3.1, in dem einige Konfigurationsmöglichkeiten des Modelvalidators vorgestellt werden.[20]

Die aktuelle Version[21] des Modelvalidators ist 4.2, jedoch wurde in dieser Arbeit die Version 4.1 verwendet, die sich jedoch hinsichtlich der Funktionalität nicht von der aktuellsten unterscheidet. Die Plugin-Datei, in diesem Fall $ModelValidatorPlugin - 4.1 - r1.jar$, muss sich im Ordner $lib\backslash plugins$ des USE-Verzeichnis befinden.

[18] Vgl. Reitmann (2012).
[19] Vgl. Gogolla/Hilken (2016).
[20] Vgl. Kuhlmann/Gogolla (2012).
[21] https://sourceforge.net/projects/useocl/files/Plugins/ModelValidator/

2.5.1 Einsatzmöglichkeiten des Modelvalidators

Der Modelvalidator kann nicht nur dazu benutzt werden, die Konsistenz des Modells zu überprüfen, sondern ist darüber hinaus für einige weitere Anwendungsfälle geeignet. Mit dem Modelvalidator kann auch die Invariantenabhängigkeit überprüft werden. Invarianten sind voneinander abhängig, wenn es keinen Fall gibt, in dem eine der beiden Invarianten wahr und die andere falsch ist. Somit kann durch den Modelvalidator die Abhängigkeit der Invarianten bestimmt werden, indem eine der zu überprüfenden Invarianten deaktiviert wird. Wenn es dann kein Objektdiagramm gibt, das nur eine Invariante erfüllt, die andere aber nicht, hat man festgestellt, dass die beiden Invarianten von einander abhängig sind. Der Modelvalidator kann dies mit der Funktion *invIndep* automatisiert für alle geladenen Invarianten durchführen. Der Modelvalidator stellt damit eine Hilfe dar, eine minimale Anzahl von Invarianten, d. h. ein minimales Modell zu finden.[22]

Damit einhergehend kann auch untersucht werden, ob eine Invariante eine Implikation des Modells ist. Implikation bedeutetet, dass es keinen Unterschied macht, ob die Invariante geladen ist oder nicht, da die enthaltenen Eigenschaften bereits vom Modell implizit gefordert werden. Dies kann überprüft werden, indem die in Frage stehende Invariante negiert und nach einer Instanziierung gesucht wird. Wenn es keinen Zustand gibt, der dies erfüllt, ist bewiesen, dass die Invarianten eine Implikation des Systems sind.[23]

Eine weitere Funktionalität ist die Partial Solution Completion, die es ermöglicht, ein partiell vorgegebenes Objektdiagramm zu vervollständigen. Diese kommt im Abschnitt 4.4 zum Einsatz, in dem ein Streckennetz vorgegeben und durch die Partial Solution Completion vervollständigt wird.[24]

[22] Gogolla/Hilken (2015).
[23] Vgl. Gogolla/Hilken (2016).
[24] Vgl. Kuhlmann/Hamann/Gogolla (2011).

2.5.2 Benutzung des Modelvalidators

Die *.properties*-Dateien enthalten Spezifizierungen, die den Validierungsprozess beschleunigen, da der Modelvalidator nur Objektdiagramme suchen muss, die den Einschränkungen gerecht werden. Wenn dem MV genaue Grenzen vorgegeben werden, wie viele Objekte eines bestimmten Typs er erzeugen soll, beschleunigt dies den Auswertungsprozess ungemein.

In der Konfigurationsdatei kann zu jedem konfigurierbaren Basistyp (Integer, Real, String), jeder Klasse, zu jedem Attribut einer Klasse und zu Assoziationen jeweils eine untere und obere Grenze angegeben werden, die festlegt, wie viele Instanzen jeweils im Objektdiagramm vorkommen sollen. Außerdem ist es möglich, eine Menge von möglichen Ausprägungen des Feldes als Set anzugeben.

Dabei wird der Name der Klasse und *min* bzw. *max* durch Unterstrich getrennt, z. B. *Student_min = 2* zur Angabe der minimalen Anzahl an Studenten. Um die Attributsausprägungen dieser Objekte genauer zu spezifizieren, kann z. B. der Name durch die Angabe einer Menge in der Form $Student_name = Set\{'Hans', 'Peter'\}$ eingeschränkt werden. Eine besondere Bedeutung hat der Wert -1. Wenn ein Attribut die Minimalanzahl -1 hat, bedeutet dies, dass das Attribut einen definierten Wert annehmen muss. Die Maximalanzahl -1 sagt aus, dass nach oben keine Grenze besteht. Wenn keine Angabe zu einem Element gemacht wird, ist dies gleichbedeutend mit -1.

Des weiteren ist es möglich, innerhalb der Konfigurationsdatei eine oder mehrere Invarianten zu deaktivieren, um herauszufinden, ob diese verantwortlich dafür sind, dass der Modelvalidator kein zulässiges Objektdiagramm findet.[25][26]

[25] Vgl. Reitmann (2012).
[26] Vgl. Gogolla et al. (2015a).

2.6 Filmstrip-Modell

Der Begriff Filmstrip beschreibt eine Aneinanderreihung von Momentaufnahmen, auf englisch Snapshots genannt, und dient als Analogie für eine Sequenz von Zuständen eines Systems.

Bei der Transformation in das Filmstrip-Modell wird ein Modell, in dem das Verhalten mit Pre- und Postconditons sowie Invarianten modelliert ist, zu einem Modell, das nur mit strukturellen Invarianten beschrieben und dabei inhaltlich nicht verändert ist und die selbe Aussagekraft hat. Die Semantik des Modells soll nach Transformation die gleiche sein. Eine Reihe von Systemzuständen wird zu einem einzelnen Zustand im Filmstrip-Modell, in dem jeder Zustand im Application-Modell (Das ursprüngliche durch Klassendiagramm und OCL Constraints beschriebene Modell) durch einen Snapshot repräsentiert wird.

Im Application-Modell finden Zustandsübergänge mit dazwischenliegenden Operationsaufrufen statt, während im Filmstrip-Modell explizite Objekte die Zustände und Operationsaufrufe darstellen. Es wird also eine Folge von Systemzuständen im Application-Modell durch ein einziges Objektdiagramm im Filmstrip-Modell abgebildet. Somit wird eine Menge von Objektdiagrammen und Operationsaufrufen in einem einzigen Objektdiagramm dargestellt.[27] Objektdiagramme des Filmstrip-Modell sind identisch mit Sequenzdiagrammen des Application-Modells, d. h. in einem einzelnen Objektdiagramm sind alle Operationsaufrufe abgebildet. Durch das Filmstrip-Modell können somit Objektdiagramme des gesamten Zustandsverlaufs durch den Modelvalidator erzeugt werden.

Jeder Systemzustand wird durch ein Objekt Snapshot repräsentiert. Jeder Snapshot hat genau einen Vor- und einen Nachfolger, da es sich quasi um einen linearen Zeitstrahl handelt. Die abstrakte Klasse Opc (Operation Call) stellt die Operationsaufrufe zwischen den Snapshots dar, also die Zustandsübergänge. Aus den Operationen der Klassen sind nun

[27] Vgl. Gogolla et al. (2014).

eigene Klassen geworden, die von der OpC-Oberklasse erben. Die Pre- und Postconditions der Operationen werden dabei zu Invarianten der entsprechenden OperationCall-Klasse. Die Aggregation *PredSucc* verbindet zwei Objekte, die Vorgänger bzw. Nachfolger voneinander sind. Dies ist als Aggregation implementiert, um Zyklen zu verhindern. PredSucc emuliert das *@pre*, das im Application-Modell nur in Postconditions möglich ist, indem der Predecessor, also der Vorgängerzustand, nach dem entsprechenden Wert überprüft wird. Die ternäre Assoziation *Filmstrip* verbindet zwei Snapshots und einen OpC miteinander, sie zeigt, welcher Snapshot welchen Nachfolgezustand hat und welche Operation ausgeführt wurde, um diesen zu erreichen. Das Interface OpC enthält das Attribut *aSelf*, also das Objekt, auf dem die Operation aufgerufen wurde. Das *self* aus dem ursprünglichen Modell wird zu *aSelf*, damit die Referenzierung der Objekte stimmt.[28]

Die untenstehenden Grafik 2.6 versucht die Transformation kompakt in einem Bild darzustellen. Die grau unterlegten Elemente sind das ursprüngliche Application-Modell einer Bibliothek. Anhand der Grafik ist nun zu sehen, wie aus dem Klassendiagramm und dem Sequenzdiagramm ein Filmstrip-Klassendiagramm und ein Objektdiagramm wird. Es ist jedoch zu beachten, dass in der abgebildeten Grafik das Filmstrip-Modell etwas anderes realisiert wurde als beim Filmstrip-Transformations-Plugin. Die reflexive Assoziation am Snapshot besteht ebenso, jedoch wurde keine ternäre Assoziation verwendet, sondern eine einfache Assoziation zwischen dem Ausgangszustand und der zustandsüberführenden Klasse *OperationCall*.[29]

[28] Vgl. Hilken/Hamann/Gogolla (2014).
[29] Vgl. Gogolla/Hilken/Desai (2016).

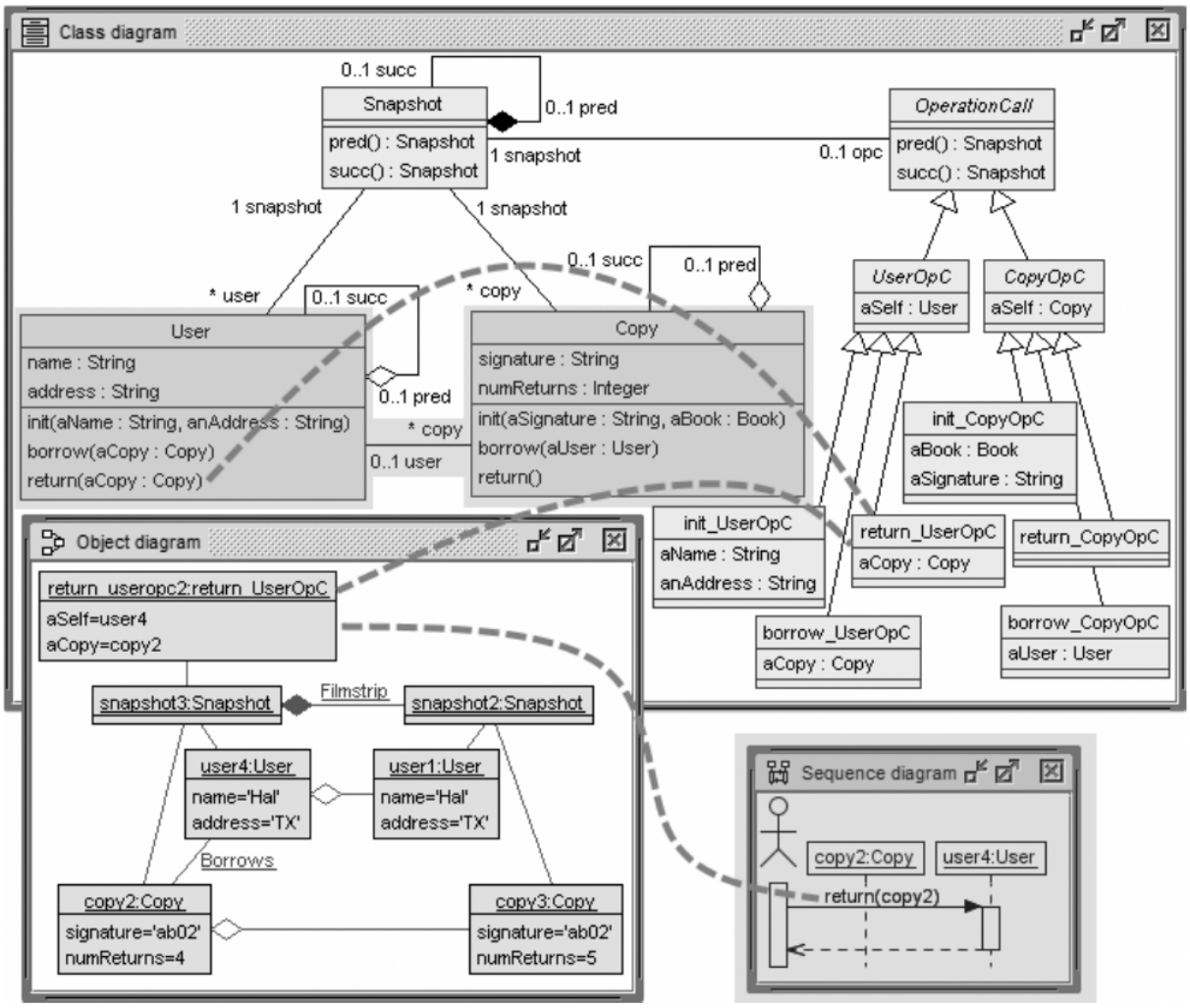


Abbildung 2.6: Transformation in das Filmstrip-Modell

In der AG Datenbanksysteme an der Universität Bremen, beginnend mit der Diplomarbeit[30] von Frank Hilken, wurde ein USE-Plugin entwickelt, das die Transformation automatisiert vornimmt. Es liegt aktuell in der Version 3[31] vor, die auch im Abschnitt 4.2 verwendet wurde.

[30] Vgl. Hilken (2013).

[31] https://sourceforge.net/projects/useocl/files/Plugins/FilmstripTransformation/

3 Beispielmodell: Toll Collect

Bei dem in dieser Arbeit verwendeten Modell handelt es sich um eine Simplifizierung von Toll Collect[32], dem in Deutschland eingesetzten System zur Erhebung und Kontrolle von LKW-Maut.[33]

Das Modell ist in einer .use-Datei (vollständig im Abschnitt I.1 des Anhangs) mit UML und OCL beschrieben. Dort enthalten sind alle Klassen, Assoziationen, Operationen und Constraints. Das Klassendiagramm in der untenstehenden Abbildung 3.1 zeigt die zwei zentralen Klassen *Point* und *Truck*. Ein Point stellt einen Punkt im Streckennetz dar. Die reflexive Assoziation *Connection* hat an den Assoziationsenden die Rollennamen *north* und *south*. Das bedeutet, dass ein Punkt mit anderen Punkten in Beziehung stehen kann und dabei nördlich oder südlich vom anderen Punkt liegt. Die Klasse *Truck* beschreibt einen Truck, der der einzige Akteur im System ist. Die *Current*-Assoziation gibt an, wo sich der Truck aktuell befindet.

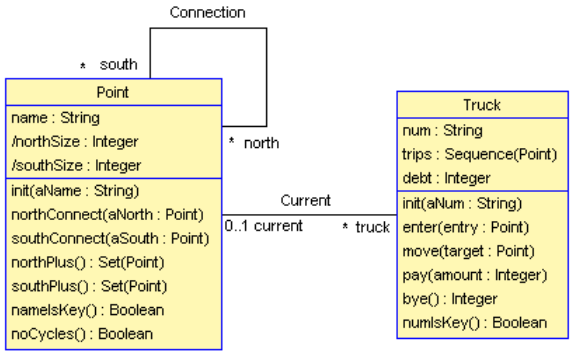


Abbildung 3.1: Ursprüngliches Klassendiagramm des Toll Collect-Modells

Das Listing 3.1 zeigt, wie die Klassenspezifikation des Point, aus dem auch der entsprechende Teil des Klassendiagramm entspringt, in USE aussieht. Das erste Attribut ist der *name* des Point, der den Datentyp String. Die abgeleiteten Attribute *northSize* und *southSize* berechnen jeweils für die entsprechende Himmelsrichtung, wie viele Punkte sich nördlich

[32] https://www.toll-collect.de/
[33] Vgl. Gogolla et al. (2015b).

bzw. südlich vom aktuellen Punkt befinden. Die *init*-Operation erhält als Parameter den gewünschten Namen und setzt das *name*-Attribut entsprechend. Die Operationen *northConnect* und *southConnect* sind Komfortfunktionen zum Herstellen der Assoziationen zwischen den Punkten.

Listing 3.1: Klasse *Point* in USE

```
1 class Point
2 attributes
3   name:String init: ''
4   northSize:Integer derived: self.northPlus()->size()
5   southSize:Integer derived: self.southPlus()->size()
6 operations
7   init(aName:String)
8     begin self.name:=aName end
9   northConnect(aNorth:Point)
10    begin insert (aNorth,self) into Connection end
11  southConnect(aSouth:Point)
12    begin insert (self,aSouth) into Connection end
```

Connection ist eine reflexive Assoziation, die Objekte der selben Klasse verbindet. Die verbundenen Punkte erhalten dabei die Rollennamen *north* und *south*. Die Multiplizitäten geben an, dass es beliebig viele Verbindungen zwischen Punkten geben darf. Die Assoziation *Connection* ist in USE so definiert:

Listing 3.2: *Connection* in USE

```
1 association Connection between
2   Point[0..*] role north
3   Point[0..*] role south
4 end
```

In jedem Truck-Objekt ist eine Identifikationsnummer *num*, der aktuelle Schuldenstand *debt* und die Sequenz der bisher besuchten Punkte *trips* gespeichert. Ein Truck hat fünf verschiedene Operationen. Die Operationen sind *init* zur Initialisierung, *enter* zum Betreten eines Point, *move*, um zu einem anderen Punkt zu gelangen, *pay*, um die Schuld zu bezahlen und *bye*, um das System zu verlassen. Die Definition des Trucks und dessen Operationen sind in Listing 3.3 zu sehen.

Listing 3.3: *Truck* in USE

```
1 class Truck
2 attributes
3   num:String            init: ''
4   debt:Integer          init: 0
5 operations
6   init(aNum:String)
7     begin self.num:=aNum end
8   enter(entry:Point)
9     begin insert (self,entry) into Current; self.debt:=1;
10    end
11  move(target:Point)
12    begin
13    self.debt:=self.debt+1; delete (self,self.current) from Current;
14    insert (self,target) into Current end
15  pay(amount:Integer)
16    begin self.debt:=self.debt-amount end
17  bye():Integer
18    begin delete (self,self.current) from Current;
19    result:=self.debt.abs(); self.debt:=0 end
```

Die ungerichtete Assoziation *Current* stellt die Verbindung zwischen den zwei Klassen Point und Truck her. Sie hat am Assoziationsende *current*, also am akutellen Point, die Multiplizität 0..1 und am anderen Assoziationsende *truck* die Multiplizität *. Somit hat jeder Truck genau einen oder keinen aktuellen Standpunkt, jeder Punkt kann beliebig viele Trucks haben. Die Rollennamen wurden explizit definiert, wäre dies nicht der Fall, würden automatisch kleingeschrieben die Namen der Klassen genommen werden. Die Definitionen der Assoziation *Current* sieht in USE wie folgt aus:

Listing 3.4: *Current* in USE

```
1 association Current between
2   Truck[0..*] role truck
3   Point[0..1] role current
4 end
```

Eine andere Möglichkeit, das Modell in USE zu betrachten, ist das Zustandsdiagramm. Wie in Abbildung 3.2 zu sehen ist, zeigt die state machine der Truck-Klasse den Lebenszyklus

eines Objektes des Typs Truck. Nach dem Initialzustand *born* gibt es die Zustände *debt* und *nodebt*. Die Operation *enter* ist der Zustandsübergang von *nodebt* zu *debt*. Die Operationen *move* und *pay* verändern den Zustand nicht, somit bleibt der Truck bei Ausführung der Operationen im Zustand debt. Durch die Operation *bye* kann der Zustand von *debt* in *nodebt* gewechselt werden.

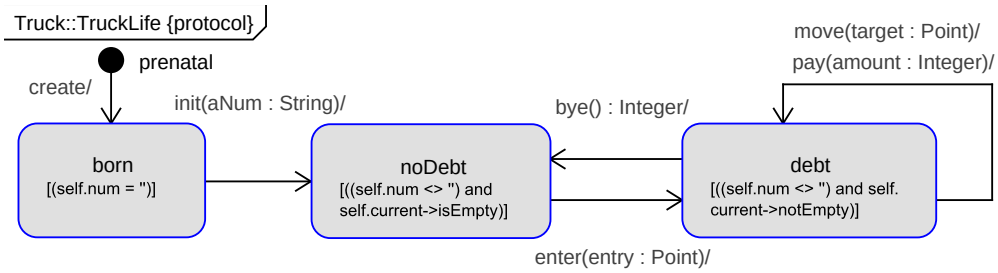


Abbildung 3.2: Zustandsmaschinendiagramm des Trucks

Das Zustandsdiagramm des Punktes, zu sehen in Abbildung 3.3, ist weniger interessant. Es unterscheidet zwischen dem Zustand direkt nach Erzeugung des Point *born* und dem Zustand nach der Initialisierung *growing*. Der Zustand *growing* kann dann nicht mehr verlassen werden, da die Operationsaufrufe von *northConnect* und *southConnect* wieder zum Zustand *growing* führen.

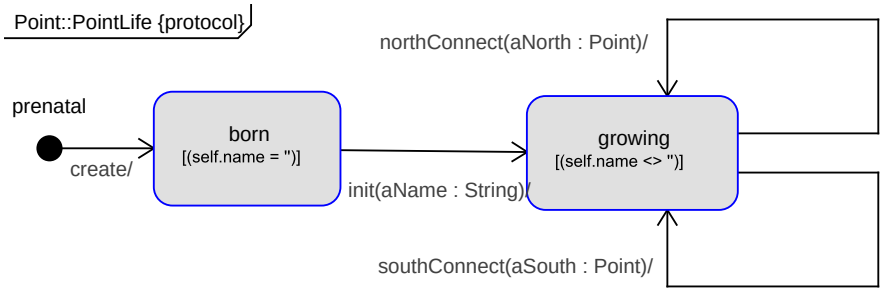


Abbildung 3.3: Zustandsmaschinendiagramm des Points

Des Weiteren sind in der USE-Spezifikation Constraints, also die Invarianten, sowie Pre- und Postcondtions des Systems enthalten. Jede einzelne Pre- und Postcondition des Systems zu beschreiben und zu erklären würde den Rahmen sprengen, daher werden hier nur

beispielhaft einige vorgestellt, um einen Eindruck zu vermitteln, was für Einschränkungen in Pre- und Postcondtions ausgedrückt werden können. Die Constraints beschreiben viele verschiedene Eigenschaften. Einige sind eher triviale, allgemeingültige Constraints, die so oder so ähnlich auch in anderen Systemen auftreten könnten. Solche Constraints, wie *freshPoint*, überprüfen beispielsweise, ob eine Eingabe sinnvoll ist, oder ob ein Attribut vor einer Operation *empty* und danach nicht *empty* ist. Andere, wie *targetreachable*, sind etwas komplexer und anwendungsfallspezifischer.

Die Precondition *freshPoint* (Listing 3.5) überprüft, ob dem Punkt bereits ein Name zugewiesen wurde, d. h. nur wenn es sich um einen unangetasteten Punkt handelt, dieser also den leeren String als Namen hat. Außerdem soll sich der Punkt weder in einer nördlichen noch südlichen Relation mit einem anderen Punkt befinden. Nur wenn beide Bedingungen erfüllt sind, kann *init* ausgeführt werden.

Listing 3.5: *freshPoint*-Precondition in USE

```
1 pre freshPoint:
2   self.name='' and self.north->isEmpty and self.south->isEmpty
```

Die Precondtion *targetreachable* (Listing 3.6) der Operation *move* sorgt dafür, dass sich der Truck nur zu den nördlich oder südlich direkt angrenzenden, benachbarten Punkten bewegen darf, d. h. in einem Straßennetz, in dem Bremen südlich von Hamburg und München südlich von Bremen liegt, wäre ein Übergang von Hamburg nach München nicht zulässig, ohne zwischendurch in Bremen gewesen zu sein.

Listing 3.6: *targetReachable*-Precondition in USE

```
1 pre targetReachable:
2   self.current.north->union(self.current.south)->includes(target)
```

Eine Postcondition von *move* ist *debtIncresed* (Listing 3.7), die überprüft, ob nach der Operationsausführung der Schuldenstand entsprechend um 1 erhöht wurde.

Listing 3.7: *debtIncreased*-Postcondition in USE

```
1 post debtIncreased:
2   self.debt@pre+1=self.debt
```

4 Durchführung der Fallstudie

4.1 Einführung von Frameconditions

Vor der Betrachtung der Testfälle müssen zunächst Frameconditions eingeführt werden. Diese sollen die Erzeugung unzulässiger Systemzustände im Filmstrip-Modell verhindern. Framecondtitions sind Postconditions, das heißt, sie sollen festlegen, was nach einer Operationsausführung nicht verändert sein soll. Da beim Generieren der Snapshots durch den Modelvalidator einiges vom Zufall abhängt, könnte es ansonsten sein, dass sich z. B. das Geschlecht einer Person durch einen Operationsaufruf verändert. Das Geschlecht einer Person sollte jedoch in allen Nachfolgezuständen gleich bleiben, da sonst die Integrität des Modells verletzt ist.[34]

Jede Operation benötigt so viele Frameconditions wie es Klassen gibt. Wir unterscheiden drei Arten von Frameconditions. Diese sind nach einem bestimmten Muster aufgebaut. Die Benennung folgt ebenfalls einer Systematik.

Die erste Condition betrifft die Objekte der Klasse, auf der die Operation nicht ausgeführt wird. Bei einer Operation auf einem Truck, z. B. enter, wird zunächst die Condition bezüglich der Points definiert. Wenn keiner der Punkte betroffen sein soll, heißt die Condition pointUnchanged.

Die zweite Art von Condition lässt alle Objekte der Klasse unverändert mit der Ausnahme von einem bestimmten Objekt. Wenn nur ein bestimmter Punkt geändert werden soll, heißt sie pointUnchanchedExcept.

Im letzten Fall, wird die Ausnahme für eine Menge von Objekten einer Klasse gemacht. Wenn eine bestimmte Menge von Punkten verändert werden soll, heißt sie pointUnchanchedExceptSet.

[34] Vgl. Gogolla et al. (2014).

Bei den Point-Operationen ist die Benennung analog dazu, also z. B. truckUnchanged, wenn kein Truck verändert werden soll. Der Aufruf *allInstances* gibt alle dem System bekannten Objekte des angegebenen Typs zurück. Die Notation *@pre* besagt, dass der Zustand des entsprechenden Objektes vor der Ausführung der Operation gemeint ist. Das Gegenstück *@post* ist implizit und muss somit nicht hingeschrieben werden.

4.1.1 Point-Operationen

Die Tabelle 4.1 bietet einen Überblick über die verwendeten Arten von Frameconditions für die Point-Operationen.

Operation	**Point::init**	**Point:: south-Connect**	**Point:: north-Connect**
TruckUnchanged	√	√	√
TruckUnchangedExcept	×	×	×
TruckUnchangedExceptSet	×	×	×
PointUnchanged	×	×	×
PointUnchangedExcept	√	×	×
PointUnchangedExceptSet	×	√	√

Tabelle 4.1: Übersicht über Frameconditions des Point

4.1.1.1 Point::init

Im Listing 4.1 sind die Frameconditions für die *init*-Funktion des Points zu sehen. Mit *truckUnchanged* wird verhindert, dass irgendein Truck durch die Initialisierung des Points verändert wird. Durch *pointUnchangedExcept* wird sichergestellt, dass nur der betroffene Point verändert wird.

Listing 4.1: Frameconditions für Operation *init* des Points

```
1 post truckUnchanged:
2   Truck.allInstances@pre=Truck.allInstances and
3   Truck.allInstances ->forAll(t| (t.num@pre=t.num) and
4   (t.debt@pre=t.debt) and
5   (t.current@pre=t.current))
6 post pointUnchangedExcept: let x= self in
7   Point.allInstances@pre = Point.allInstances and
8   Point.allInstances-> forAll (p| (p<> x implies p.name@pre=p.name) and
9   (p<>x implies p.truck@pre=p.truck))
```

4.1.1.2 Point::northConnect

Das Listing 4.2 zeigt die Frameconditions für die *northConnect*-Funktion des Points. In *truckUnchanged* wird verhindert, dass irgendein Truck durch das Herstellen einer Verbindung zu einem anderen Punkt verändert wird. Durch *pointUnchangedExceptSet* wird sichergestellt, dass nur die zwei betroffenen, der nördliche und der südliche Point, verändert werden. Bei den beiden beteiligten Points soll sich nur die Assoziation *Connection*, also die Assoziationsenden *north* und *south* ändern.

Listing 4.2: Frameconditions für Operation *northConnect* des Points

```
1 post TruckUnchanged:
2   Truck.allInstances@pre=Truck.allInstances and
3   Truck.allInstances->forAll(t| (t.num@pre=t.num) and
4   (t.debt@pre=t.debt) and
5   (t.current@pre=t.current))
6 post PointUnchangedExceptSet: let x=Set{aNorth, self} in
7   Point.allInstances@pre=Point.allInstances and
8   Point.allInstances->forAll(p| (p.name@pre=p.name) and
9   (p.truck@pre=p.truck) and
10  (x->excludes(p) implies p.north@pre=p.north) and
11  (x->excludes(p) implies p.south@pre=p.south))
```

4.1.1.3 Point::southConnect

Die Frameconditions für die *southConnect*-Funktion des Points (Listing 4.3) sind strukturell identisch mit den Frameconditions zu *northConnect*. Lediglich die Bezeichnung des Eingabeparameters lautet nun *aSouth* an Stelle von *aNorth*.

Listing 4.3: Frameconditions für Operation *southConnect* des Points

```
1 post TruckUnchanged:
2   Truck.allInstances@pre=Truck.allInstances and
3   Truck.allInstances->forAll(t|(t.num@pre=t.num) and
4   (t.debt@pre=t.debt) and
5   (t.current@pre=t.current))
6 post PointUnchangedExceptSet: let x=Set{aSouth, self} in
7   Point.allInstances@pre=Point.allInstances and
8   Point.allInstances->forAll(p| (p.name@pre=p.name) and
9   (p.truck@pre=p.truck) and
10  (x->excludes(p) implies p.north@pre=p.north) and
11  (x->excludes(p) implies p.south@pre=p.south))
```

4.1.2 Truck-Operationen

Die Tabelle 4.2 gibt einen Überblick über die für die Truck-Operationen verwendeten Arten von Frameconditions.

Operationen	**Truck:: init**	**Truck:: enter**	**Truck:: move**	**Truck:: pay**	**Truck:: bye**
TruckUnchanged	×	×	×	×	×
TruckUnchangedExcept	√	√	√	√	√
TruckUnchangedExceptSet	×	×	×	×	×
PointUnchanged	√	×	×	√	×
PointUnchangedExcept	×	√	×	×	√
PointUnchangedExceptSet	×	×	√	×	×

Tabelle 4.2: Übersicht über Framecondtions des Trucks

4.1.2.1 Truck::init

Im Listing 4.4 sind die Frameconditions für die *init*-Funktion des Trucks zu sehen. In *PointUnchanged* wird verhindert, dass irgendein Point durch die Initialisierung des Trucks verändert wird. Durch *truckUnchangedExcept* wird sichergestellt, dass nur der betroffene Truck verändert wird.

Listing 4.4: Frameconditions für Operation *init* des Trucks

```
1 post PointUnchanged:
2   Point.allInstances@pre=Point.allInstances and
3   Point.allInstances->forAll(p|(p.name@pre=p.name) and
4   (p.truck@pre=p.truck))
5 post truckUnchangedExcept: let x= self@pre in
6   Truck.allInstances@pre = Truck.allInstances and
7   Truck.allInstances-> forAll (t| (t<> x implies t.num@pre=t.num)
8   and (t.debt@pre=t.debt) and
9   (t<>x implies t.current@pre=t.current))
```

4.1.2.2 Truck::enter

Die Frameconditions für die *enter*-Funktion des Trucks sieht man im Listing 4.5. Durch *PointUnchangedExcept* wird verhindert, dass irgendein anderer Point als der zu betretende sich durch das *enter* des Trucks ändert. Durch *truckUnchangedExcept* wird sichergestellt, dass nur der betroffene Truck seine Position ändert.

Listing 4.5: Frameconditions für Operation *enter* des Trucks

```
1 post PointUnchangedExcept: let x=entry in
2   Point.allInstances@pre=Point.allInstances and
3   Point.allInstances->forAll(p| (p.name@pre=p.name) and
4   (p.north@pre=p.north) and
5   (p.south@pre=p.south) and
6   (p<>x implies p.truck@pre=p.truck))
7 post truckUnchangedExcept: let x= self in
8   Truck.allInstances@pre = Truck.allInstances and
9   Truck.allInstances-> forAll (t| (t.num@pre=t.num) and
10  (t<>x implies t.debt@pre=t.debt) and (
11  t<>x implies t.current@pre=t.current))
```

4.1.2.3 Truck::move

Die Frameconditions für die *move*-Funktion des Trucks sind im Listing 4.6 dargestellt. Durch *PointUnchangedExceptSet* wird verhindert, dass sich irgendein anderer Point als der zu betretende und der verlassene durch das *move* des Trucks verändert. Durch *truckUnchangedExcept* wird sichergestellt, dass nur der betroffene Truck seine Position ändert.

Listing 4.6: Frameconditions für Operation *move* des Trucks

```
1 post PointUnchangedExceptSet: let x=Set{target, self.current@pre} in
2   Point.allInstances@pre=Point.allInstances and
3   Point.allInstances->forAll(p| (p.name@pre=p.name) and
4   (p.north@pre=p.north) and
5   (p.south@pre=p.south) and
6   (x->excludes(p) implies p.truck@pre=p.truck))
7 post truckUnchangedExcept: let x= self in
8   Truck.allInstances@pre = Truck.allInstances and
9   Truck.allInstances-> forAll (t| (t.num@pre=t.num) and
10  (t<>x implies t.debt@pre=t.debt) and
11  (t<>x implies t.current@pre=t.current))
```

4.1.2.4 Truck::pay

Im Listing 4.7 sind die Frameconditions für die *pay*-Funktion des Trucks zu sehen. Durch *PointUnchanged* wird verhindert, dass irgendein anderer Point durch das *pay* des Trucks verändert wird. Durch *truckUnchangedExcept* wird sichergestellt, dass nur der betroffene Truck seine *debt* ändert.

Listing 4.7: Frameconditions für Operation *pay* des Trucks

```
1 post PointUnchanged:
2   Point.allInstances@pre=Point.allInstances and
3   Point.allInstances->forAll(p|(p.name@pre=p.name) and
4   (p.truck@pre=p.truck) and
5   (p.north@pre=p.north) and
6   (p.south@pre=p.south))
7 post truckUnchangedExcept: let x = self in
8   Truck.allInstances@pre=Truck.allInstances and
9   Truck.allInstances->forAll(t|
10    (t.num@pre=t.num) and (t<>x implies t.debt@pre=t.debt) and
11    (t.current@pre=t.current))
```

4.1.2.5 Truck::bye

Die Frameconditions für die *bye*-Funktion (Listing 4.8) des Trucks sehen folgendermaßen aus: Durch *PointUnchangedExcept* wird verhindert, dass irgendein anderer Point, außer dem, der vom Truck verlassen wird, durch das *bye* des Trucks verändert wird. Durch *truckUnchangedExcept* wird sichergestellt, dass nur der betroffene Truck geändert wird.

Listing 4.8: Frameconditions für Operation *bye* des Trucks

```
1   post PointUnchangedExcept: let x=self.current@pre in
2   Point.allInstances@pre=Point.allInstances and
3   Point.allInstances->forAll(p| (p.name@pre=p.name) and
4   (p.north@pre=p.north) and
5   (p.south@pre=p.south) and
6   (p<>x implies p.truck@pre=p.truck))
7 post truckUnchangedExcept : let x = self in
8   Truck.allInstances@pre=Truck.allInstances and
9   Truck.allInstances->forAll(t|
10    (t.num@pre=t.num) and
11    (t<>x implies t.debt@pre=t.debt) and
12  (t<>x implies t.current@pre=t.current))
```

4.2 Transformation ins Filmstrip-Modell

Nachdem das Toll Collect-Modell nun um Frameconditions ergänzt wurde, wird im folgenden Abschnitt die Transformation des Application-Modells ins Filmstrip-Modell vorgenommen. Vor der Transformation des Toll Collect-Modells müssen zunächst die sequenzwertigen Attribute entfernt werden. Im Fall von Toll Collect war dies die abgeleitete Sequenz *trips*, die in den Objekten der Klasse Truck gespeichert wird. Abgeleitet bedeutet, dass das Attribut trips nicht anhand von durch den Benutzer eingegebenen Anwendungsdaten, sondern vom System durch die vorhandenen Daten bestimmt wird. Im Filmstrip-Modell lässt sich dieses abgeleitete Attribut nicht mehr bestimmen, da für jeden Zustand (Snapshot) neue Objekte erzeugt werden, die die Information nicht mehr gespeichert haben.[35] Ähnlich wie das Modelvalidator-Plugin, muss auch das Filmstrip-Plugin eingebunden werden. Die Plugin-Datei, in diesem Fall $use - filmstrip - v2.jar$, muss sich im Ordner $lib\backslash plugins$ des USE-Verzeichnis befinden. Bevor nun die Transformation durchgeführt werden kann, muss zunächst die Spezifikation des Modells in USE geöffnet sein (Listing 4.9). Über die GUI oder in der Konsole kann nun die *.use*-Spezifikationsdatei mit dem Befehl *open* die in das USE-System geladen werden.

Listing 4.9: Öffnen des Modells in USE

```
1 use> open tollcoll-sc-soil_ohne_trips_FS.use
2 compiling specification...
3 done
4 Model TollCollect_FS (15 classes, 9 associations, 78 invariants, 18 operations, 1
    pre-/postcondition, 0 state machines)
5 use>
```

Anschließend zeigt USE, siehe untenstehende Abbildung 4.1, das zu transformierende Klassendiagramm. Die Transformation lässt sich über die GUI des Plugins und über die Kommandozeile durchführen. In der GUI von USE im Reiter Plugins unter Filmstrip wird die Funktion „Transform Model into Filmstrip Model“ aufgerufen.

[35] Vgl. Hilken et al. (2015).

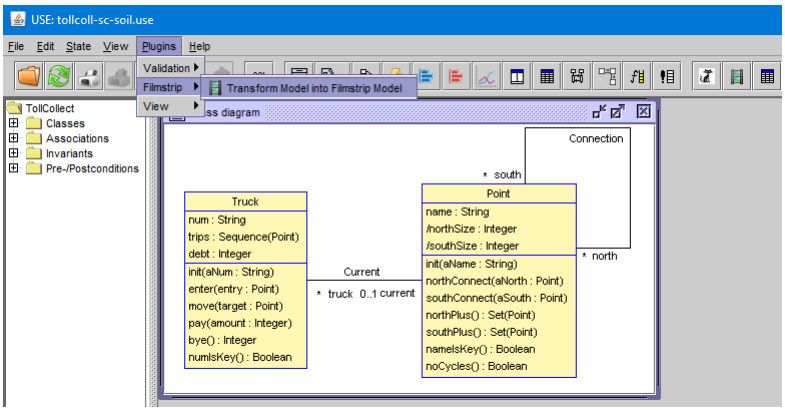


Abbildung 4.1: Klassendiagramm vor der Transformierung

Die Abbildung 4.2 zeigt das geöffnete Fenster mit den Einstellungsmöglichkeiten für die Transformation. Hier wird zuerst der Name des auszugebenden Modells und die Ziel-Datei angegeben. Außerdem muss im Dropdown-Menü der Punkt „Model validator compatible transformation“ ausgewählt werden, da das transformierte Modell sonst nicht mit dem Modelvalidator kompatibel ist. Dies liegt an einigen fortgeschrittenen Elementen der Transformation, die vom Modelvalidator (noch) nicht behandelt werden können. Das Modell kann jedoch auch unter Verwendung der kompatiblen Elementen vollständig umgewandelt werden. Die Auswahl der Option führt somit zu keinerlei Nachteilen oder zu einer Veränderung der Semantik des Systems. Im Normalfall zeigt die GUI dann eine Erfolgsmeldung.

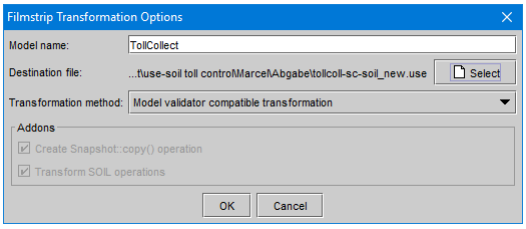


Abbildung 4.2: Optionen des Transformationstools

Im USE-Tool kann man nun das transformierte Modell öffnen. Das deutlich veränderte Klassendiagramm ist in der untenstehenden Abbildung 4.3 zu sehen.

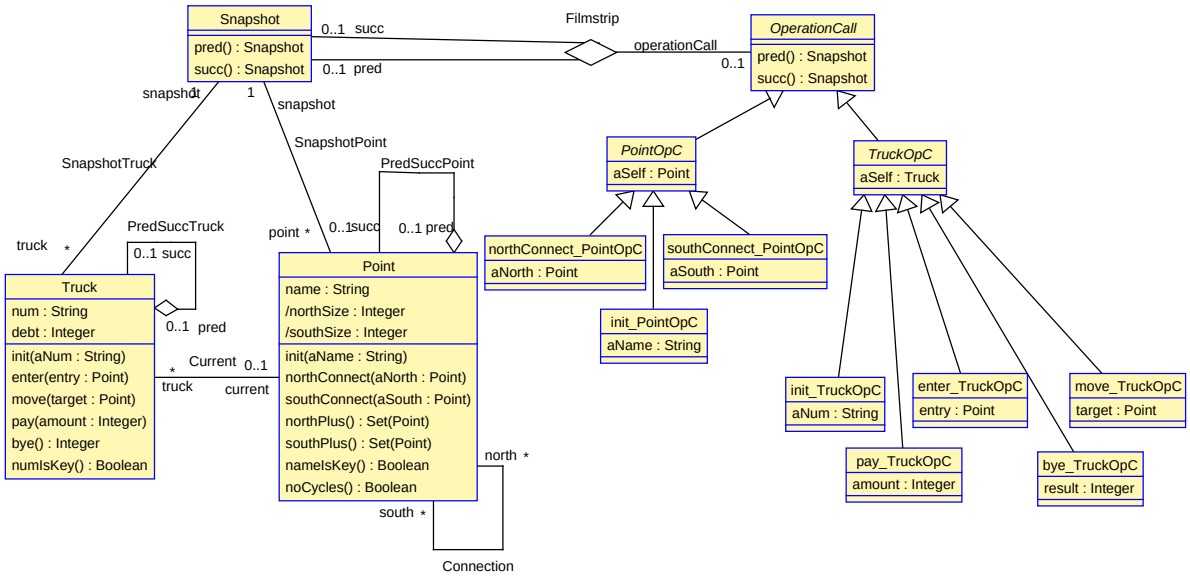


Abbildung 4.3: Neues Klassendiagramm

Im Abschnitt 2.6 wurden bereits einige Details des Transformationsprozess beschrieben. Das ursprüngliche Modell mit den Klassen *Truck* und *Point* sowie den Assoziationen *Current* und *Connection* ist nur noch am linken Rand der Abbildung 4.3 zu erkennen. Diese wurden durch die reflexive Assoziation *PredSuccTruck* respektive *PredSuccPoint* ergänzt, welche die Vorgänger-Nachfolger-Beziehung zwischen den Objekten der verschiedenen Zustände darstellen. Außerdem neu hinzugekommen ist die *Snapshot*-Klasse, welche den Objekte durch die SnapshotTruck- und SnapshotPoint-Assoziationen einen Zustand zuordnet. Zwei Snapshots sind immer durch die ternäre Assoziation *Filmstrip* verbunden, an der auch die Klasse *OperationCall* hängt. Die OprationCall-Klasse ist abstrakt und hat als erbende Klassen *TruckOpc* und *PointOpc* Klassen, welche alle Operationsaufrufe der jeweiligen Klassen unter sich vereinen. Die Operationen werden nun durch Klassen repräsentiert. Für den Truck sind dies die Klassen $init_TruckOpC$, $bye_TruckOpC$, $enter_TruckOpC$, $move_TruckOpC$, $pay_TruckOpC$ und $bye_TruckOpC$. Für den Point gibt es die drei „Operationsaufruf-Klassen“ $init_PointOpC$, $southConnect_PointOpC$ und $northConnect_PointOpC$.

4.3 Testfälle

Im folgenden Kapitel werden nun mit Hilfe des Modelvalidators Objektdiagramme im Filmstrip-Modell erzeugt, die verschiedene Zustände des Systems abbilden.

Die Tabelle 4.3 bietet einen Überblick über die Testfälle der Fallstudie. Es sind für die folgenden fünf Szenarien die Konfigurationen mit allen Objekten und Operationsaufrufen angegeben. Die verwendeten Konfigurationsdateien befinden sich vollständig im Abschnitt II des Anhangs.

Testfälle	**Truck init-Ops**	**Point init-Ops**	**Alle init-Ops**	**Alle Truck-Ops**	**Alle Ops**
Objekte	2 Points, 1 Truck	2 Points, 1 Truck	2 Points, 1 Truck	2 Points, 1 Truck	2 Points, 1 Truck
Snapshot	3..3	4..4	6..6	6..6	9..9
Filmstrip	2..2	3..3	5..5	5..5	8..8
Truck	3..3	4..4	6..6	6..6	9..9
Point	6..6	8..8	12..12	12..12	18..18
Point::init	0..0	2..2	1..1	0..0	1..1
Point::northConnect	0..0	0..0	0..0	0..0	0..0
Point::southConnect	0..0	1..1	1..1	0..0	1..1
Truck::init	1..1	0..0	1..1	1..1	1..1
Truck::enter	1..1	0..0	1..1	1..1	1..1
Truck::move	0..0	0..0	0..0	1..1	1..1
Truck::pay	0..0	0..0	0..0	1..1	1..1

Tabelle 4.3: Übersicht über alle Konfigurationen der Fallstudie

Es stellt sich zunächst die Frage, ob das transformierte Modell überhaupt instantiierbar ist oder ob sich etwa die neu dazugekommenen Constraints oder andere Eigenschaften des Modell, wie die Einschränkungen durch Multiplizitäten, widersprechen. Wenn der Modelvalidator eine Lösung findet, besteht die Möglichkeit, das Modell auf Fehler zu prüfen, da anhand der Objektdiagramme erkennbar wird, ob ein unzulässiges Verhalten vorliegt. So kann man feststellen, ob die vorhandenen Invarianten bereits für das gewünschte Verhalten sorgen oder ob es doch noch Verhalten gibt, das nicht den Erwartungen entspricht.

Die Testfälle sind unterschiedlich groß und haben eine variierende Anzahl von verschiedenen Operationsaufrufen. Ziel ist es herauszufinden, ob der Modelvalidator mit dem Modell in der Filmstrip-Version in verschiedenen Konstellationen mit unterschiedlichen Kombinationen von Operationsaufrufen immer sinnvolle Zustände liefert. Allerdings werden zur Übersichtlichkeit und Vergleichbarkeit der Testfälle hier nur zwei Punkte und ein Truck pro Zustand betrachtet, da dies die minimale Anzahl von Objekten pro Zustand ist, um alle Operationen ausführen zu können. Wären es noch weitere Punkte oder Trucks pro Zustand, würde die Wartezeit bei der Berechnung der Objektdiagramme deutlich ansteigen.

Die ersten drei Fälle befassen sich mit der Initialisierung. Im ersten Fall, im Abschnitt 4.3.1.1, wird ein Truck initialisiert. Im Abschnitt 4.3.1.2 werden zwei Punkte initialisiert. Als Drittes werden im Abschnitt 4.3.1.3 sowohl ein Truck als auch zwei Punkte initialisiert. Der vierte Fall, im Abschnitt 4.3.2, konzentriert sich auf die Operationen des Trucks. Als letztes sollen im Abschnitt 4.3.3 alle Operationen von Truck und Point ausgeführt werden.

4.3.1 Initialisierungsoperationen

Testfälle	**Initialisierung eines Trucks**	**Initialisierung zweier Punkte**	**Alle init-Operationen**
Objekte	2 Points, 1 Truck	2 Points, 1 Truck	2 Points, 1 Truck
Konfig	Snapshots: 3..3 Filmstrip: 2..2 Truck: 3..3 *init_TruckOpC*: 1..1 *enter_TruckOpC*: 1..1 *move_TruckOpC*: 0..0 *pay_TruckOpC*: 0..0 *bye_TruckOpC*: 0..0 SnapshotTruck: 3..3 PredSuccTruck: 2..2 Point: 6..6 SnapshotPoint: 6..6 PredSuccPoint: 4..4 *init_PointOpC*: 0..0 *southConnect_PointOpC*:0..0 *northConnect_PointOpC*:0..0 Connection: 0..-1 Current: 0..-1	Snapshots: 4..4 Filmstrip: 3..3 Truck: 4..4 *init_TruckOpC*: 0..0 *enter_TruckOpC*: 0..0 *move_TruckOpC*: 0..0 *pay_TruckOpC*: 0..0 *bye_TruckOpC*: 0..0 SnapshotTruck:4..4 PredSuccTruck: 3..3 Point: 8..8 SnapshotPoint:8..8 PredSuccPoint:6..6 *init_PointOpC*: 2..2 *southConnect_PointOpC*:1..1 *northConnect_PointOpC*:0..0 Connection: 0..-1 Current: 0..-1	Snapshots: 6..6 Filmstrip: 5..5 Truck: 6..6 *init_TruckOpC*: 1..1 *enter_TruckOpC*: 1..1 *move_TruckOpC*: 0..0 *pay_TruckOpC*: 0..0 *bye_TruckOpC*: 0..0 SnapshotTruck: 6..6 PredSuccTruck: 5..5 Point: 12..12 SnapshotPoint: 12..12 PredSuccPoint: 10..10 *init_PointOpC*: 2..2 *southConnect_PointOpC*:1..1 *northConnect_PointOpC*:0..0 Connection: 0..-1 Current: 0..-1
deaktivierte Invarianten	- -	*init_PointOpC* :: *post_truckUnchanged* *init_PointOpC* :: *post_pointUnchangedExcept*	*init_PointOpC* :: *post_truckUnchanged* *init_PointOpC* :: *post_pointUnchangedExcept*

Tabelle 4.4: Übersicht über Konfigurationen der Initialisierungstestfälle

Die Tabelle 4.4 bietet einen Überblick über die Konfigurationen der Testfälle mit den Initialisierungsoperationen. Im ersten Testfall wird der Truck initialisiert und betritt die Autobahn. Im 2. Testfall werden nur die zwei Punkte und die Verbindung zwischen ihnen hergestellt. Im 3. Fall werden alle Initialisierungsoperationen in einem ausgeführt, d. h. es werden zwei Punkte und ein Truck initialisiert und eine südliche Verbindung zwischen den Punkten etabliert. Leider mussten beim zweiten und dritten Testfall die Frameconditions der *init*-Operation des Point deaktiviert werden. Warum mit den aktivierten Invarianten kein Objektdiagramm gefunden wurde, ist unklar. Bei der *init*-Operation des Trucks tritt das Problem nicht auf, obwohl die Operationen die gleiche Funktionsweise haben und auch die Frameconditions quasi identisch sind.

Zunächst werden einige Einstellungen (Listing 4.10) am Modelvalidator vorgenommen. Es lässt sich der SatSolver einstellen, da die verschiedenen Solver unterschiedlich schnell sind. So ist zum Beispiel MiniSat besonders bei komplexeren Aufgaben schneller als der standardmäßig eingestellte DefaultSAT4J.

Listing 4.10: Einstellung des SatSolvers

```
1 use> mv -config SatSolver:=MiniSat
2 16:04:03 INFO  KodkodModelValidatorConfiguration: Set SatSolver to 'MiniSat'.
```

Im Listing 4.11 wird durch die Bitbreite die Ganzzahlendarstellung spezifiziert.

Listing 4.11: Einstellung der Bitbreite

```
1 use> mv -config bitwidth:=12
2 16:03:34 INFO  KodkodModelValidatorConfiguration: Set bitwidth to 12.
```

Die aktuelle Konfiguration kann dann mit dem Befehl im Listing 4.12 gespeichert werden, sodass sie nach Neustart oder Absturz des USE-Systems weiterhin aktiv ist.

Listing 4.12: Speichern der Einstellungen

```
1 use> mv -config save
2 16:05:05 INFO  KodkodModelValidatorConfiguration: Saved KodkodSolver configuration
```

In den folgenden Testfällen wird der Befehl *mv -validate* benutzt, um nach einem Objektdiagramm zu suchen, das den Einschränkungen der übergebenen .properties-Datei entspricht. Sofern es eine Lösung gibt, wird *Satisfiable* ausgegeben. Falls es keinen Zustand gibt, der alle Bedingungen erfüllt, wird *Unsatisfiable* gemeldet. Ebenfalls möglich ist der Fall *Trivially_Unsatisfiable*, dies bedeutet, dass schon anhand der angegebenen Konfiguration erkennbar ist, dass es keine der Konfiguration entsprechende Instanziierung geben kann.

4.3.1.1 Initialisierung eines Trucks

Das Objektdiagramm in der untenstehenden Abbildung 4.4 basiert auf der Konfiguration in der ersten Spalte der Tabelle 4.4. Der Testfall besteht aus drei Systemzuständen, die durch Objekte der Klasse *Snapshot* repräsentiert werden. Die drei Systemzustände werden durch zwei *Filmstrip*-Assoziationen verbunden. Die *Truck*-Operationen *init* und *enter* sollen jeweils einmal ausgeführt werden. Es werden drei Trucks, einen für jeden Zustand, benötigt. Da es drei *Snapshots* geben soll, müssen die drei Trucks durch die Assoziation *SnapshotTruck* verbunden werden, um sie dem jeweiligen Zustand zuzuordnen. Die *PredSuccTruck* Assoziation wird fünfmal benötigt, um die verschiedenen Zustände des Trucks miteinander zu verbinden. Die *init* und *enter*-Operationen werden jeweils einmal ausgeführt.

Die Darstellung von zwei Punkten in drei Systemzuständen benötigt insgesamt sechs Point-Objekte. Jeder Punkt bekommt eine *SnapshotPoint*-Verknüpfung. Vier *PredSucc-Point*-Verknüpfungen sind nötig, da es zwei Startzustände gibt, die naheliegenderweise keine Vorgänger haben. In diesem Fall werden noch keine Operationen des Point ausgeführt. Die Relation *Connection* hat den Maximalwert *-1* und ist somit in der Anzahl ihrer Vorkommen nicht beschränkt. *Current* ist auf *-1* gesetzt, das bedeutet, dass beliebig viele Verknüpfungen zwischen Point und Trucks bestehen können. Die Ausgabe auf der Konsole sieht folgendermaßen aus:

Listing 4.13: Konsolenausgabe für Truck-Initialisierungs-Testfall

```
use> mv -validate tollcoll_marcel_FS_mv_Truck_InitOps.properties

21:44:17 INFO  ModelTransformator: Start model transformation for 'TollCollect'

21:44:17 INFO  InvariantTransformator: Invariant transformation successful

21:44:17 INFO  ModelTransformator: Model transformation successful

21:44:17 INFO  ModelTransformator: Translation time: 184 ms

21:44:17 INFO  Model: Reset successful

21:44:17 INFO  ConfigurablePlugin: Model configuration successful

21:44:17 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

21:44:18 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and
   bitwidth 12...

21:44:18 INFO  KodkodModelValidator: SATISFIABLE

21:44:18 INFO  KodkodModelValidator: Translation time: 290 ms; Solving time: 21 ms

21:44:18 INFO  UseKodkodModelValidator: Create object diagram
```

Die Kommandozeilenausgabe im Listing 4.13 dokumentiert die einzelnen Schritte des Validationsprozess. Zuerst wird das Modell in ein SAT-Problem transformiert (*Translation*) und dieses anschließend gelöst (*Solving*). Es werden jeweils die Zeiten protokolliert, die es gedauert hat, die Transformation bzw. das Lösen durchzuführen.

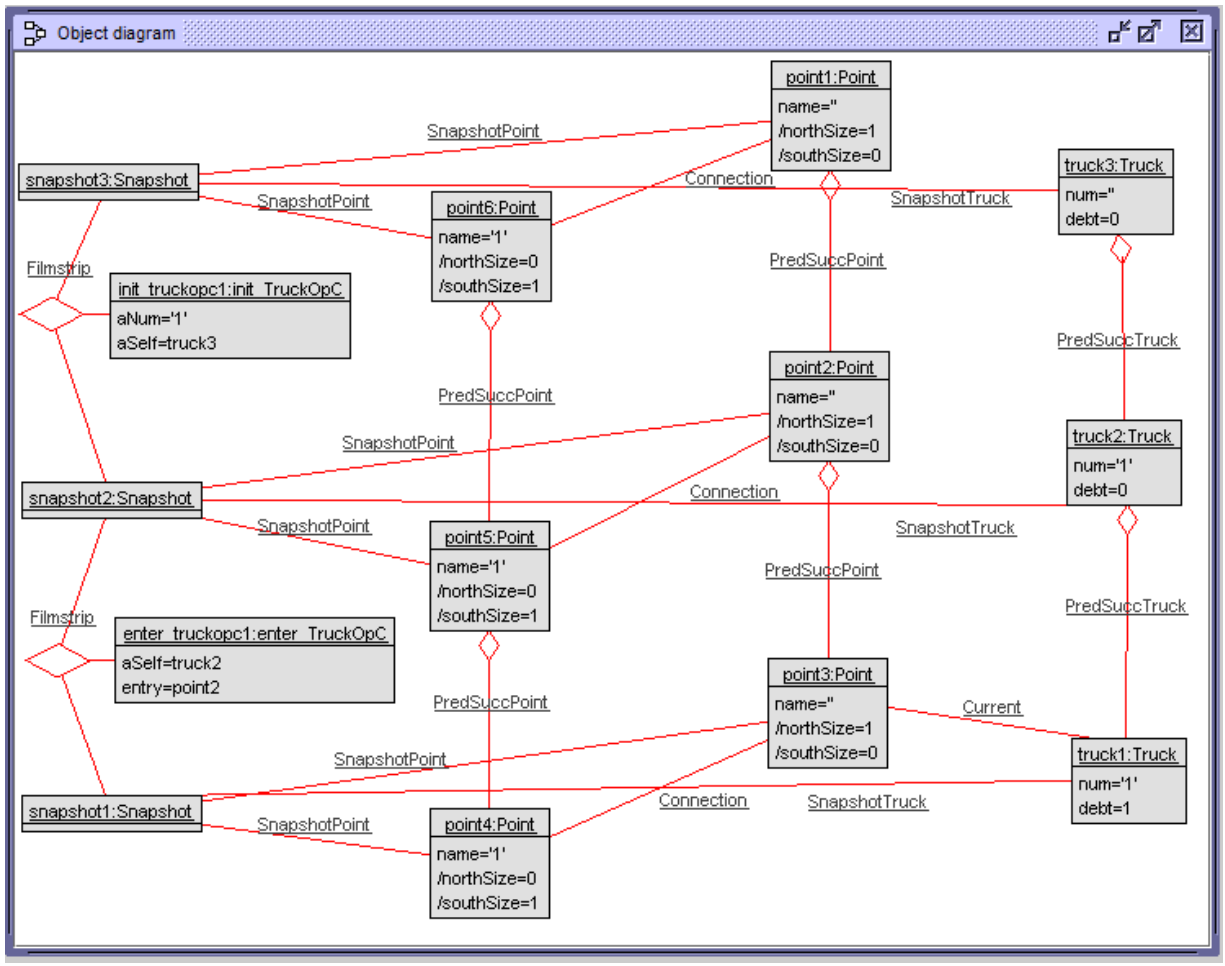


Abbildung 4.4: Objektdiagramm mit Initialisierungsoperationen eines Trucks

Die Kommandozeilenausgaben, auch für alle weiteren Diagramme, befinden sich im Abschnitt V des Anhangs. Das gelieferte Objektdiagramm ist wie erwartet. Die Attribute des Truck werden entsprechend der Operationen gesetzt, so wird die *num* des Trucks erst nach dem Aufruf der Initialisierungsoperation auf 1 gesetzt und die *debt* steigt wie gewünscht beim Betreten durch die Operation *enter* von 0 auf 1. Die Link *Current* wurde ebenfalls wie gewünscht im dritten Zustand hinzugefügt. Auch die Reihenfolge der Ausführung der Operationen (*enter* nach *init*) ist richtig.

4.3.1.2 Initialisierung zweier Punkte

Als Nächstes sollen in der untenstehenden Abbildung 4.5 zwei Punkte, anders als im vorherigen Diagramm, initialisiert werden. Da diesmal drei Operationsaufrufe vorgesehen sind, sind im Objektdiagramm vier Systemzustände (*Snapshots*) enthalten. Diese werden durch drei Filmstrip-Assoziationen verknüpft. Es wird für jeden Zustand ein *Truck*, also insgesamt vier Stück, erzeugt. Da es vier *Snapshots* geben soll, werden die vier Trucks durch die Assoziation *SnapshotTruck* verbunden, um sie dem jeweiligen Zustand zuzuordnen. Die *PredSuccTruck*-Assoziation wird dreimal benötigt, um die verschiedenen Zustände des Trucks miteinander zu verbinden. Die *Truck*-Operationen bleiben außen vor. Da es zwei Punkte und vier Zustände gibt, entstehen insgesamt acht Punkte. Davon bekommt jeder eine *SnapshotPoint*-Verknüpfung und alle außer den Punkten, die dem Startzustand zugeordnet sind, werden durch die *PredSuccPoint*-Verknüpfungen mit ihren Vorgängern verbunden. Somit gibt es die *PredSuccPoint*-Verknüpfung insgesamt sechsmal.

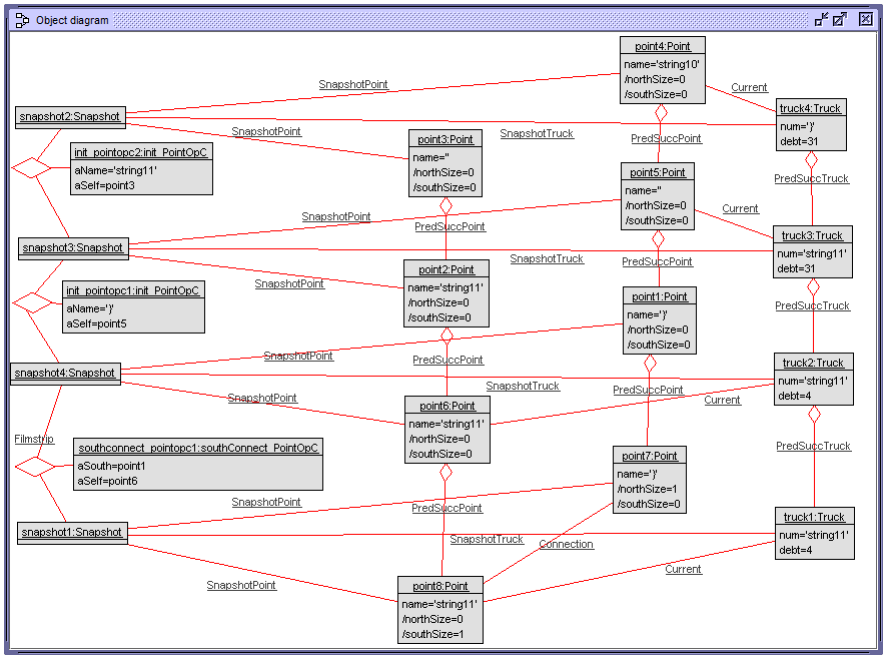


Abbildung 4.5: Objektdiagramm mit Initialisierungsoperationen zweier Punkte

Es wird zweimal die Punkt-Operation *init* und einmal *southConnect* ausgeführt. Die Testfälle hätten genauso gut mit *northConnect* funktioniert. Die Frameconditions für die Point-Operation *init* mussten deaktiviert werden, da sonst nur *Unsatisfiable* zurückgegeben wurde.

Das Objektdiagramm zeigt nicht ganz das gewünschte Verhalten, da sich die Attribute des Trucks willkürlich ändern, was durch die deaktivierten Frameconditions der *init*-Operation zu erklären ist. Alles andere ist den Erwartungen entsprechend. So wurden die Initialisierungen vor dem *southConnect* ausgeführt und die Namen der Punkte bleiben nach der Initialisierung konstant. Der Link *Connection* wurde, wie erhofft, nach dem Operationsaufruf von *southConnect* entsprechend gesetzt.

4.3.1.3 Alle Initialisierungsoperationen

In der untenstehenden Abbildung 4.6 werden alle Operationen der zwei vorherigen Zustände im gleichen Diagramm ausgeführt. Das Objektdiagramm soll somit sechs Systemzustände, die durch Objekte der Klasse *Snapshot* repräsentiert werden, enthalten. Die sechs Systemzustände werden durch die Filmstrip-Assoziation verknüpft. Die *Truck*-Operationen *init* und *enter* sollen jeweils einmal ausgeführt werden. Es werden sechs *Trucks*, einen für jeden Zustand, benötigt. Da es sechs *Snapshots* geben soll, müssen die sechs *Trucks* durch die Assoziation *SnapshotTruck* verbunden werden, um sie dem jeweiligen Zustand zuzuordnen. Die *PredSuccTruck*-Assoziation wird fünfmal benötigt, um die verschiedenen Zustände des Trucks miteinander zu verbinden.

Es werden zwei Punkte in sechs Systemzuständen dargestellt, daher werden insgesamt 12 Point-Objekte benötigt. Jeder Punkt bekommt eine *SnapshotPoint* Verknüpfung. 10 *PredSuccPoint* Verknüpfungen sind nötig, da es zwei Startzustände gibt, die keine Vorgänger haben. Es wird zweimal die Punkt-Operation *init* und einmal *southConnect* ausgeführt.

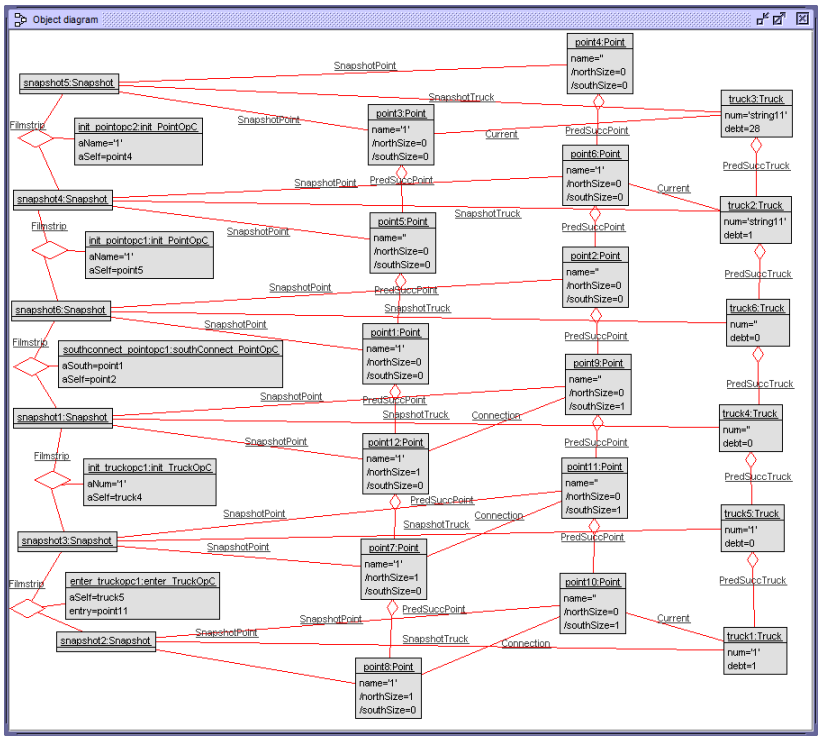


Abbildung 4.6: Objektdiagramm mit allen Initialisierungsoperationen

Auch hier mussten, da wie in Fall Initialisierung zweier Punkte (Abschnitt 4.3.1.2) die Operation *init* des Points ausgeführt wird, die Frameconditions der Operation deaktiviert werden, um ein Objektdiagramm zu erhalten. Das Diagramm zeigt eine valide Reihenfolge der Operationsaufrufe. Es ist zu sehen, dass wie erhofft, nach dem *enter* des Trucks im Folgezustand der *current*-Link erscheint.

4.3.2 Alle Operationen des Trucks

Das folgende Diagramm in Abbildung 4.7 enthält alle Operationen des Trucks in einfacher Ausführung. Um das gewünschte Objektdiagramm mit allen Operationen des Trucks zu erhalten, wurde die Konfigurationsdatei entsprechend der Tabelle 4.5 eingestellt.

Testfälle	**alle Truck-Operationen**
Objekte	2 Points, 1 Truck
Konfig	Snapshots: 6..6 Filmstrip: 5..5 Truck: 6..6 *init_TruckOpC*: 1..1 *enter_TruckOpC*: 1..1 *move_TruckOpC*: 1..1 *pay_TruckOpC*: 1..1 *bye_TruckOpC*: 1..1 SnapshotTruck: 6..6 PredSuccTruck: 5..5 Point: 12..12 SnapshotPoint: 12..12 PredSuccPoint: 10..10 *init_PointOpC*: 0..0 *southConnect_PointOpC*: 0..0 *northConnect_PointOpC*: 0..0 Connection: 0..-1 Current: 0..-1

Tabelle 4.5: Konfiguration für Testfall mit allen Truck-Operationen

Für den Testfall werden, wie im vorherigen Fall in Abschnitt 4.3.1.3, ebenfalls sechs Snapshots und fünf Filmstrip-Assoziationen erlaubt. Die Anzahl an *Trucks* und *Points* und somit auch *SnapshotPoint* und *PredSuccPoint* sind ebenfalls wie im vorherigen Diagramm gewählt. Diesmal werden jedoch alle Operation des Trucks jeweils einmal ausgeführt, um zu sehen, ob auch diese Operationen nach der Transformation des Modells wie erwartet funktionieren. Die Operationen *init*, *enter*, *move* und *pay* sollen jeweils einmal aufgerufen werden. Der Modelvalidator findet in überschaubarer Zeit das Objektdiagramm in Abbildung 4.7. Anhand des Objektdiagramms ist nun unter anderem möglich, die Abfolge der Operationsaufrufe abzulesen.

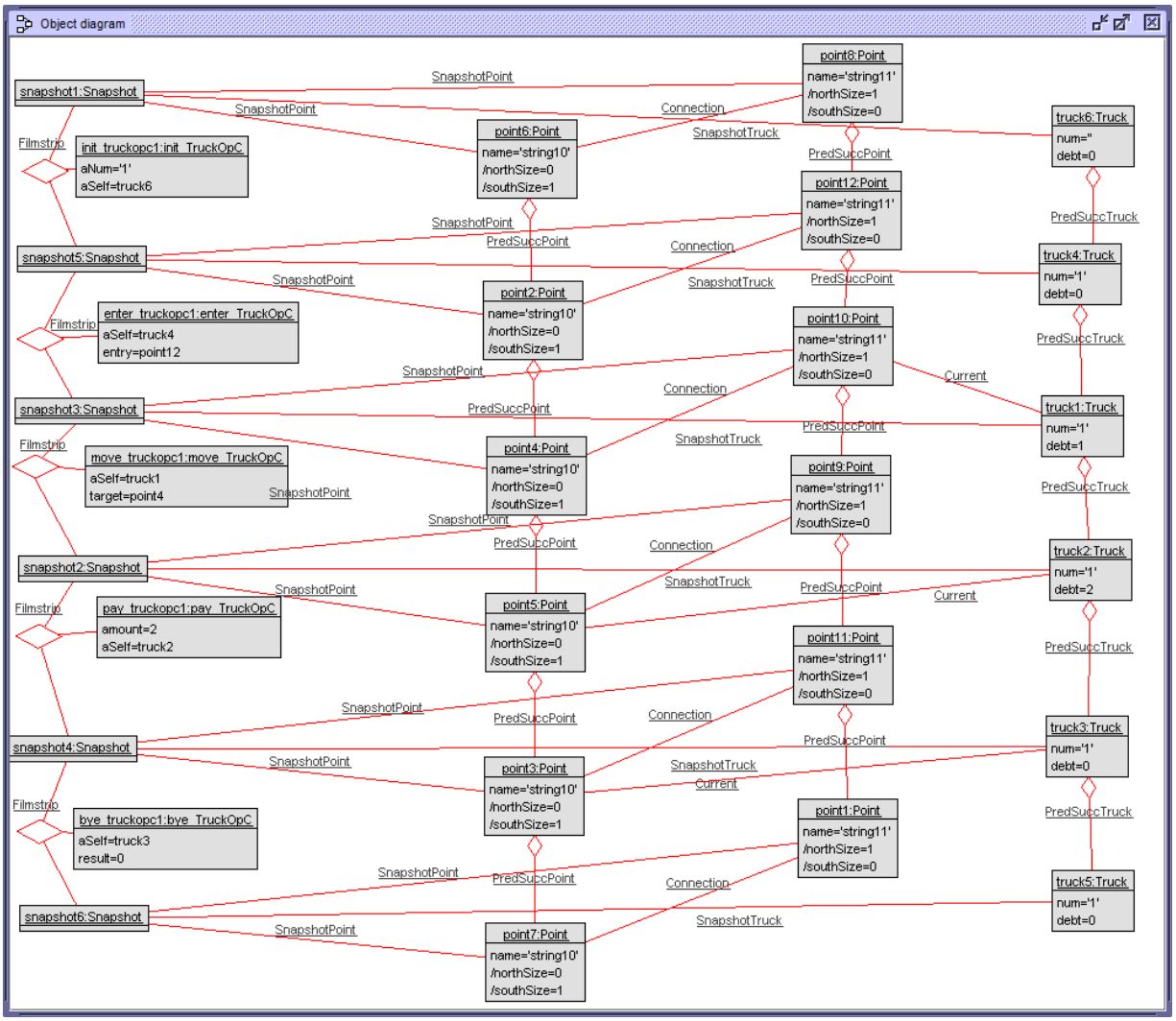


Abbildung 4.7: Objektdiagramm mit allen Operationen des Trucks

Im vorliegenden Diagramm ist die Reihenfolge der Operationsaufrufe genau wie erwünscht, so wird, entsprechend der state machine, erst initialisiert, dann betreten und anschließend bewegt, bezahlt und die Autobahn verlassen. Der Link *Current* erscheint erst, so wie es sein soll, nach dem Ausführen der Operation *enter* und ändert sich nach Ausführung von *move* entsprechend. Auch der Wert der *debt* verhält sich den Erwartungen gerecht.

4.3.3 Alle Operationen von Truck und Point

In dieser Konfiguration, abzulesen in Tabelle 4.6, werden alle Operationen einmal ausgeführt. Die Punkte werden *initialisiert*, es wird eine *südliche Verbindung* zwischen den Punkten hergestellt, der Truck wird *initialisiert*, durch *enter* auf einem der Punkte platziert, und soll sich anschließend bewegen (*move*), dann die aufgekommene Schuld bezahlen (*pay*) und letztendlich das System verlassen (*bye*).

Testfälle	**Alle Operationen**
Objekte	2 Points, 1 Truck
Konfig	Snapshots: 9..9 Filmstrip: 8..8 Truck: 9..9 *init_TruckOpC*: 1..1 *enter_TruckOpC*: 1..1 *move_TruckOpC*: 1..1 *pay_TruckOpC*: 1..1 *bye_TruckOpC*: 1..1 SnapshotTruck: 6..6 PredSuccTruck: 5..5 Point: 18..18 SnapshotPoint: 18..18 PredSuccPoint: 16..16 *init_PointOpC*: 1..1 *southConnect_PointOpC*: 1..1 *northConnect_PointOpC*: 0..0 Connection: 0..-1 Current: 0..-1
deaktivierte Invarianten	init_PointOpC::post _truckUnchanged init_PointOpC::post _pointUnchangedExcept

Tabelle 4.6: Konfiguration für Testfall mit allen Operationen

Es werden neun Zustände und acht Filmstrips benötigt, da es insgesamt acht Operationsaufrufe sind. Alle *Truck*-Operationen werden jeweils einmal ausgeführt. Da neun *Snapshots* generiert werden sollen, muss auch für jeden *Snapshot* ein *Truck* erstellt werden. Es sollen zwei Punkte in neun Zuständen generiert werden. Beide Punkte werden *initialisiert* und ein Aufruf von *southConnect* soll ausgeführt werden. Bei dieser Konfiguration hat der Modelvalidator jedoch auch nach mehreren Stunden des Rechnens keine Lösung gefunden. Dies liegt vermutlich an der hohen Anzahl von Objekten und der daraus resultierenden Komplexität des Lösungs-Objektdiagramms. In machen Fällen kam es dann zu einer OutOfMemoryError-Exception, siehe Anhang V.5. Durch ein entsprechendes .SOIL-Skript lässt sich dennoch ein Objektdiagramm „per Hand“ erzeugen, das die geforderten Eigenschaften erfüllt. Wie ein Objektdiagramm mit all den Operationen aussehen müsste ist in Abbildung 4.8 zu sehen.

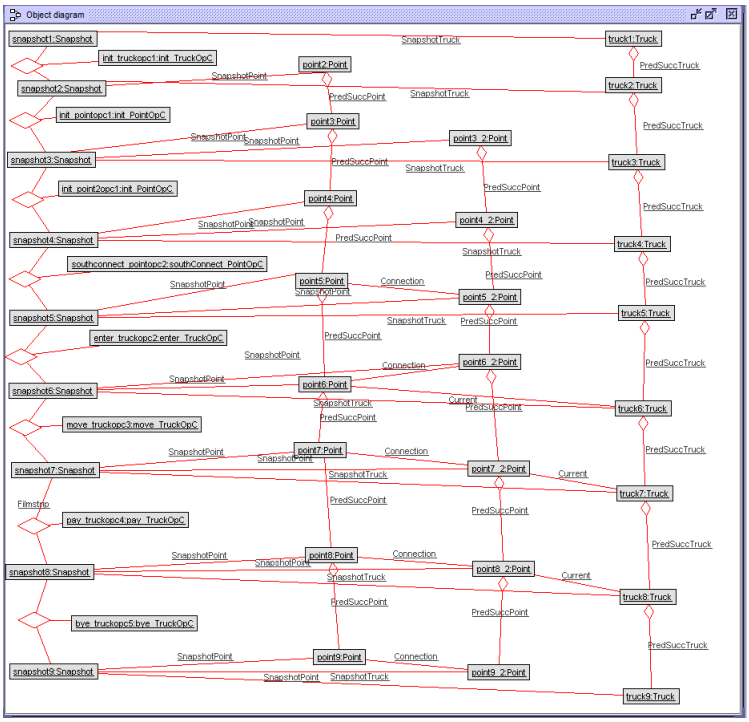


Abbildung 4.8: Objektdiagramm mit allen Operationen

4.3.4 Ausführungszeiten

Die Tabelle 4.7 zeigt, wie lange es bei den verschiedenen Testfällen gedauert hat, bis der Modelvalidator ein Ergebnis zurückgab. Die Zeiten variieren bei mehrfacher Ausführung des selben Testfall jedoch meist nur gering, daher ist auch bei Betrachtung von nur einem Wert eine Tendenz erkennbar. Wenig überrascht steigt die solving time analog zur Komplexität der Konfigurationen. Die Lösungszeiten (abzulesen im Abschnitt V des Anhangs) sind im vertretbaren Rahmen geblieben, wenn denn eine Lösung möglich war.

Testfälle	**Truck init-Ops**	**Point init-Ops**	**Alle init-Ops**	**Alle Truck-Ops**	**Alle Ops**
solving time	0,021sec	0,913sec	3,40min	2,94min	-

Tabelle 4.7: Übersicht über die Lösungszeiten der Testfälle

4.4 Traversieren über ein Streckennetz

Als nächstes wird ein Streckennetz vorgeben, sodass nur noch die Bewegung des Trucks durch den Modelvalidator generiert wird. Der Truck soll sich auf verschieden großen Streckennetzen unterschiedlich oft bewegen. Dadurch soll sichergestellt werden, dass er sich wie gewünscht zu anderen Punkten bewegt und keine ungewollte Seiteneffekte oder Ähnliches auftreten. Dazu benötigen man die Partial Solution Completion-Funktion, die über die Konsole unter dem Namen *automaticDiagramExtraction* aktiviert wird.

Listing 4.14: automaticDiagramextraction Option aktivieren

```
1 use> mv -config automaticDiagramextraction:=On
2 23:27:02 INFO  KodkodModelValidatorConfiguration: Enable automatic extraction of an
    available object diagram.
```

Nun wird das bereits in USE vorhandene Objektdiagramm als Grundlage für den Modelvalidator genommen, d. h. das bestehende Diagramm wird ergänzt und nicht komplett neu erzeugt.

Das vorbereitete Objektdiagramm kann per .SOIL-Datei in das USE-System geladen werden.

Listing 4.15: Streckennetz in USE laden

```
1 use> open PartialSolution_2Pont_1Connect_1Snap.soil
```

Der Validierungsprozess wird unter Angabe der Konfigurationsdatei folgendermaßen gestartet:

Listing 4.16: Validierung starten

```
1 use> mv -validate "partialSolution_2Point_1Connect_1Move.properties"
```

Die Partial Solution Completion baut auf dem bestehenden Objektdiagramm auf, in dem in der Konfigurationsdatei die bestehenden, schon in der vorherigen Lösung vorhandenen Objekte, mit einbezogen werden. In der .properties-Konfigurationsdatei wird die Anzahl der Objekte so gewählt wie sie im erweiterten Objektdiagramm vorhanden sein sollen. Die Objekte, bei denen die angegebene Mindestanzahl noch nicht erreicht ist, werden durch die Partial Solution Completion ergänzt.[36] Die Konfigurationsdateien zu allen folgenden Testfällen befinden sich im Anhang im Abschnitt III.2.

4.4.1 Streckennetze

Es sollen, wie die Tabelle 4.8 zeigt, Streckennetze drei verschiedener Größen ausprobiert werden. Zunächst wird ein Objektdiagramm mit zwei Punkten und einer Verbindung zwischen den Punkten als Grundlage genommen (Abbildung 4.9), dann eines mit drei Punkten

[36] Vgl. Gogolla/Hilken (2016).

und drei Verbindungen (Abbildung 4.10) und zum Schluss fünf Punkte mit sechs Verbindungen (Abbildung 4.11). Dafür wurden entsprechende .SOIL-Skripte (siehe Abschnitt III.1 im Anhang) geschrieben, die die Objekte erzeugen. Somit ist die Verbindung zwischen den Punkten bereits vorgegeben. Der Truck, der später das Streckennetz befahren soll, ist bereits durch das .SOIL-Skript initialisiert. Er hat bereits einen Punkt betreten, daher ist die Schuld (*debt*) bereits auf 1 gesetzt, da der Truck beim Betreten durch die *enter*-Operation eine Geldeinheit bezahlen müsste. Somit geht es nur noch um die Bewegung des Trucks, da der Zustand, der vom .SOIL-Skript vorgegeben ist, eine Situation nach dem Betreten der Autobahn darstellt.

Die Punkte sind zur Veranschaulichung nach deutschen Städten benannt, HB steht für Bremen, HH für Hamburg, H für Hannover, K für Köln und M für München. Um die Orientierung zu erleichtern, entsprechen die Nord- und Südbeziehungen der Realität.

Steckennetze	**2 Städte**	**3 Städte**	**5 Städte**
Objekte und Links	Snapshots: 1 Filmstrip: 0 Point: 2 SnapshotPoint: 2 Truck: 1 SnapshotTruck: 1 Current: 1 Connection: 1	Snapshots: 1 Filmstrip: 0 Point: 3 SnapshotPoint:3 Truck: 1 SnapshotTruck:1 Current: 1 Connection:3	Snapshots: 1 Filmstrip: 0 Point: 5 SnapshotPoint: 5 Truck: 1 SnapshotTruck: 1 Current: 1 Connection: 6

Tabelle 4.8: Überblick über Streckennetze

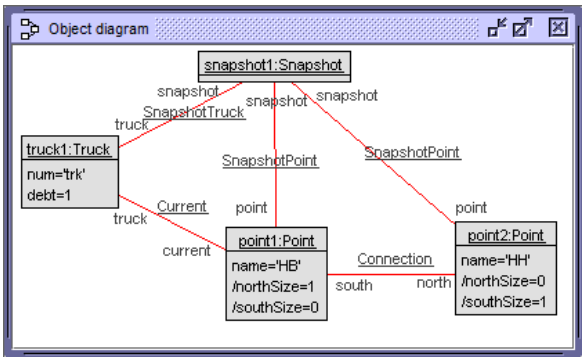


Abbildung 4.9: Streckennetz mit zwei Städten und einer Verbindungen

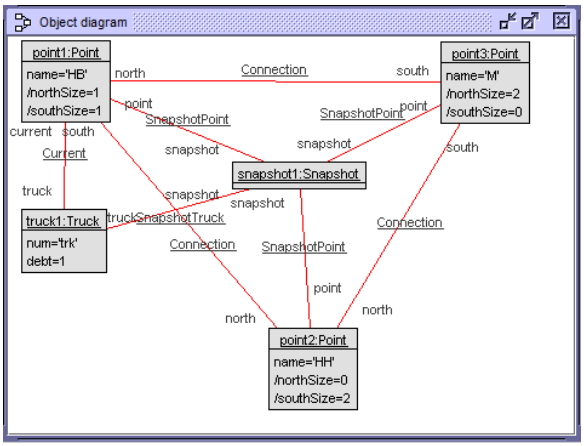


Abbildung 4.10: Streckennetz mit drei Städten und drei Verbindungen

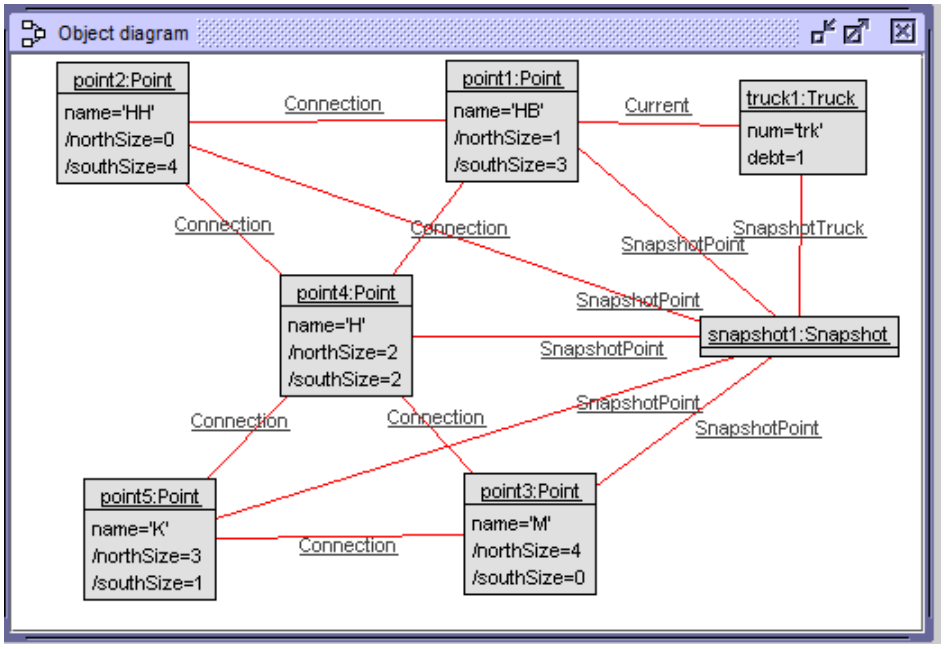


Abbildung 4.11: Streckennetz mit fünf Städten und sechs Verbindungen

4.4.2 Bewegung des Trucks

Nun soll sich der Truck entsprechend der Angaben in der Konfigurationsdatei über das Streckennetz bewegen. Der Startpunkt, also der Punkt an dem sich der Truck im ersten Zustand befindet, ist durch das Skript auf HB festgelegt.

4.4.2.1 Streckennetz mit zwei Städten

In der Tabelle 4.11 sind alle wichtigen Konfigurationen für einen, zwei und drei Operationsaufrufe von *move* auf dem Streckennetz mit zwei Städten angegeben.

Testfall	**1 Move**	**2 Moves**	**3 Moves**
Objekte und Links	2 Points, 1 Connection	2 Points, 1 Connection	2 Points, 1 Connection
Konfig	Snapshots: 2..2 Filmstrip: 1..1 *move_TruckOpC*: 1..1 Truck: 2..2 SnapshotTruck: 2..2 PredSuccTruck: 1..1 Point: 4..4 SnapshotPoint: 4..4 PredSuccPoint: 2..2	Snapshots: 3..3 Filmstrip: 2..2 *move_TruckOpC*: 2..2 Truck: 3..3 SnapshotTruck:3..3 PredSuccTruck: 2..2 Point: 6..6 SnapshotPoint:6..6 PredSuccPoint:4..4	Snapshots: 4..4 Filmstrip: 3..3 *move_TruckOpC*:3..3 Truck: 4..4 SnapshotTruck: 4..4 PredSuccTruck: 3..3 Point: 8..8 SnapshotPoint: 8..8 PredSuccPoint: 6..6

Tabelle 4.9: Bewegungen über Streckennetz mit zwei Städten

4.4.2.1.1 Zwei Städte, eine Bewegung

Die Einstellung aus Tabelle 4.11 wurden in der .properties-Datei vorgenommen, um den Truck vom Punkt HB zum Punkt HH fahren zu lassen. Anhand des Links *current* zwischen dem Truck und den Punkten ist abzulesen, wo dieser sich gerade befindet. Die Strecke, die der Truck wählt, ist in diesem Fall vorhersehbar, da nur die Option nach HH zu fahren besteht. Die hier angegebenen Minimal- und Maximalanzahlen sind nur solche, die das Ergebnis beeinflussen, das heißt, alle Konfigurationen der nicht verwendeten Operationen sind auf 0 gesetzt. Das vom Modelvalidator erzeugte Objektdiagramm ist in Abbildung 4.12 zu sehen. Das *move_TruckOpC*-Objekt hat das Attribut *target* mit der Belegung point2. Da im nächsten Zustand die Verbindung *current* zwischen dem Truck und point4 besteht und point4 der Nachfolgezustand von point2 ist, entspricht das Objektdiagramm genau den Erwartungen. Auch die *debt* ist erwartungsgemäß um genau 1 gestiegen.

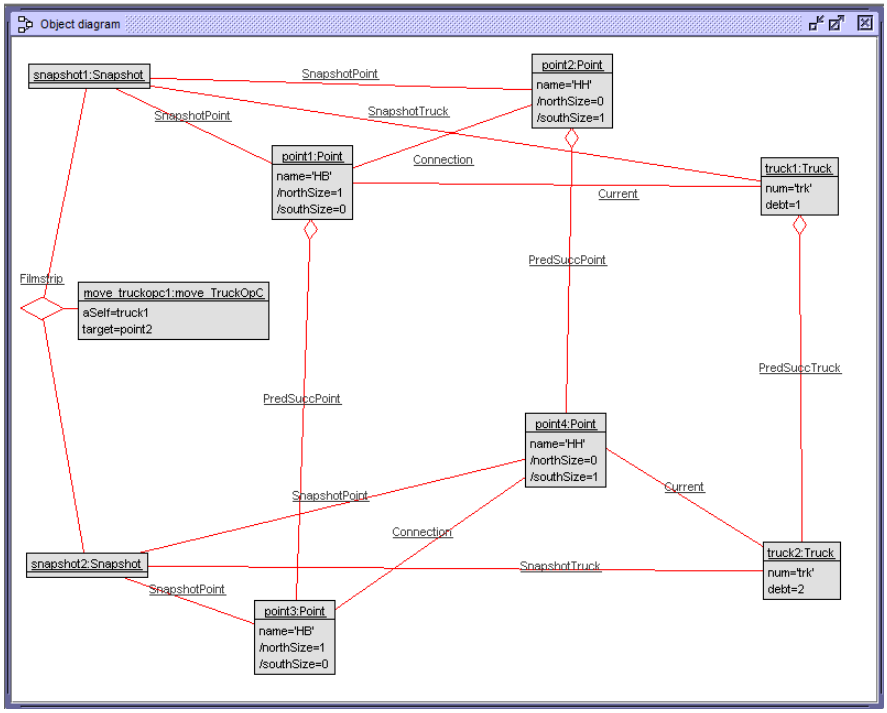


Abbildung 4.12: Objektdiagramm mit zwei Städten und einer Bewegung

4.4.2.1.2 Zwei Städte, zwei Bewegungen

Im folgenden Fall soll sich der Truck auf dem gleichen Streckennetz zweimal bewegen. Das vom Modelvalidator erzeugte Objektdiagramm (Abbildung 4.13) zeigt die erwartbare Bewegung des Trucks von Bremen nach Hamburg und wieder zurück nach Bremen. Wie zu sehen sind alle Objekte, Attribute und Links wie gewünscht.

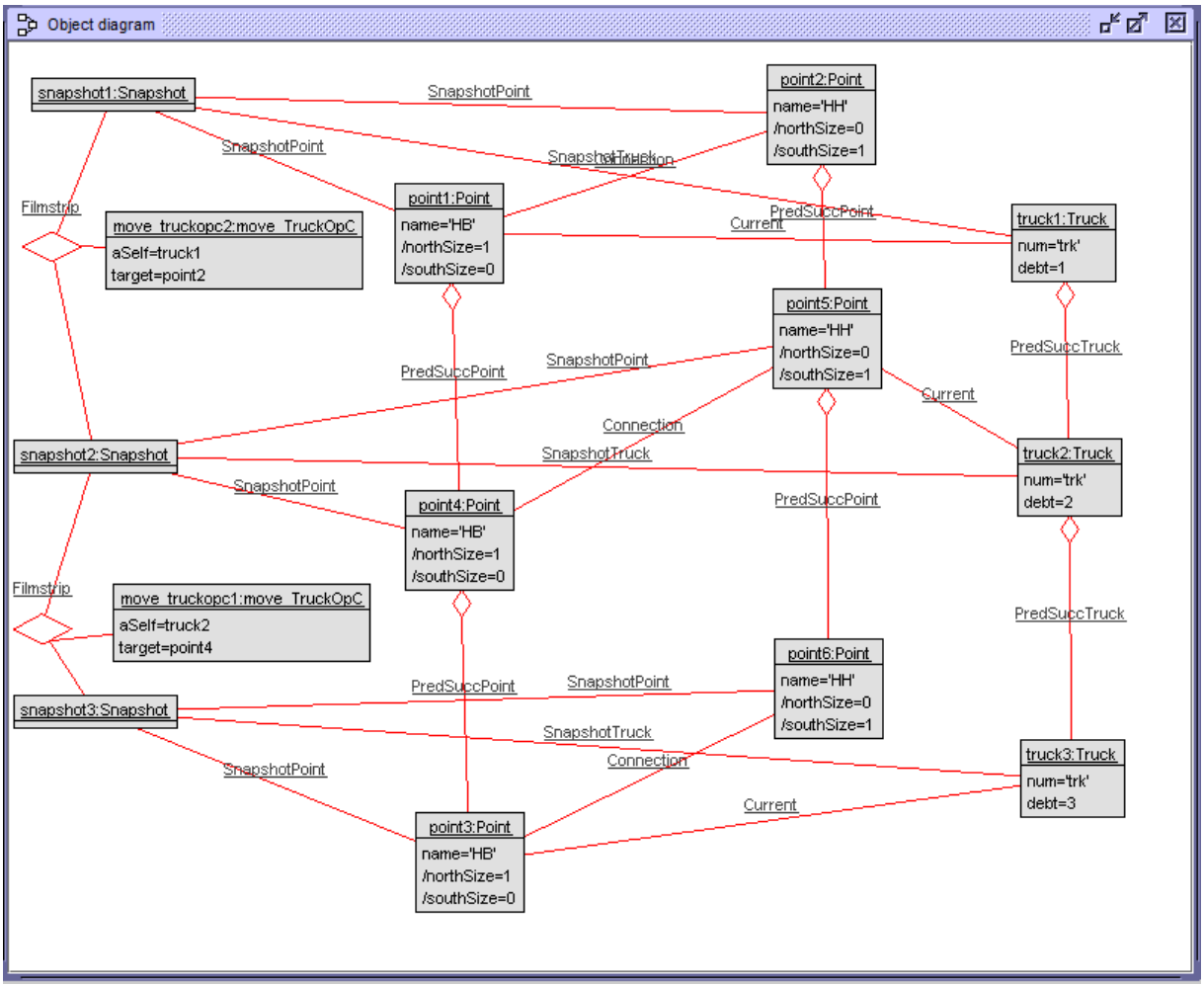


Abbildung 4.13: Objektdiagramm mit zwei Städten und zwei Bewegungen

4.4.2.1.3 Zwei Städte, drei Bewegungen

Der Modelvalidator erzeugt ein Objektdiagramm, wie in der Abbildung 4.14 zu sehen ist, bei dem sich der Truck dreimal bewegt. Er beginnt in HB und fährt nach HH. Anschließend fährt er hin und zurück, da er aufgrund des kleinen Streckennetzes keine andere Wahl hat als zum jeweils anderen der zwei Punkten zu fahren.

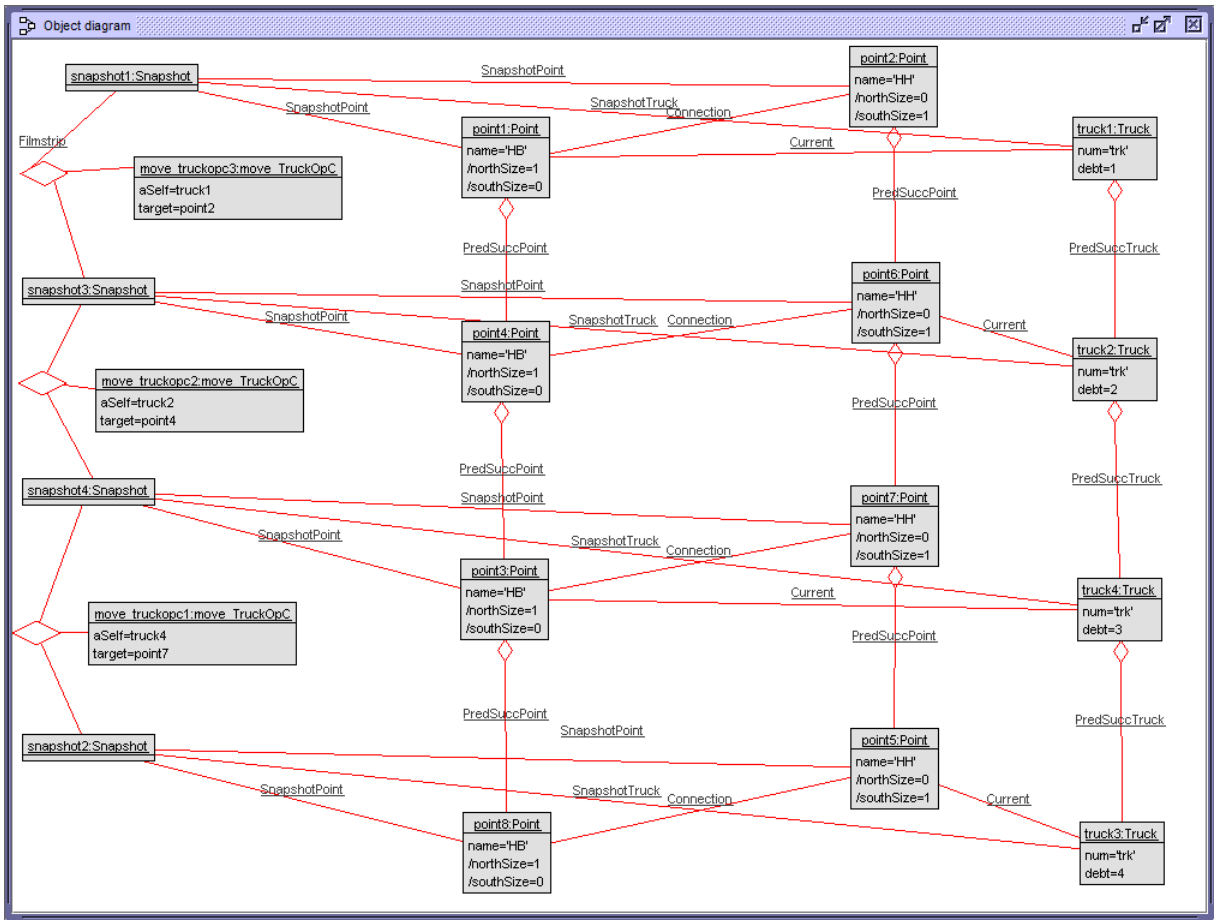


Abbildung 4.14: Objektdiagramm mit zwei Städten und drei Bewegungen

4.4.2.2 Streckennetz mit drei Städten

Nun lassen wir den Truck über ein etwas größeres Straßennetz mit drei Punkten und drei Verbindungen fahren. Die Tabelle 4.10 bietet einen Überblick über die drei Testfälle mit dem Streckennetz mit drei Städten.

Testfall	**1 Move**	**2 Moves**	**3 Moves**
Objekte und Links	3 Points, 3 Connection	3 Points, 3 Connection	3 Points, 3 Connection
Konfig	Snapshots: 2..2 Filmstrip: 1..1 *move_TruckOpC*: 1..1 Truck: 2..2 SnapshotTruck: 2..2 PredSuccTruck: 1..1 Point: 6..6 SnapshotPoint: 6..6 PredSuccPoint: 3..3	Snapshots: 3..3 Filmstrip: 2..2 *move_TruckOpC*: 2..2 Truck: 3..3 SnapshotTruck:3..3 PredSuccTruck: 2..2 Point: 9..9 SnapshotPoint:9..9 PredSuccPoint:6..6	Snapshots: 4..4 Filmstrip: 3..3 *move_TruckOpC*:3..3 Truck: 4..4 SnapshotTruck: 4..4 PredSuccTruck: 3..3 Point: 12..12 SnapshotPoint: 12..12 PredSuccPoint: 9..9

Tabelle 4.10: Bewegungen über Streckennetz mit drei Städten

4.4.2.2.1 Drei Städte, eine Bewegung

Die Abbildung 4.15 zeigt das Streckennetz mit drei Städten und einer Bewegung. Der Truck startet diesmal in München. Da die Partial Solution Completion das Diagramm um die fehlenden Elemente vervollständigt, kann es auch vorkommen, dass wie in diesem Fall der vorgegebene Zustand als Folgezustand genommen und um einen Vorgänger ergänzt wird. Da debt im Skript auf 1 gesetzt war, entsteht auch in dieser Hinsicht kein Widerspruch. Da das Streckennetz nun aus drei Städten besteht, gibt es zwei mögliche Startpunkte für den Truck, in diesem Fall München.

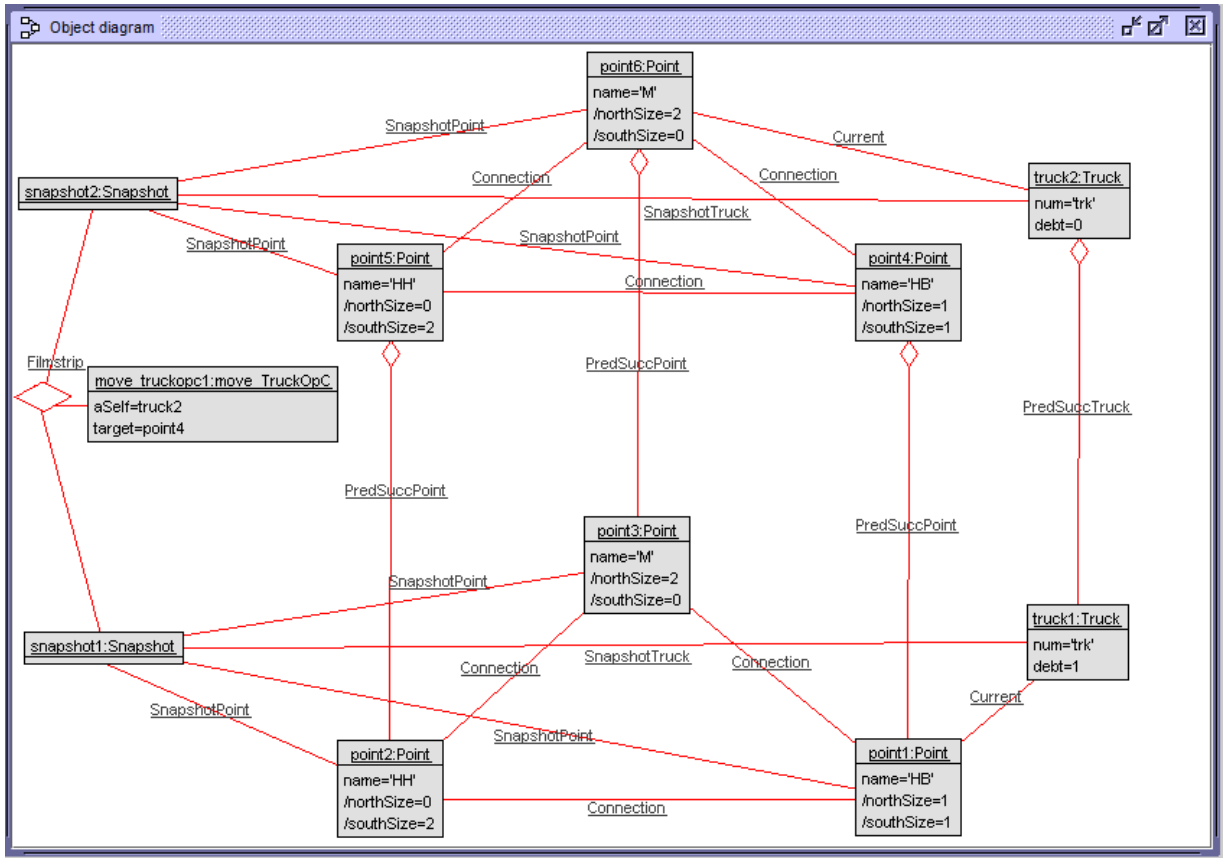


Abbildung 4.15: Objektdiagramm mit drei Städten und einer Bewegung

4.4.2.2.2 Drei Städte, zwei Bewegungen

Auf dem nächsten erzeugten Objektdiagramm (Abbildung 4.16) soll sich der Truck auf dem selben Netz zweimal bewegen. Um die Lesbarkeit des Diagramms zu bewahren, werden hier die Attribute der Objekte nicht mehr angezeigt. Die vertikale Abfolge von Punkten beginnend mit point1, gefolgt von point7 und point9 repräsentieren die verschiedenen Zustände des Punktes HB. Die mit point2 anfangende Reihe ist die von HH, die mit point3 beginnende, sind die Zustände von M. Der Truck fährt von HB los, zunächst nach HH und dann nach M.

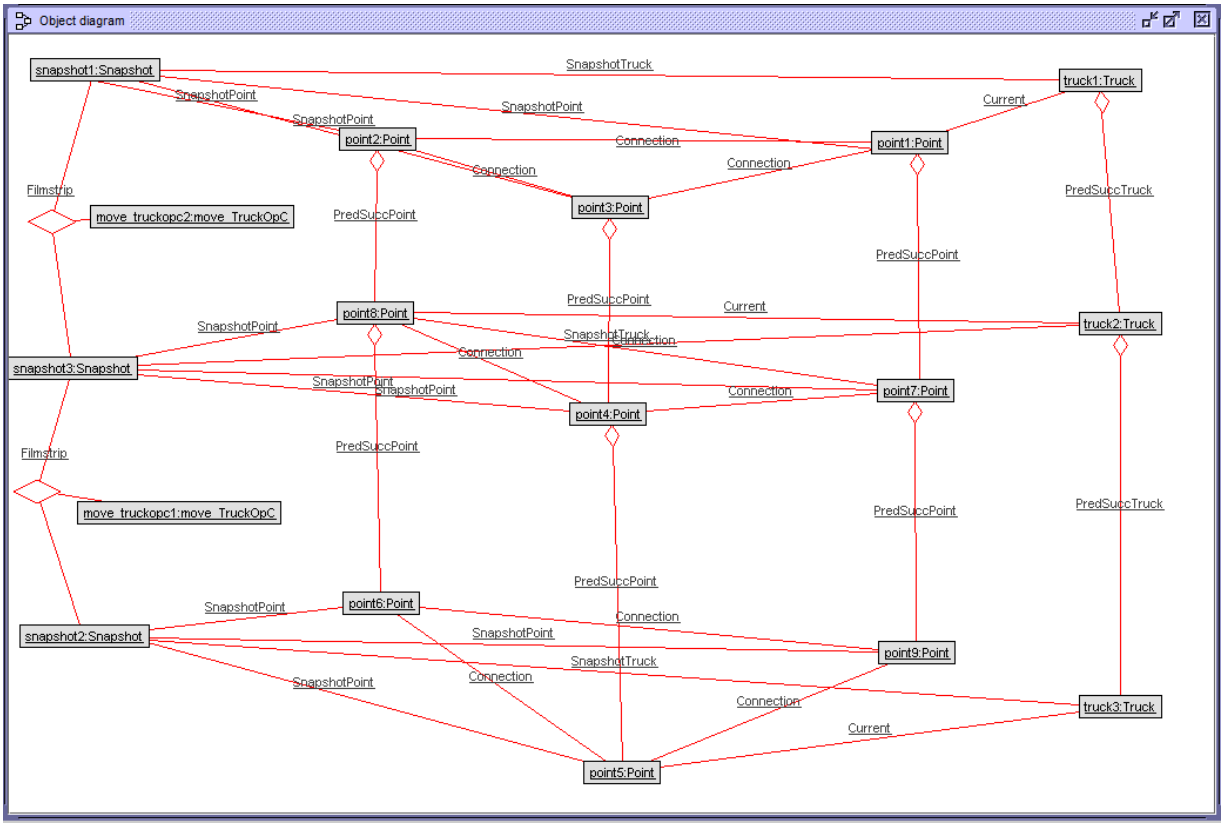


Abbildung 4.16: Objektdiagramm mit drei Städten und zwei Bewegungen

4.4.2.2.3 Drei Städte, drei Bewegungen

Als letztes soll sich der Truck dreimal auf diesem Streckennetz bewegen. Wie auf Abbildung 4.17 zu erkennen ist, fährt der Truck in HB los, zunächst nach HH, dann zurück nach HB und letztendlich nach M.

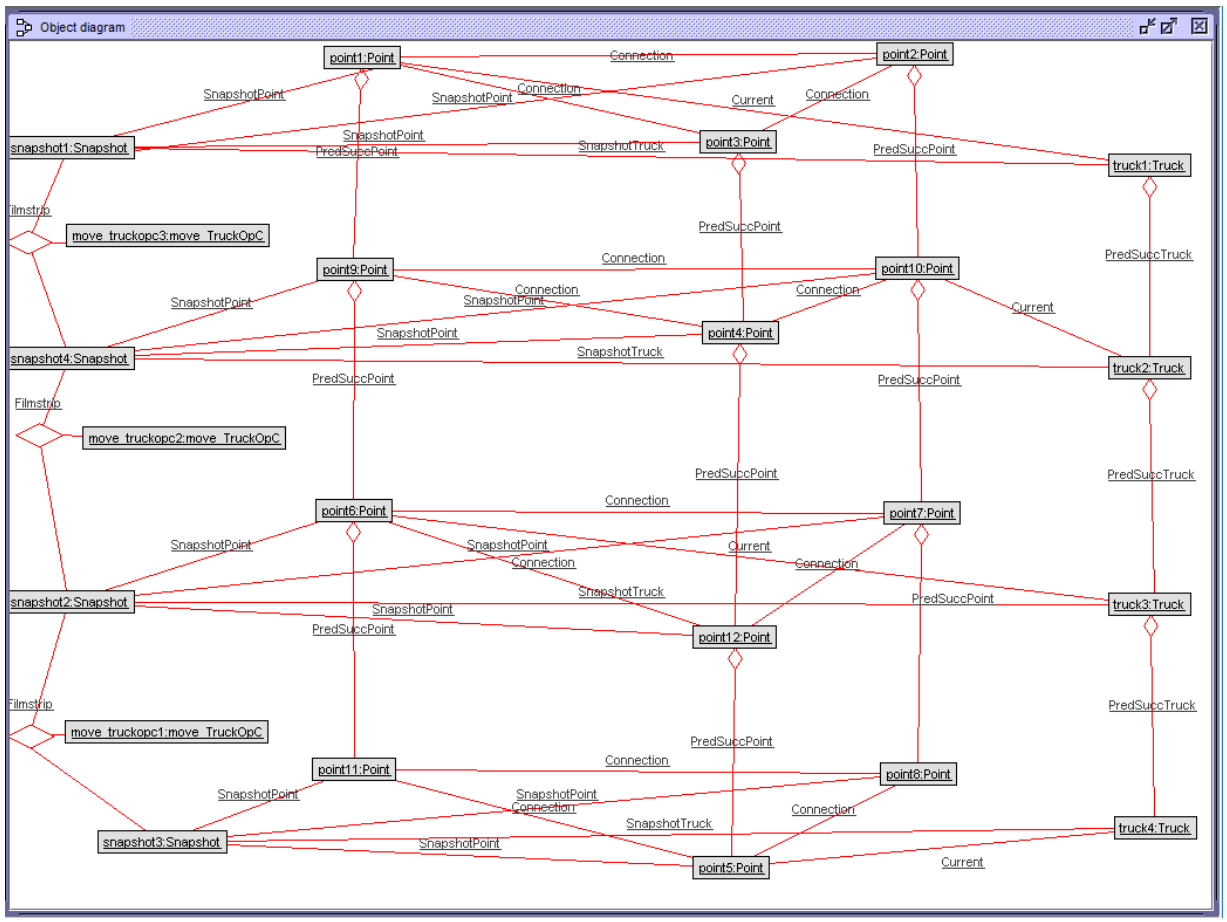


Abbildung 4.17: Objektdiagramm mit drei Städten und drei Bewegungen

4.4.2.3 Streckennetz mit fünf Städten

Nun soll das letzte und größte Streckennetz mit fünf Städten und sechs Verbindungen zwischen den Städten befahren werden. Zur Übersicht über die drei Testfälle dient die Tabelle 4.11.

Testfall	**1 Move**	**2 Moves**	**3 Moves**
Objekte und Links	5 Points, 6 Connection	5 Points, 6 Connection	5 Points, 6 Connection
Konfig	Snapshots: 2..2 Filmstrip: 1..1 *move_TruckOpC*: 1..1 Truck: 2..2 SnapshotTruck: 2..2 PredSuccTruck: 1..1 Point: 6..6 SnapshotPoint: 10..10 PredSuccPoint: 5..5	Snapshots: 3..3 Filmstrip: 2..2 *move_TruckOpC*: 2..2 Truck: 3..3 SnapshotTruck:3..3 PredSuccTruck: 2..2 Point: 9..9 SnapshotPoint:15..15 PredSuccPoint:10..10	Snapshots: 4..4 Filmstrip: 3..3 *move_TruckOpC*:3..3 Truck: 4..4 SnapshotTruck: 4..4 PredSuccTruck: 3..3 Point: 20..20 SnapshotPoint: 20..20 PredSuccPoint: 15..15

Tabelle 4.11: Bewegungen über Streckennetz mit fünf Städten

4.4.2.3.1 Fünf Städte, eine Bewegung

Nun soll sich der Truck auf dem größten Streckennetz bewegen. Auf dem erzeugte Objektdiagramm, zu sehen auf der Abbildung 4.18, fährt der Truck von HB nach HH.

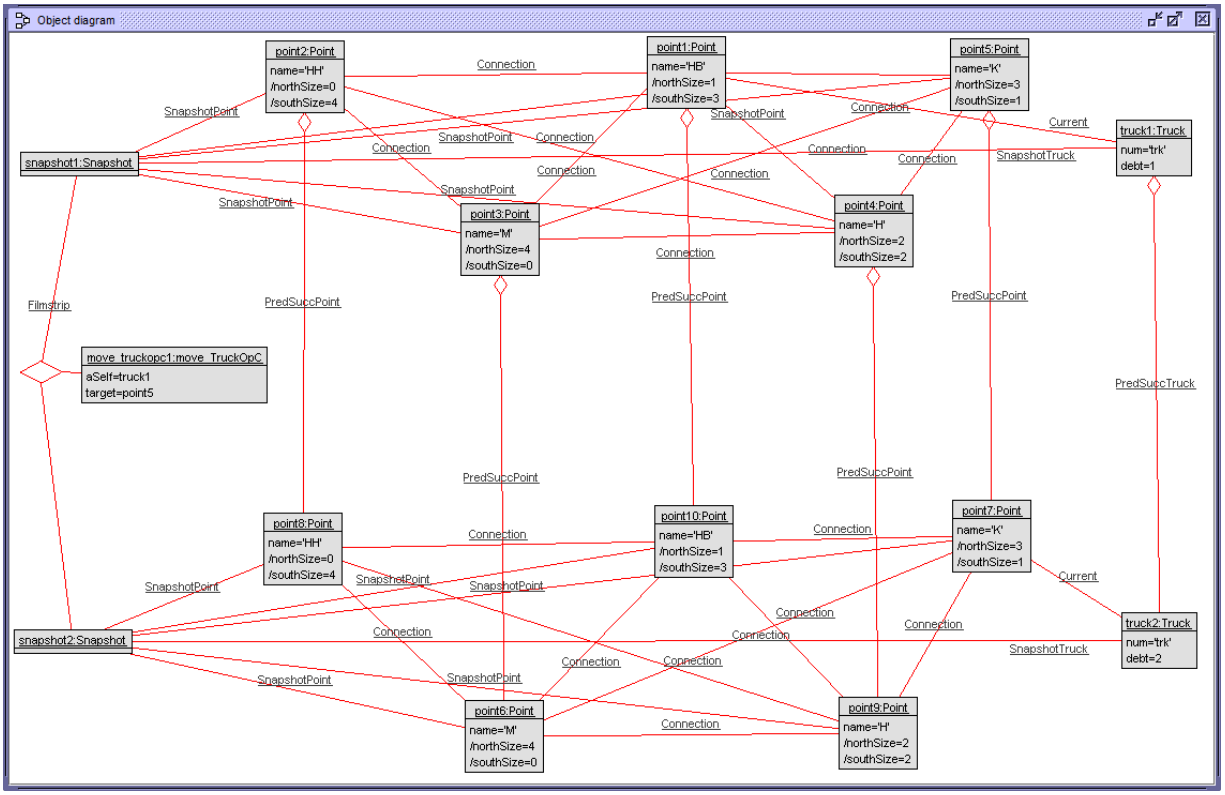


Abbildung 4.18: Objektdiagramm mit fünf Städten und einer Bewegung

4.4.2.3.2 Fünf Städte, zwei Bewegungen

Die Abbildung 4.19 zeigt das Objektdiagramm, auf dem sich der Truck nun zweimal bewegt. Der Truck startet in HB, fährt dann nach K und schließlich nach HH.

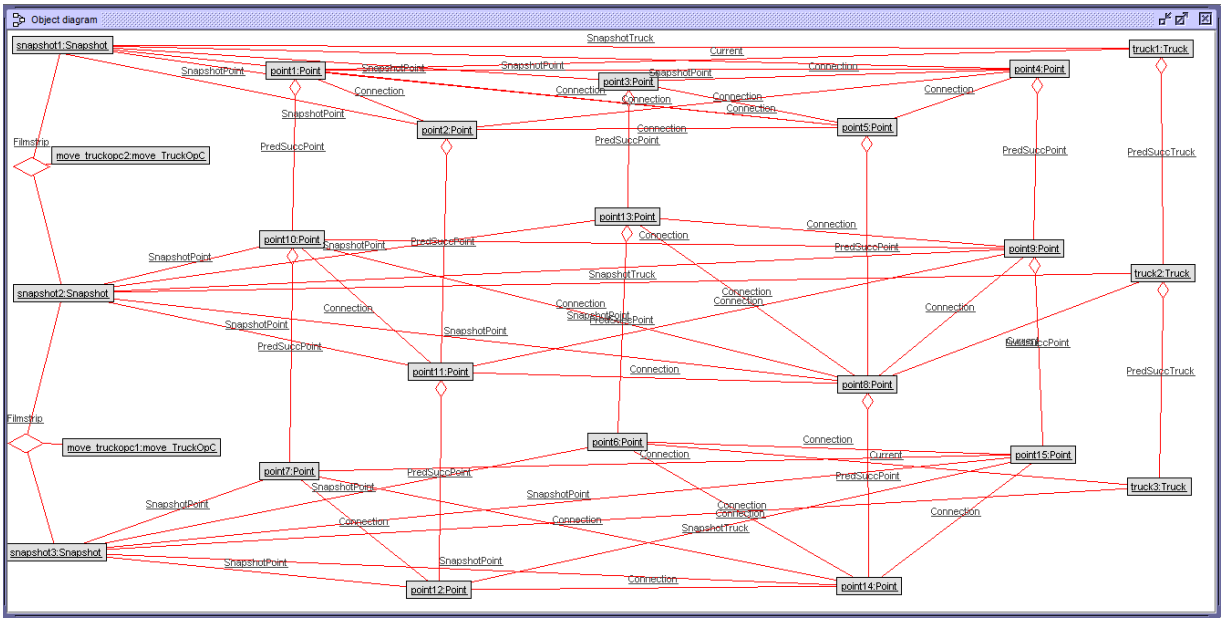


Abbildung 4.19: Objektdiagramm mit fünf Städten und zwei Bewegungen

4.4.2.3.3 Fünf Städte, drei Bewegungen

Jetzt soll sich der Truck im größten Netz dreimal bewegen. Auf dem vom Modelvalidator erzeugten Objektdiagramm (Abbildung 4.20) sind folgende Bewegungen zu erkennen: Der Truck fährt von HB nach K, dann nach HH und letztlich nach M.

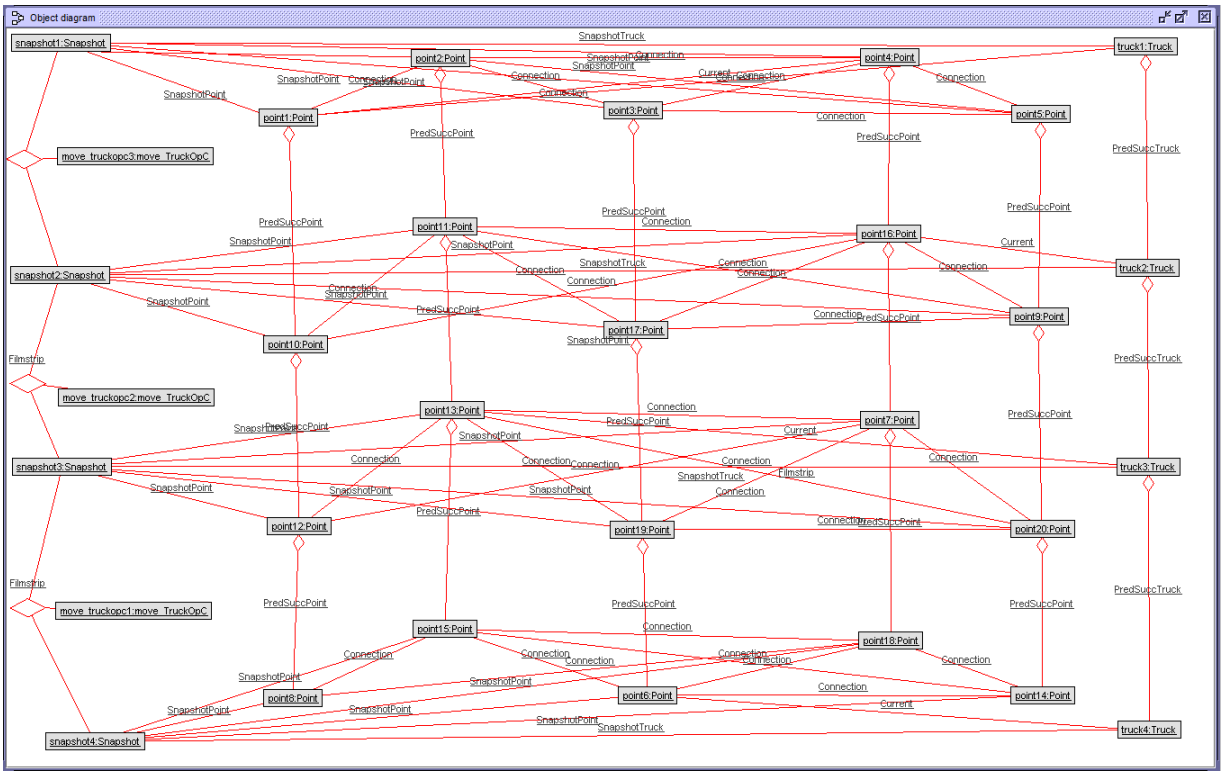


Abbildung 4.20: Objektdiagramm mit fünf Städten und drei Bewegungen

4.4.3 Ausführungszeiten

Die Tabelle 4.12 zeigt die Zeiten, die es jeweils gedauert hat, bis der Modelvalidator ein Objektdiagramm berechnet hat. Die benötigte Zeit nimmt sowohl durch eine Erhöhung der Anzahl der Objekte, also durch die Vergrößerung des Streckennetzes zu, als auch durch das Hinzukommen weiterer Zustände. Erst bei drei Städten und drei Bewegungen entstand aufgrund der steigenden Umfangs des Suchraums zum ersten Mal eine spürbare Wartezeit von ca. zwei Sekunden.

Auch bei fünf Städten und zwei Bewegungen entstand, obwohl eine Vielzahl an Objekten erzeugt werden muss, keine nennenswerte Wartezeit. Erwartungsgemäß dauert das Finden eines Objektdiagramms mit drei Bewegungen auf dem größten Streckennetz etwas länger. Die solving time ist mit ca. 15 Sekunden jedoch immer noch im praktikablen Rahmen.

Testfall	**1 Move**	**2 Moves**	**3 Moves**
2 Städte	2 ms	17 ms	137 ms
3 Städte	5 ms	236 ms	1916 ms
5 Städte	27 ms	1996 ms	15897 ms

Tabelle 4.12: Lösungszeiten aller Partial Solution Completion-Testfälle

4.5 Negativer Testfall

Es soll nun ein Fall konstruiert werden, in dem der Modelvalidator keine Lösung findet. Der Modelvalidator braucht für die Rückgabe von *Unsatisfiable* i. d. R. etwas länger, als wenn er eine Lösung findet, da der gesamte Suchraum betrachtet werden muss, bevor *Unsatisfiable* zurückgegeben werden kann.

Im ursprünglichen Modell ist es möglich zu bezahlen, auch wenn keine Schuld vorliegt, also kann *debt* <= 0 sein. Nun wird bei der Operation *pay* im Listing 4.17 eine Postcondition hinzugefügt, die besagt, dass die Schuld nach Ausführung der Operation *pay* nicht kleiner Null sein darf. In anderen Worten: Es darf kein Guthaben, also keine negative Schuld, geben.

Listing 4.17: Invariante *debtNotBelowZero*

```
1 post debtNotBelowZero :
2 self.debt >= 0
```

Für diesen Testfall wird auf dem Streckennetz des vorherigen Kapitels aufgebaut, damit keine Initialisierungen durchgeführt werden müssen. Es wird das selbe *.SOIL*-Skript wie für das Streckennetz mit zwei Städten verwendet. Nach dem Skript hat der Truck eine *debt* von 1. In einer Konfigurationsdatei wird nun einmal die Operation *move* und dreimal die Operation *pay* angegeben. Da der *amount* von *pay* immer größer Null sein muss und es sich um einen Integer handelt, ist der kleinste Zahlungsbetrag 1. In dieser Konstellation ist es egal, in welcher Reihenfolge die Operationen ausgeführt werden, es würde immer zu einer debt= -1 kommen. *Truck_debt* ist auf maximal 4 gesetzt, womit es kein Szenario geben sollte, in dem es nicht zu einer negativen Schuld kommt.

Die Tabelle 4.13 zeigt die gesamte Konfiguration für den negativen Testfall.

Testfall	**1 Move, 3 Pay**
Objekte	2 Points, 1 Truck
Konfig	Snapshots: 5..5 Filmstrip: 4..4 Truck: 5..5 *init_TruckOpC*: 0..0 *enter_TruckOpC*: 0..0 *move_TruckOpC*: 1..1 *pay_TruckOpC*: 3..3 *bye_TruckOpC*: 0..0 *Truck_debt_max*: 0..4 SnapshotTruck: 5..5 PredSuccTruck: 4..4 Point: 10..10 SnapshotPoint: 10..10 PredSuccPoint: 8..8 *init_PointOpC*: 0..0 *southConnect_PointOpC*: 0..0 *northConnect_PointOpC*: 0..0

Tabelle 4.13: Konfiguration für negativen Testfall

Es ist zu erwarten, dass der Modelvalidator *Unsatisfiable* ausgibt. Es sollte kein entsprechendes Objektdiagramm geben, da die drei Bezahlvorgänge bei einer Schuld von 1 und nur einer Bewegung im Konflikt mit der Postcondition *debtNotBelowZero* stehen.

Die Ausgabe des Modelvalidators(Listing 4.18) zeigt das erwartete Ergebnis.

Listing 4.18: Konsolenausgabe für negativen Testfall

```
1 15:03:39 INFO  KodkodModelValidator: UNSATISFIABLE
```

5 Fazit und Ausblick

5.1 Fazit

Modellierung stellt einen großen und wichtigen Teil der ersten Phasen (und darüber hinaus) der Softwareentwicklung dar. Daher ist es bedeutend das betroffene Modell zu validieren. Um ein möglichst vollständiges Bild der geplanten Software zu erhalten, gilt es dabei auch die Dynamik des Systems zu beachten. Dies gelingt, wie diese Arbeit gezeigt hat, mit Hilfe des Filmstrip-Modells und des Modelvalidators. In dieser Arbeit wurde das bestehende UML und OCL Toll Collect-Modell erweitert und umgewandelt und anschließend die Konsistenz und Validität des Modells geprüft, indem Objektdiagramme entsprechend den vorgegebenen Einschränkungen in der Konfigurationsdatei durch den Modelvalidator erzeugt wurden. Darüber hinaus wurde dann in verschiedenen Testfällen das Verhalten des Systems bei verschiedenen Operationsaufrufen über mehrere Zustände hinweg beobachtet, um festzustellen, ob die dynamischen Eigenschaften des Modells auch wirklich den Erwartungen entsprechen. Dabei kam es nur in den Testfällen mit deaktivierten Frameconditions zu Verhalten, in Form von Attributsausprägungen, wie es im Application-Modell nicht vorgekommen wäre.

Bezüglich des Workflows nach dem in dieser Arbeit vorgegangen wurde, besteht weiterhin Verbesserungspotenzial, was sowohl den Modelvalidator als auch die Filmstrip-Transformation angeht, im Hinblick auf Performance und die Einfachheit der Bedienung bzw. Zugänglichkeit für den Nutzer.

Alles in Allem stellt die Nutzung des USE-Tools in Kombination mit dem Filmstrip-Modell und dem Modelvalidator eine nützliche Möglichkeit dar, das dynamische Verhalten eines Modell zu verstehen, zu überprüfen und zu validieren.

5.2 Ausblick

Eine mögliche Weiterentwicklung des Verfahrens aus dieser Arbeit wäre, die Objektdiagramme im Filmstrip-Modell in Sequenzdiagramme im Application-Modell umzuwandeln. Durch diese gewissermaßen Zurückverwandlung könnte man die im Filmstrip-Modell generierte Abfolge von Zuständen mit dem Ausgangsmodell kompatibel machen. So könnten mehrere aufeinander folgende Zustände im Application-Modell automatisch erzeugt werden. Konkret wäre eine Umwandlung eines .SOIL-Skriptes von einem Zustand im Filmstrip-Modell zu einem .SOIL-Skript, das im Application-Modell eingesetzt werden kann, erstrebenswert.

Des Weiteren könnte untersucht werden, wie sich die Filmstrip-Transformation eines komplizierteren Modells mit mehr Klassen und Operationen auf die Performance des Modelvalidators auswirkt. Dabei wäre interessant herauszufinden, welche Faktoren Einfluss haben und ab welcher Größe eines Modells die Erzeugung von Objektdiagrammen im Filmstrip-Modell zu zeitintensiv wird.

Der Aufwand, der für die Formulierung der Frameconditions nötig ist, ist recht hoch, da jede Framecondition zur Zeit für jede Klasse einzeln definiert wird. Es wäre wünschenswert, wenn sich dieser Aufwand verringert oder sogar komplett wegfallen würde, wenn die Frameconditions direkt aus dem Modell heraus abgeleitet werden könnten.

In dieser Arbeit wurde die Implementierung mit der Filmstrip-Assoziation als ternäre Assoziation verwendet, dies ist nur eine von mehreren Möglichkeiten. Eine andere Möglichkeit wäre beispielsweise zwei Assoziationen zwischen Snapshot und der Operationsaufrufklasse zu haben. So wären jeweils der Ausgangs- und der Folgezustand per Assoziation mit der Zustandsübergangsoperation verbunden. Auf den Ergebnissen dieser Arbeit aufbauend, wird bereits in einer anderen Arbeit mit dem Toll Collect-Modell als Filmstrip-Version weiter gearbeitet, indem die Performance verschiedener Implementierungen des Filmstrip-Modells verglichen werden.

Literaturverzeichnis

Gogolla, Martin/Büttner, Fabian/Richters, Mark: USE: A UML-Based Specification Environment for Validating UML and OCL. Science of Computer Programming, 69 2007, 27–34.

Gogolla, Martin et al.: From Application Models to Filmstrip Models: An Approach to Automatic Validation of Model Dynamics. In **Fill, Hans-Georg/Karagiannis, Dimitris/Reimer, Ulrich (Hrsg.):** Proc. Modellierung (MODELLIERUNG'2014). GI, LNI 225, 2014, 273–288.

Gogolla, Martin et al.: Checking UML and OCL Model Consistency: An Experience Report on a Middle-Sized Case Study. In **Blanchette, Jasmin/Kosmatov, Nikolai (Hrsg.):** Proc. 9th Int. Conf. Tests and Proofs (TAP 2015). Springer, LNCS 9154, 2015a, LNCS paper (8 pages) and addendum (28 pages), 129–136.

Gogolla, Martin et al.: Modeling Behavior with Interaction Diagrams in a UML and OCL Tool. In **Roubtsova, Ella et al. (Hrsg.):** Behavior Modeling: Foundations and Applications, Int. Workshops BM-FA 2009-2014. Springer, LNCS 6368, 2015b, 31–58.

Gogolla, Martin/Hilken, Frank: UML and OCL Transformation Model Analysis: Checking Invariant Independence. In **Amrani, Moussa/Syriani, Eugene/ Wimmer, Manuel (Hrsg.):** Proc. 4th Int. Workshop on Verification of Model Transformation (VOLT'2015). http://ceur-ws.org/: CEUR Proceedings, 2015.

Gogolla, Martin/Hilken, Frank: Model Validation and Verification Options in a Contemporary UML and OCL Analysis Tool. In **Oberweis, Andreas/Reussner, Ralf (Hrsg.):** Proc. Modellierung (MODELLIERUNG'2016). GI, LNI 254, 2016, 203–218.

Gogolla, Martin/Hilken, Frank/Desai, Nisha: On Executing Models by Filmstripping: Evaluating Design Alternatives by Experiments. 2016.

Hilken, Frank: Transformation of UML/OCL Application Models into Filmstrip Models. Diplomarbeit, Universität Bremen, 2013.

Hilken, Frank/Hamann, Lars/Gogolla, Martin: Transformation of UML and OCL Models into Filmstrip Models. In **Ruscio, Davide Di/Varró, Dániel (Hrsg.):** Proc. 7th Int. Conf. Model Transformation (ICMT 2014). Springer, LNCS 8568, 2014, 170–185.

Hilken, Frank et al.: From UML/OCL to Base Models: Transformation Concepts for Generic Validation and Verification. In **Kolovos, Dimitris/Wimmer, Manuel (Hrsg.):** Proc. 8th Int. Conf. Model Transformation (ICMT 2015). Springer, LNCS 9152, 2015, 1–17.

Jacobson, Ivar/Booch, Grady/Rumbaugh, James: Unified Modeling Language User Guide. Addison-Wesley, 1999.

Kahlbrandt, Bernd: Software-Engineering mit der Unified Modeling Language. Springer-Verlag, 2001.

Kuhlmann, Mirco/Gogolla, Martin: From UML and OCL to Relational Logic and Back. In **France, Robert et al. (Hrsg.):** Proc. 15th Int. Conf. Model Driven Engineering Languages and Systems (MoDELS'2012). Springer, Berlin, LNCS 7590, 2012, 415–431.

Kuhlmann, Mirco/Hamann, Lars/Gogolla, Martin: Extensive Validation of OCL Models by Integrating SAT Solving into USE. In **Bishop, Judith/Vallecillo, Antonio (Hrsg.):** Proc. 49th Int. Conf. Objects, Models, Components, and Patterns (TOOLS'2011). Springer, Berlin, LNCS 6705, 2011, 289–305.

Object Management Group (OMG): OMG Unified Modeling Language (OMG UML). 2013 http://www.omg.org/spec/UML/2.5/.

Object Management Group (OMG): Object Constraint Language 2.4. 2014 http://www.omg.org/spec/OCL/2.4/.

Demuth, Birgit: The Dresden OCL Toolkit and its Role in Information Systems Development. In 13th International Conference on Information Systems Development: Methods and Tools, Theory and Practice Conference, Advances in Theory, Practice and Education (ISD'2004). 2004.

Reitmann, Hendrik: Realisierung eines fexibel konfigurierbaren SAT-basierten UML- und OCL-Modellvalidierers. Diplomarbeit, Universität Bremen, 2012.

Richters, Mark: A Precise Approach to Validating UML Models and OCL Constraints. Dissertation, Universität Bremen, 2002.

Rumbaugh, James/Jacobson, Ivar/Booch, Grady: The Unified Modeling Language Reference Manual. Addison-Wesley, 2004.

Salvanos, Alexander: UML 2.5: das umfassende Handbuch. 5. Auflage. Bonn: Galileo Computing, 2015 http://suub-bremen.ciando.com/book/index.cfm/bok_ID/1877236, Online Ressource (7708 KB, 470 S.), ISBN 3836229773 and 9783836229777.

Sommerville, Ian: Software Engineering. Pearson Studium, 2012.

Stachowiak, Herbert: Allgemeine Modelltheorie. Springer-Verlag, 1973.

Yu, Lijun/France, Robert B./Ray, Indrakshi; Czarnecki, Krzysztof et al. (Hrsg.): Scenario-Based Static Analysis of UML Class Models. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008 http://dx.doi.org/10.1007/978-3-540-87875-9_17, 234–248, ISBN 978–3–540–87875–9.

Anhang

I Toll Collect

I.1 Vollständiges Toll Collect-Modell

```
1  ------------------------------------------------------ model TollCollect
2  model TollCollect
3  ------------------------------------------------------------ class Truck
4  class Truck
5  attributes
6    num:String             init: ''
7    debt:Integer           init: 0
8  operations
9    init(aNum:String)
10     begin self.num:=aNum end
11   enter(entry:Point)
12     begin insert (self,entry) into Current; self.debt:=1;
13     end
14   move(target:Point)
15     begin
16     self.debt:=self.debt+1; delete (self,self.current) from Current;
17     insert (self,target) into Current end
18   pay(amount:Integer)
19     begin self.debt:=self.debt-amount end
20   bye():Integer
21     begin delete (self,self.current) from Current;
22     result:=self.debt.abs(); self.debt:=0 end
23   ---------------------------------------------------------------------
24   numIsKey():Boolean=
25     Truck.allInstances->forAll(self,self2|
26       self<>self2 implies self.num<>self2.num)
27   ---------------------------------------------------------------------
28 statemachines
29   psm TruckLife
30   states
31     prenatal:initial
32     born   [num='']
33     noDebt [num<>'' and current->isEmpty]
34     debt   [num<>'' and current->notEmpty]
35   transitions
36     prenatal -> born   { create }
```

```
37      born     -> noDebt { init() }
38      noDebt   -> debt   { enter() }
39      debt     -> debt   { move() }
40      debt     -> debt   { pay() }
41      debt     -> noDebt { bye() }
42    end
43  end
44  ------------------------------------------------------------ class Point
45  class Point
46  attributes
47    name:String init: ''
48    northSize:Integer derived: self.northPlus()->size()
49    southSize:Integer derived: self.southPlus()->size()
50  operations
51    init(aName:String)
52      begin self.name:=aName end
53    northConnect(aNorth:Point)
54      begin insert (aNorth,self) into Connection end
55    southConnect(aSouth:Point)
56      begin insert (self,aSouth) into Connection end
57    ----------------------------------------------------------------------
58    northPlus():Set(Point)=self.north->closure(p|p.north)
59    southPlus():Set(Point)=self.south->closure(p|p.south)
60    ----------------------------------------------------------------------
61    nameIsKey():Boolean=
62      Point.allInstances->forAll(self,self2|
63        self<>self2 implies self.name<>self2.name)
64    noCycles():Boolean=
65      Point.allInstances->forAll(self|
66        not(self.northPlus()->includes(self)))
67    ----------------------------------------------------------------------
68  statemachines
69    psm PointLife
70    states
71      prenatal:initial
72      born    [name='']
73      growing [name<>'']
74    transitions
75      prenatal -> born    { create }
76      born     -> growing { init() }
77      growing  -> growing { northConnect() }
78      growing  -> growing { southConnect() }
79    end
80  end
81  ------------------------------------------------- association Current
```

```
82 association Current between
83   Truck[0..*] role truck
84   Point[0..1] role current
85 end
86 ------------------------------------------------ association Connection
87 association Connection between
88   Point[0..*] role north
89   Point[0..*] role south
90 end
91 ------------------------------------------------------------ constraints
92 constraints
93 ------------------------------------------------------------ invariants
94 context Truck inv numIsKeyInv:
95   numIsKey()
96 context Point inv nameIsKeyInv:
97   nameIsKey()
98 context Point inv noCyclesInv:
99   noCycles()
100 ------------------------------------------------------------ Point::init
101 context Point::init(aName:String)
102 pre freshPoint:
103   self.name='' and self.north->isEmpty and self.south->isEmpty
104 pre aNameOk:
105   aName<>'' and aName<>null
106 post nameAssigned:
107   aName=self.name
108 post allPointInvs:
109   nameIsKey() and noCycles()
110 post truckUnchanged:
111   Truck.allInstances@pre=Truck.allInstances and
112   Truck.allInstances ->forAll(t| (t.num@pre=t.num) and
113   (t.debt@pre=t.debt) and
114   (t.current@pre=t.current))
115 post pointUnchangedExcept: let x= self in
116   Point.allInstances@pre = Point.allInstances and
117   Point.allInstances-> forAll (p| (p<> x implies p.name@pre=p.name) and
118   (p.truck@pre=p.truck))
119
120 ------------------------------------------------ Point::northConnect
121 context Point::northConnect(aNorth:Point)
122 pre aNorthDefined:
123   aNorth.isDefined
124 pre freshConnection:
125   self.north->excludes(aNorth) and self.south->excludes(aNorth)
126 pre notSelfLink:
```

```
127   self<>aNorth
128 pre noCycleIntroduced:
129   aNorth.northPlus()->excludes(self)
130 post connectionAssigned:
131   self.north->includes(aNorth)
132 post allPointInvs:
133   nameIsKey() and noCycles()
134 post TruckUnchanged:
135   Truck.allInstances@pre=Truck.allInstances and
136   Truck.allInstances->forAll(t| (t.num@pre=t.num) and
137   (t.debt@pre=t.debt) and
138   (t.current@pre=t.current))
139 post PointUnchangedExceptSet: let x=Set{aNorth, self} in
140   Point.allInstances@pre=Point.allInstances and
141   Point.allInstances->forAll(p| (p.name@pre=p.name) and
142   (p.truck@pre=p.truck) and
143   (x->excludes(p) implies p.north@pre=p.north) and
144   (x->excludes(p) implies p.south@pre=p.south))
145
146 ------------------------------------------------------------ Point::southConnect
147 context Point::southConnect(aSouth:Point)
148 pre aSouthDefined:
149   aSouth.isDefined
150 pre freshConnection:
151   self.south->excludes(aSouth) and self.south->excludes(aSouth)
152 pre notSelfLink:
153   self<>aSouth
154 pre noCycleIntroduced:
155   aSouth.southPlus()->excludes(self)
156 post connectionAssigned:
157   self.south->includes(aSouth)
158 post allPointInvs:
159   nameIsKey() and noCycles()
160 post TruckUnchanged:
161   Truck.allInstances@pre=Truck.allInstances and
162   Truck.allInstances->forAll(t|(t.num@pre=t.num) and
163   (t.debt@pre=t.debt) and
164   (t.current@pre=t.current))
165 post PointUnchangedExceptSet: let x=Set{aSouth, self} in
166   Point.allInstances@pre=Point.allInstances and
167   Point.allInstances->forAll(p| (p.name@pre=p.name) and
168   (p.truck@pre=p.truck) and
169   (x->excludes(p) implies p.north@pre=p.north) and
170   (x->excludes(p) implies p.south@pre=p.south))
171
```

```
172 ------------------------------------------------------------ Truck::init
173 context Truck::init(aNum:String)
174 pre freshTruck:
175   self.num='' and self.debt=0 and
176   self.current->isEmpty
177 pre aNumOk:
178   aNum<>'' and aNum<>null
179 post numAssigned:
180   aNum=self.num
181 post allTruckInvs:
182   numIsKey()
183 post PointUnchanged:
184   Point.allInstances@pre=Point.allInstances and
185   Point.allInstances->forAll(p|(p.name@pre=p.name) and
186     (p.north@pre=p.north) and
187     (p.south@pre=p.south) and
188   (p.truck@pre=p.truck))
189 post truckUnchangedExcept: let x= self@pre in
190   Truck.allInstances@pre = Truck.allInstances and
191   Truck.allInstances-> forAll (t| (t<> x implies t.num@pre=t.num)
192   and (t.debt@pre=t.debt) and
193   (t.current@pre=t.current))
194
195 ------------------------------------------------------------ Truck::enter
196 context Truck::enter(entry:Point)
197 pre noDebt:
198   0=self.debt
199 pre currentEmpty:
200   self.current->isEmpty
201 pre entryOk:
202   entry<>null
203 pre numOk:
204   self.num<>'' and self.num<>null
205 post debtAssigned:
206   1=self.debt
207 post currentAssigned:
208   entry=self.current
209 post allTruckInvs:
210   numIsKey()
211 post PointUnchangedExcept: let x=entry in
212   Point.allInstances@pre=Point.allInstances and
213   Point.allInstances->forAll(p| (p.name@pre=p.name) and
214   (p.north@pre=p.north) and
215   (p.south@pre=p.south) and
216   (p<>x implies p.truck@pre=p.truck))
```

```
217 post truckUnchangedExcept: let x= self in
218   Truck.allInstances@pre = Truck.allInstances and
219   Truck.allInstances-> forAll (t| (t.num@pre=t.num) and
220   (t<>x implies t.debt@pre=t.debt) and (
221   t<>x implies t.current@pre=t.current))
222
223 ------------------------------------------------------------ Truck::move
224 context Truck::move(target:Point)
225 pre numOk :
226   self.num<>'' and self.num<>null
227 pre currentExists:
228   self.current->notEmpty
229 pre targetReachable: self.current.north->union(self.current.south)->includes(target)
230 post debtIncreased:
231   self.debt@pre+1=self.debt
232 post currentAssigned:
233   target=self.current
234 post allTruckInvs:
235   numIsKey()
236 post PointUnchangedExceptSet: let x=Set{target, self.current@pre} in
237   Point.allInstances@pre=Point.allInstances and
238   Point.allInstances->forAll(p| (p.name@pre=p.name) and
239   (p.north@pre=p.north) and
240   (p.south@pre=p.south) and
241   (x->excludes(p) implies p.truck@pre=p.truck))
242 post truckUnchangedExcept: let x= self in
243   Truck.allInstances@pre = Truck.allInstances and
244   Truck.allInstances-> forAll (t| (t.num@pre=t.num) and
245   (t<>x implies t.debt@pre=t.debt) and
246   (t<>x implies t.current@pre=t.current))
247 post debtNotBelowZero:
248 self.debt >= 0
249
250 ------------------------------------------------------------ Truck::pay
251 context Truck::pay(amount:Integer)
252 pre numOk :
253   self.num<>'' and self.num<>null
254 pre amountPositive:
255   amount>0
256 pre currentExists:
257   self.current->notEmpty
258 post debtReduced:
259   (self.debt@pre-amount)=(self.debt)
260 post allTruckInvs:
261   numIsKey()
```

```
262   --nicht mehr bezahlen als die Schuld
263 post debtNotBelowZero:
264 self.debt >= 0
265 post PointUnchanged:
266   Point.allInstances@pre=Point.allInstances and
267   Point.allInstances->forAll(p|(p.name@pre=p.name) and
268   (p.truck@pre=p.truck) and
269   (p.north@pre=p.north) and
270   (p.south@pre=p.south))
271 post truckUnchangedExcept: let x = self in
272   Truck.allInstances@pre=Truck.allInstances and
273   Truck.allInstances->forAll(t|
274     (t.num@pre=t.num) and (t<>x implies t.debt@pre=t.debt) and
275     (t.current@pre=t.current))
276
277 ------------------------------------------------------------ Truck::bye
278 context Truck::bye():Integer
279 pre numOk :
280   self.num<>'' and self.num<>null
281 pre currentExists:
282   self.current->notEmpty
283 pre noDebt:
284   self.debt<=0
285 post resultEqualsOverPayment:
286   self.debt@pre.abs()=result
287 post zeroDebt:
288   self.debt=0
289 post currentEmpty:
290   self.current->isEmpty
291 post allTruckInvs:
292   numIsKey()
293   post PointUnchangedExcept: let x=self.current@pre in
294   Point.allInstances@pre=Point.allInstances and
295   Point.allInstances->forAll(p| (p.name@pre=p.name) and
296   (p.north@pre=p.north) and
297   (p.south@pre=p.south) and
298   (p<>x implies p.truck@pre=p.truck))
299 post truckUnchangedExcept : let x = self in
300   Truck.allInstances@pre=Truck.allInstances and
301   Truck.allInstances->forAll(t|
302     (t.num@pre=t.num) and
303     (t<>x implies t.debt@pre=t.debt) and
304   (t<>x implies t.current@pre=t.current))
305 ------------------------------------------------------------------------
```

I.2 Transformiertes Toll Collect-Modell

```
model TollCollect


aggregation PredSuccTruck between
  Truck[0..1] role pred
  Truck[0..1] role succ
end

aggregation PredSuccPoint between
  Point[0..1] role pred
  Point[0..1] role succ
end

class southConnect_PointOpC < PointOpC
attributes
  aSouth : Point
end

association Filmstrip between
  Snapshot[0..1] role pred
  Snapshot[0..1] role succ
  OperationCall[0..1]
end

class pay_TruckOpC < TruckOpC
attributes
  amount : Integer
end

class northConnect_PointOpC < PointOpC
attributes
  aNorth : Point
end

class init_PointOpC < PointOpC
attributes
  aName : String
end

abstract class PointOpC < OperationCall
attributes
  aSelf : Point
end
```

```
44
45 class Snapshot
46 operations
47   pred() : Snapshot =
48     if (self.pred->size() = 1) then self.pred->any( $elem0:Snapshot | true ) else null
       endif
49   succ() : Snapshot =
50     if (self.succ->size() = 1) then self.succ->any( $elem0:Snapshot | true ) else null
       endif
51 end
52
53 class init_TruckOpC < TruckOpC
54 attributes
55   aNum : String
56 end
57
58 abstract class TruckOpC < OperationCall
59 attributes
60   aSelf : Truck
61 end
62
63 abstract class OperationCall
64 operations
65   pred() : Snapshot =
66     if (self.pred->size() = 1) then self.pred->any( $elem0:Snapshot | true ) else null
       endif
67   succ() : Snapshot =
68     if (self.succ->size() = 1) then self.succ->any( $elem0:Snapshot | true ) else null
       endif
69 end
70
71 association SnapshotTruck between
72   Truck[*]
73   Snapshot[1]
74 end
75
76 class bye_TruckOpC < TruckOpC
77 attributes
78   result : Integer
79 end
80
81 association SnapshotPoint between
82   Point[*]
83   Snapshot[1]
84 end
```

```
85
86 class enter_TruckOpC < TruckOpC
87 attributes
88   entry : Point
89 end
90
91 class move_TruckOpC < TruckOpC
92 attributes
93   target : Point
94 end
95
96 class Truck
97 attributes
98   num : String init : ''
99   debt : Integer init : 0
100 operations
101   init(aNum : String)
102     begin
103       self.num := aNum
104     end
105   enter(entry : Point)
106     begin
107       insert (self,entry) into Current;
108       self.debt := 1;
109     end
110   move(target : Point)
111     begin
112       self.debt := (self.debt + 1);
113       delete (self,self.current) from Current;
114       insert (self,target) into Current;
115     end
116   pay(amount : Integer)
117     begin
118       self.debt := (self.debt - amount)
119     end
120   bye() : Integer
121     begin
122       delete (self,self.current) from Current;
123       result := self.debt.abs;
124       self.debt := 0;
125     end
126   numIsKey() : Boolean =
127     self.snapshot.truck->forAll( self:Truck, self2:Truck | ((self <> self2) implies (self
        .num <> self2.num)) )
128 end
```

```
129
130 class Point
131 attributes
132   name : String init : ''
133   northSize : Integer derived : self.northPlus()->size()
134   southSize : Integer derived : self.southPlus()->size()
135 operations
136   init(aName : String)
137     begin
138       self.name := aName
139     end
140   northConnect(aNorth : Point)
141     begin
142       insert (aNorth,self) into Connection
143     end
144   southConnect(aSouth : Point)
145     begin
146       insert (self,aSouth) into Connection
147     end
148   northPlus() : Set(Point) =
149     self.north->closure( p:Point | p.north )
150   southPlus() : Set(Point) =
151     self.south->closure( p:Point | p.south )
152   nameIsKey() : Boolean =
153     self.snapshot.point->forAll( self:Point, self2:Point | ((self <> self2) implies (self
        .name <> self2.name)) )
154   noCycles() : Boolean =
155     self.snapshot.point->forAll( self:Point | not self.northPlus()->includes(self) )
156 end
157
158 association Current between
159   Truck[*]
160   Point[0..1] role current
161 end
162
163 association Connection between
164   Point[*] role north
165   Point[*] role south
166 end
167
168 constraints
169 context OperationCall inv assocClassBehavior:
170   (((self.pred->size() = 1) and (self.succ->size() = 1)) and OperationCall.allInstances()
       ->forAll( op:OperationCall | (((self.pred = op.pred) and (self.succ = op.succ))
       implies (self = op)) ))
```

```
171
172 context Point inv validLinkingConnection:
173   self.south->forAll( c:Point | (c.snapshot = self.snapshot) )
174
175 context Point inv validSnapshotLinking:
176   (self.succ.isDefined() implies (self.succ.snapshot = self.snapshot.succ()))
177
178 context PointOpC inv aSelfDefined:
179   self.aSelf.isDefined()
180
181 context PointOpC inv aSelfInPred:
182   (self.aSelf.snapshot = self.pred())
183
184 context Snapshot inv cycleFree:
185   Set{self.succ()}->closure( s:Snapshot | s.succ() )->excludes(self)
186
187 context Snapshot inv oneFilmstrip:
188   ((Snapshot.allInstances()->select( s:Snapshot | s.pred().isUndefined() )->size() = 1)
      and (Snapshot.allInstances()->select( s:Snapshot | s.succ().isUndefined() )->size() =
       1))
189
190 context Truck inv validLinkingCurrent:
191   (self.current.isDefined() implies (self.snapshot = self.current.snapshot))
192
193 context Truck inv validSnapshotLinking:
194   (self.succ.isDefined() implies (self.succ.snapshot = self.snapshot.succ()))
195
196 context TruckOpC inv aSelfDefined:
197   self.aSelf.isDefined()
198
199 context TruckOpC inv aSelfInPred:
200   (self.aSelf.snapshot = self.pred())
201
202 context bye_TruckOpC inv post_PointUnchangedExcept:
203   (let x:Point = aSelf.current.succ in ((self.pred().point->collectNested( p1:Point | p1.
      succ )->asSet() = self.succ().point) and self.succ().point->forAll( p:Point | ((((p.
      pred.name = p.name) and (p.pred.north->collectNested( p1:Point | p1.succ )->asSet() =
       p.north)) and (p.pred.south->collectNested( p1:Point | p1.succ )->asSet() = p.south)
      ) and ((p <> x) implies (p.pred.truck->collectNested( t1:Truck | t1.succ )->asSet() =
       p.truck))) )))
204
205 context bye_TruckOpC inv post_allTruckInvs:
206   aSelf.succ.numIsKey()
207
208 context bye_TruckOpC inv post_currentEmpty:
```

```
209   aSelf.succ.current->isEmpty()
210
211 context bye_TruckOpC inv post_resultEqualsOverPayment:
212   (aSelf.debt.abs() = result)
213
214 context bye_TruckOpC inv post_truckUnchangedExcept:
215   (let x:Truck = aSelf.succ in ((self.pred().truck->collectNested( t1:Truck | t1.succ )->
        asSet() = self.succ().truck) and self.succ().truck->forAll( t:Truck | (((t.pred.num =
         t.num) and ((t <> x) implies (t.pred.debt = t.debt))) and ((t <> x) implies (t.pred.
        current.succ = t.current))) )))
216
217 context bye_TruckOpC inv post_zeroDebt:
218   (aSelf.succ.debt = 0)
219
220 context bye_TruckOpC inv pre_currentExists:
221   aSelf.current->notEmpty()
222
223 context bye_TruckOpC inv pre_noDebt:
224   (aSelf.debt <= 0)
225
226 context bye_TruckOpC inv pre_numOk:
227   ((aSelf.num <> '') and (aSelf.num <> null))
228
229 context enter_TruckOpC inv paramEntryInPred:
230   (not self.entry.isUndefined() implies (self.entry.snapshot = self.pred()))
231
232 context enter_TruckOpC inv post_PointUnchangedExcept:
233   (let x:Point = entry.succ in ((self.pred().point->collectNested( p1:Point | p1.succ )->
        asSet() = self.succ().point) and self.succ().point->forAll( p:Point | ((((p.pred.name
         = p.name) and (p.pred.north->collectNested( p1:Point | p1.succ )->asSet() = p.north)
        ) and (p.pred.south->collectNested( p1:Point | p1.succ )->asSet() = p.south)) and ((p
         <> x) implies (p.pred.truck->collectNested( t1:Truck | t1.succ )->asSet() = p.truck)
        )) )))
234
235 context enter_TruckOpC inv post_allTruckInvs:
236   aSelf.succ.numIsKey()
237
238 context enter_TruckOpC inv post_currentAssigned:
239   (entry.succ = aSelf.succ.current)
240
241 context enter_TruckOpC inv post_debtAssigned:
242   (1 = aSelf.succ.debt)
243
244 context enter_TruckOpC inv post_truckUnchangedExcept:
245   (let x:Truck = aSelf.succ in ((self.pred().truck->collectNested( t1:Truck | t1.succ )->
```

```
      asSet() = self.succ().truck) and self.succ().truck->forAll( t:Truck | (((t.pred.num =
       t.num) and ((t <> x) implies (t.pred.debt = t.debt))) and ((t <> x) implies (t.pred.
      current.succ = t.current))) )))
246
247 context enter_TruckOpC inv pre_currentEmpty:
248   aSelf.current->isEmpty()
249
250 context enter_TruckOpC inv pre_entryOk:
251   (entry <> null)
252
253 context enter_TruckOpC inv pre_noDebt:
254   (0 = aSelf.debt)
255
256 context enter_TruckOpC inv pre_numOk:
257   ((aSelf.num <> '') and (aSelf.num <> null))
258
259 context init_PointOpC inv post_allPointInvs:
260   (aSelf.succ.nameIsKey() and aSelf.succ.noCycles())
261
262 context init_PointOpC inv post_nameAssigned:
263   (aName = aSelf.succ.name)
264
265 context init_PointOpC inv post_pointUnchangedExcept:
266   (let x:Point = aSelf.succ in ((self.pred().point->collectNested( p1:Point | p1.succ )->
      asSet() = self.succ().point) and self.succ().point->forAll( p:Point | (((p <> x)
      implies (p.pred.name = p.name)) and (p.pred.truck->collectNested( t1:Truck | t1.succ
      )->asSet() = p.truck)) )))
267
268 context init_PointOpC inv post_truckUnchanged:
269   ((self.pred().truck->collectNested( t1:Truck | t1.succ )->asSet() = self.succ().truck)
      and self.succ().truck->forAll( t:Truck | (((t.pred.num = t.num) and (t.pred.debt = t.
      debt)) and (t.pred.current.succ = t.current)) ))
270
271 context init_PointOpC inv pre_aNameOk:
272   ((aName <> '') and (aName <> null))
273
274 context init_PointOpC inv pre_freshPoint:
275   (((aSelf.name = '') and aSelf.north->isEmpty()) and aSelf.south->isEmpty())
276
277 context init_TruckOpC inv post_PointUnchanged:
278   ((self.pred().point->collectNested( p1:Point | p1.succ )->asSet() = self.succ().point)
      and self.succ().point->forAll( p:Point | ((((p.pred.name = p.name) and (p.pred.north
      ->collectNested( p1:Point | p1.succ )->asSet() = p.north)) and (p.pred.south->
      collectNested( p1:Point | p1.succ )->asSet() = p.south)) and (p.pred.truck->
      collectNested( t1:Truck | t1.succ )->asSet() = p.truck)) ))
```

```
279
280 context init_TruckOpC inv post_allTruckInvs:
281   aSelf.succ.numIsKey()
282
283 context init_TruckOpC inv post_numAssigned:
284   (aNum = aSelf.succ.num)
285
286 context init_TruckOpC inv post_truckUnchangedExcept:
287   (let x:Truck = aSelf.succ in ((self.pred().truck->collectNested( t1:Truck | t1.succ )->
        asSet() = self.succ().truck) and self.succ().truck->forAll( t:Truck | ((((t <> x)
        implies (t.pred.num = t.num)) and (t.pred.debt = t.debt)) and (t.pred.current.succ =
        t.current)) )))
288
289 context init_TruckOpC inv pre_aNumOk:
290   ((aNum <> '') and (aNum <> null))
291
292 context init_TruckOpC inv pre_freshTruck:
293   (((aSelf.num = '') and (aSelf.debt = 0)) and aSelf.current->isEmpty())
294
295 context move_TruckOpC inv paramTargetInPred:
296   (not self.target.isUndefined() implies (self.target.snapshot = self.pred()))
297
298 context move_TruckOpC inv post_PointUnchangedExceptSet:
299   (let x:Set(Point) = Set{target.succ, aSelf.current.succ} in ((self.pred().point->
        collectNested( p1:Point | p1.succ )->asSet() = self.succ().point) and self.succ().
        point->forAll( p:Point | ((((p.pred.name = p.name) and (p.pred.north->collectNested(
        p1:Point | p1.succ )->asSet() = p.north)) and (p.pred.south->collectNested( p1:Point
        | p1.succ )->asSet() = p.south)) and (x->excludes(p) implies (p.pred.truck->
        collectNested( t1:Truck | t1.succ )->asSet() = p.truck))) )))
300
301 context move_TruckOpC inv post_allTruckInvs:
302   aSelf.succ.numIsKey()
303
304 context move_TruckOpC inv post_currentAssigned:
305   (target.succ = aSelf.succ.current)
306
307 context move_TruckOpC inv post_debtIncreased:
308   ((aSelf.debt + 1) = aSelf.succ.debt)
309
310 context move_TruckOpC inv post_debtNotBelowZero:
311   (aSelf.succ.debt >= 0)
312
313 context move_TruckOpC inv post_truckUnchangedExcept:
314   (let x:Truck = aSelf.succ in ((self.pred().truck->collectNested( t1:Truck | t1.succ )->
        asSet() = self.succ().truck) and self.succ().truck->forAll( t:Truck | (((t.pred.num =
```

```
      t.num) and ((t <> x) implies (t.pred.debt = t.debt))) and ((t <> x) implies (t.pred.
    current.succ = t.current))) )))
315
316 context move_TruckOpC inv pre_currentExists:
317   aSelf.current->notEmpty()
318
319 context move_TruckOpC inv pre_numOk:
320   ((aSelf.num <> '') and (aSelf.num <> null))
321
322 context move_TruckOpC inv pre_targetReachable:
323   aSelf.current.north->union(aSelf.current.south)->includes(target)
324
325 context northConnect_PointOpC inv paramANorthInPred:
326   (not self.aNorth.isUndefined() implies (self.aNorth.snapshot = self.pred()))
327
328 context northConnect_PointOpC inv post_PointUnchangedExceptSet:
329   (let x:Set(Point) = Set{aNorth.succ, aSelf.succ} in ((self.pred().point->collectNested(
       p1:Point | p1.succ )->asSet() = self.succ().point) and self.succ().point->forAll( p:
      Point | ((((p.pred.name = p.name) and (p.pred.truck->collectNested( t1:Truck | t1.
      succ )->asSet() = p.truck)) and (x->excludes(p) implies (p.pred.north->collectNested(
       p1:Point | p1.succ )->asSet() = p.north))) and (x->excludes(p) implies (p.pred.south
      ->collectNested( p1:Point | p1.succ )->asSet() = p.south))) )))
330
331 context northConnect_PointOpC inv post_TruckUnchanged:
332   ((self.pred().truck->collectNested( t1:Truck | t1.succ )->asSet() = self.succ().truck)
      and self.succ().truck->forAll( t:Truck | (((t.pred.num = t.num) and (t.pred.debt = t.
      debt)) and (t.pred.current.succ = t.current)) ))
333
334 context northConnect_PointOpC inv post_allPointInvs:
335   (aSelf.succ.nameIsKey() and aSelf.succ.noCycles())
336
337 context northConnect_PointOpC inv post_connectionAssigned:
338   aSelf.succ.north->includes(aNorth.succ)
339
340 context northConnect_PointOpC inv pre_aNorthDefined:
341   aNorth.isDefined()
342
343 context northConnect_PointOpC inv pre_freshConnection:
344   (aSelf.north->excludes(aNorth) and aSelf.south->excludes(aNorth))
345
346 context northConnect_PointOpC inv pre_noCycleIntroduced:
347   aNorth.northPlus()->excludes(aSelf)
348
349 context northConnect_PointOpC inv pre_notSelfLink:
350   (aSelf <> aNorth)
```

```
351
352 context pay_TruckOpC inv post_PointUnchanged:
353   ((self.pred().point->collectNested( p1:Point | p1.succ )->asSet() = self.succ().point)
      and self.succ().point->forAll( p:Point | ((((p.pred.name = p.name) and (p.pred.truck
      ->collectNested( t1:Truck | t1.succ )->asSet() = p.truck)) and (p.pred.north->
      collectNested( p1:Point | p1.succ )->asSet() = p.north)) and (p.pred.south->
      collectNested( p1:Point | p1.succ )->asSet() = p.south)) ))
354
355 context pay_TruckOpC inv post_allTruckInvs:
356   aSelf.succ.numIsKey()
357
358 context pay_TruckOpC inv post_debtNotBelowZero:
359   (aSelf.succ.debt >= 0)
360
361 context pay_TruckOpC inv post_debtReduced:
362   ((aSelf.debt - amount) = aSelf.succ.debt)
363
364 context pay_TruckOpC inv post_truckUnchangedExcept:
365   (let x:Truck = aSelf.succ in ((self.pred().truck->collectNested( t1:Truck | t1.succ )->
      asSet() = self.succ().truck) and self.succ().truck->forAll( t:Truck | (((t.pred.num =
       t.num) and ((t <> x) implies (t.pred.debt = t.debt))) and (t.pred.current.succ = t.
      current)) )))
366
367 context pay_TruckOpC inv pre_amountPositive:
368   (amount > 0)
369
370 context pay_TruckOpC inv pre_currentExists:
371   aSelf.current->notEmpty()
372
373 context pay_TruckOpC inv pre_numOk:
374   ((aSelf.num <> '') and (aSelf.num <> null))
375
376 context southConnect_PointOpC inv paramASouthInPred:
377   (not self.aSouth.isUndefined() implies (self.aSouth.snapshot = self.pred()))
378
379 context southConnect_PointOpC inv post_PointUnchangedExceptSet:
380   (let x:Set(Point) = Set{aSouth.succ, aSelf.succ} in ((self.pred().point->collectNested(
       p1:Point | p1.succ )->asSet() = self.succ().point) and self.succ().point->forAll( p:
      Point | ((((p.pred.name = p.name) and (p.pred.truck->collectNested( t1:Truck | t1.
      succ )->asSet() = p.truck)) and (x->excludes(p) implies (p.pred.north->collectNested(
       p1:Point | p1.succ )->asSet() = p.north))) and (x->excludes(p) implies (p.pred.south
      ->collectNested( p1:Point | p1.succ )->asSet() = p.south))) )))
381
382 context southConnect_PointOpC inv post_TruckUnchanged:
383   ((self.pred().truck->collectNested( t1:Truck | t1.succ )->asSet() = self.succ().truck)
```

```
      and self.succ().truck->forAll( t:Truck | (((t.pred.num = t.num) and (t.pred.debt = t.
      debt)) and (t.pred.current.succ = t.current)) ))
384
385 context southConnect_PointOpC inv post_allPointInvs:
386   (aSelf.succ.nameIsKey() and aSelf.succ.noCycles())
387
388 context southConnect_PointOpC inv post_connectionAssigned:
389   aSelf.succ.south->includes(aSouth.succ)
390
391 context southConnect_PointOpC inv pre_aSouthDefined:
392   aSouth.isDefined()
393
394 context southConnect_PointOpC inv pre_freshConnection:
395   (aSelf.south->excludes(aSouth) and aSelf.south->excludes(aSouth))
396
397 context southConnect_PointOpC inv pre_noCycleIntroduced:
398   aSouth.southPlus()->excludes(aSelf)
399
400 context southConnect_PointOpC inv pre_notSelfLink:
401   (aSelf <> aSouth)
402
403 context self : Truck inv numIsKeyInv:
404   self.numIsKey()
405
406 context self : Point inv nameIsKeyInv:
407   self.nameIsKey()
408
409 context self : Point inv noCyclesInv:
410   self.noCycles()
```

II Konfigurationsdateien der Testfälle

II.1 Initialisierung eines Trucks

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 3
13 Snapshot_max = 3
14
15 Filmstrip_min = 2
16 Filmstrip_max = 2
17
18 init_TruckOpC_min = 1
19 init_TruckOpC_max = 1
20 init_TruckOpC_aNum = Set{'1', ''}
21
22 enter_TruckOpC_min = 1
23 enter_TruckOpC_max = 1
24
25 move_TruckOpC_min = 0
26 move_TruckOpC_max = 0
27
28 pay_TruckOpC_min = 0
29 pay_TruckOpC_max = 0
30
31 bye_TruckOpC_min = 0
32 bye_TruckOpC_max = 0
33
34 Truck_min = 3
35 Truck_max = 3
36
37 SnapshotTruck_min = 3
```

```
38 SnapshotTruck_max = 3
39
40 PredSuccTruck_min = 2
41 PredSuccTruck_max = 2
42
43 Point_min = 6
44 Point_max = 6
45
46 SnapshotPoint_min = 6
47 SnapshotPoint_max = 6
48
49 PredSuccPoint_min = 4
50 PredSuccPoint_max = 4
51
52 init_PointOpC_min = 0
53 init_PointOpC_max = 0
54
55 southConnect_PointOpC_min = 0
56 southConnect_PointOpC_max = 0
57
58 northConnect_PointOpC_min = 0
59 northConnect_PointOpC_max = 0
60
61 Connection_min = 0
62 Connection_max = -1
63
64 Current_min = 0
65 Current_max = -1
```

II.2 Initialisierung zweier Punkte

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 4
13 Snapshot_max = 4
```

```
14
15 Filmstrip_min = 3
16 Filmstrip_max = 3
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20 init_TruckOpC_aNum = Set{'1', ''}
21
22 enter_TruckOpC_min = 0
23 enter_TruckOpC_max = 0
24
25 move_TruckOpC_min = 0
26 move_TruckOpC_max = 0
27
28 pay_TruckOpC_min = 0
29 pay_TruckOpC_max = 0
30
31 bye_TruckOpC_min = 0
32 bye_TruckOpC_max = 0
33
34 Truck_min = 4
35 Truck_max = 4
36
37 SnapshotTruck_min = 4
38 SnapshotTruck_max = 4
39
40 PredSuccTruck_min = 3
41 PredSuccTruck_max = 3
42
43 Point_min = 8
44 Point_max = 8
45
46 SnapshotPoint_min = 8
47 SnapshotPoint_max = 8
48
49 PredSuccPoint_min = 6
50 PredSuccPoint_max = 6
51
52 init_PointOpC_min = 2
53 init_PointOpC_max = 2
54
55 southConnect_PointOpC_min = 1
56 southConnect_PointOpC_max = 1
57
58 northConnect_PointOpC_min = 0
```

```
59 northConnect_PointOpC_max = 0
60
61 Connection_min = 0
62 Connection_max = -1
63
64 Current_min = 0
65 Current_max = -1
66
67 init_PointOpC_post_pointUnchangedExcept inactive
68
69 init_PointOpC_post_truckUnchanged inactive
```

II.3 Alle Initialisierung

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ----------------------------------------------------------------------
11
12 Snapshot_min = 6
13 Snapshot_max = 6
14
15 Filmstrip_min = 5
16 Filmstrip_max = 5
17
18 init_TruckOpC_min = 1
19 init_TruckOpC_max = 1
20 init_TruckOpC_aNum = Set{'1', ''}
21
22 enter_TruckOpC_min = 1
23 enter_TruckOpC_max = 1
24
25 move_TruckOpC_min = 0
26 move_TruckOpC_max = 0
27
28 pay_TruckOpC_min = 0
29 pay_TruckOpC_max = 0
30
```

```
31 bye_TruckOpC_min = 0
32 bye_TruckOpC_max = 0
33
34 Truck_min = 6
35 Truck_max = 6
36
37 SnapshotTruck_min = 6
38 SnapshotTruck_max = 6
39
40 PredSuccTruck_min = 5
41 PredSuccTruck_max = 5
42
43 Point_min = 12
44 Point_max = 12
45
46 SnapshotPoint_min = 12
47 SnapshotPoint_max = 12
48
49 PredSuccPoint_min = 10
50 PredSuccPoint_max = 10
51
52 init_PointOpC_min = 2
53 init_PointOpC_max = 2
54
55 southConnect_PointOpC_min = 1
56 southConnect_PointOpC_max = 1
57
58 northConnect_PointOpC_min = 0
59 northConnect_PointOpC_max = 0
60
61 Connection_min = 0
62 Connection_max = -1
63
64 Current_min = 0
65 Current_max = -1
66
67 init_PointOpC_post_pointUnchangedExcept inactive
68
69 init_PointOpC_post_truckUnchanged inactive
```

II.4 Alle Operationen des Trucks

```
1
2 [default]
```

```
3
4  Integer_min = 0
5  Integer_max = 31
6
7  String_min = 0
8  String_max = 31
9
10 # ------------------------------------------------------------------------
11 Snapshot_min = 6
12 Snapshot_max = 6
13
14 Filmstrip_min = 5
15 Filmstrip_max = 5
16
17 init_TruckOpC_min = 1
18 init_TruckOpC_max = 1
19 init_TruckOpC_aNum = Set{'1', ''}
20
21 enter_TruckOpC_min = 1
22 enter_TruckOpC_max = 1
23
24 move_TruckOpC_min = 1
25 move_TruckOpC_max = 1
26
27 pay_TruckOpC_min = 1
28 pay_TruckOpC_max = 1
29
30 bye_TruckOpC_min = 1
31 bye_TruckOpC_max = 1
32
33 Truck_min = 6
34 Truck_max = 6
35
36 Truck_debt_min = 0
37 Truck_debt_min = -1
38
39 SnapshotTruck_min = 6
40 SnapshotTruck_max = 6
41
42 PredSuccTruck_min = 5
43 PredSuccTruck_max = 5
44
45 Point_min = 12
46 Point_max = 12
47
```

```
48 SnapshotPoint_min = 12
49 SnapshotPoint_max = 12
50
51 PredSuccPoint_min = 10
52 PredSuccPoint_max = 10
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnectPointOpC_min = 0
58 southConnectPointOpC_max = 0
59
60 northConnectPointOpC_min = 0
61 northConnectPointOpC_max = 0
62
63 Current_min = 0
64 Current_min = -1
```

II.5 Alle Operationen von Truck und Point

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 9
13 Snapshot_max = 9
14
15 Filmstrip_min = 8
16 Filmstrip_max = 8
17
18 init_TruckOpC_min = 1
19 init_TruckOpC_max = 1
20 init_TruckOpC_aNum = Set{'1', ''}
21
22 enter_TruckOpC_min = 1
23 enter_TruckOpC_max = 1
24
```

```
25 move_TruckOpC_min = 1
26 move_TruckOpC_max = 1
27
28 pay_TruckOpC_min = 1
29 pay_TruckOpC_max = 1
30
31 bye_TruckOpC_min = 1
32 bye_TruckOpC_max = 1
33
34 Truck_min = 9
35 Truck_max = 9
36
37 SnapshotTruck_min = 9
38 SnapshotTruck_max = 9
39
40 PredSuccTruck_min = 8
41 PredSuccTruck_max = 8
42
43 Point_min = 18
44 Point_max = 18
45
46 SnapshotPoint_min = 18
47 SnapshotPoint_max = 18
48
49 PredSuccPoint_min = 16
50 PredSuccPoint_max = 16
51
52 init_PointOpC_min = 2
53 init_PointOpC_max = 2
54
55 southConnect_PointOpC_min = 1
56 southConnect_PointOpC_max = 1
57
58 northConnect_PointOpC_min = 0
59 northConnect_PointOpC_max = 0
60
61 Connection_min = 0
62 Connection_max = -1
63
64 Current_min = 0
65 Current_min = -1
66
67 init_PointOpC_post_pointUnchangedExcept inactive
68
69 init_PointOpC_post_truckUnchanged inactive
```

III Konfigurationsdateien für Streckennetze

III.1 Skripte zur Erstellung der Streckennetze

III.1.1 Zwei Städte

```
1 -- Script generated by USE 4.2.0
2
3 !new Point('point1')
4 !new Point('point2')
5 !new Truck('truck1')
6 !new Snapshot('snapshot1')
7 !point1.name := 'HB'
8 !point2.name := 'HH'
9 !truck1.num := 'trk'
10 !truck1.debt := 1
11 !insert (truck1,point1) into Current
12 !insert (point2,point1) into Connection
13 !insert (truck1,snapshot1) into SnapshotTruck
14 !insert (point1,snapshot1) into SnapshotPoint
15 !insert (point2,snapshot1) into SnapshotPoint
```

III.1.2 Drei Städte

```
1 -- Script generated by USE 4.2.0
2
3 !new Point('point1')
4 !new Point('point2')
5 !new Point('point3')
6 !new Truck('truck1')
7 !new Snapshot('snapshot1')
8 !point1.name := 'HB'
9 !point2.name := 'HH'
10 !point3.name := 'M'
11 !truck1.num := 'trk'
12 !truck1.debt := 1
13 !insert (truck1,point1) into Current
14 !insert (point2,point1) into Connection
15 !insert (point1,point3) into Connection
```

```
16 !insert (point2,point3) into Connection
17 !insert (point1,snapshot1) into SnapshotPoint
18 !insert (point2,snapshot1) into SnapshotPoint
19 !insert (point3,snapshot1) into SnapshotPoint
20 !insert (truck1,snapshot1) into SnapshotTruck
```

III.1.3 Fünf Städte

```
1 -- Script generated by USE 4.2.0
2
3 !new Point('point1')
4 !new Point('point2')
5 !new Point('point3')
6 !new Point('point4')
7 !new Point('point5')
8 !new Truck('truck1')
9 !new Snapshot('snapshot1')
10 !point1.name := 'HB'
11 !point2.name := 'HH'
12 !point3.name := 'M'
13 !point4.name := 'H'
14 !point5.name := 'K'
15 !truck1.num := 'trk'
16 !truck1.debt := 1
17 !insert (truck1,point1) into Current
18 !insert (point2,point1) into Connection
19 !insert (point1,point4) into Connection
20 !insert (point2,point4) into Connection
21 !insert (point4,point3) into Connection
22 !insert (point4,point5) into Connection
23 !insert (point5,point3) into Connection
24 !insert (point1,snapshot1) into SnapshotPoint
25 !insert (point2,snapshot1) into SnapshotPoint
26 !insert (point3,snapshot1) into SnapshotPoint
27 !insert (point4,snapshot1) into SnapshotPoint
28 !insert (point5,snapshot1) into SnapshotPoint
29 !insert (truck1,snapshot1) into SnapshotTruck
```

III.2 Konfigurationsdateien für Bewegungen

III.2.1 Zwei Städte, eine Bewegung

```
# mv -config automaticDiagramExtraction:=on

[default]

Integer_min = 0
Integer_max = 31

String_min = 0
String_max = 31

# ------------------------------------------------------------------------

Snapshot_min = 2
Snapshot_max = 2

Filmstrip_min = 1
Filmstrip_max = 1

init_TruckOpC_min = 0
init_TruckOpC_max = 0

enter_TruckOpC_min = 0
enter_TruckOpC_max = 0

move_TruckOpC_min = 1
move_TruckOpC_max = 1

pay_TruckOpC_min = 0
pay_TruckOpC_max = 0

bye_TruckOpC_min = 0
bye_TruckOpC_max = 0

Truck_min = 2
Truck_max = 2

Truck_debt_min = 0
Truck_debt_min = 1

SnapshotTruck_min = 2
SnapshotTruck_max = 2

PredSuccTruck_min = 1
PredSuccTruck_max = 1

```

```
Point_min = 4
Point_max = 4

SnapshotPoint_min = 4
SnapshotPoint_max = 4

PredSuccPoint_min = 2
PredSuccPoint_max = 2

init_PointOpC_min = 0
init_PointOpC_max = 0

southConnect_PointOpC_min = 0
southConnect_PointOpC_max = 0

northConnect_PointOpC_min = 0
northConnect_PointOpC_max = 0

Connection_min = 0
Connection_max = -1

Current_min = 0
Current_max = -1
```

III.2.2 Zwei Städte, zwei Bewegung

```

[default]

Integer_min = 0
Integer_max = 31

String_min = 0
String_max = 31

# ------------------------------------------------------------------------

Snapshot_min = 3
Snapshot_max = 3

Filmstrip_min = 2
Filmstrip_max = 2

init_TruckOpC_min = 0
```

```
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 2
25 move_TruckOpC_max = 2
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 3
34 Truck_max = 3
35
36 Truck_debt_min = 0
37 Truck_debt_min = 2
38
39 SnapshotTruck_min = 3
40 SnapshotTruck_max = 3
41
42 PredSuccTruck_min = 2
43 PredSuccTruck_max = 2
44
45 Point_min = 6
46 Point_max = 6
47
48 SnapshotPoint_min = 6
49 SnapshotPoint_max = 6
50
51 PredSuccPoint_min = 4
52 PredSuccPoint_max = 4
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
```

```
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.3 Zwei Städte, drei Bewegung

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 4
13 Snapshot_max = 4
14
15 Filmstrip_min = 3
16 Filmstrip_max = 3
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 3
25 move_TruckOpC_max = 3
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 4
34 Truck_max = 4
35
36 Truck_debt_min = 0
37 Truck_debt_min = 3
```

```
38
39 SnapshotTruck_min = 4
40 SnapshotTruck_max = 4
41
42 PredSuccTruck_min = 3
43 PredSuccTruck_max = 3
44
45 Point_min = 8
46 Point_max = 8
47 SnapshotPoint_min = 8
48 SnapshotPoint_max = 8
49
50 PredSuccPoint_min = 6
51 PredSuccPoint_max = 6
52
53 init_PointOpC_min = 0
54 init_PointOpC_max = 0
55
56 southConnect_PointOpC_min = 0
57 southConnect_PointOpC_max = 0
58
59 northConnect_PointOpC_min = 0
60 northConnect_PointOpC_max = 0
61
62 Connection_min = 0
63 Connection_max = -1
64
65 Current_min = 0
66 Current_max = -1
```

III.2.4 Drei Städte, eine Bewegung

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 2
```

```
13 Snapshot_max = 2
14
15 Filmstrip_min = 1
16 Filmstrip_max = 1
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 1
25 move_TruckOpC_max = 1
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 2
34 Truck_max = 2
35
36 Truck_debt_min = 0
37 Truck_debt_min = 2
38
39 SnapshotTruck_min = 2
40 SnapshotTruck_max = 2
41
42 PredSuccTruck_min = 1
43 PredSuccTruck_max = 1
44
45 Point_min = 6
46 Point_max = 6
47
48 SnapshotPoint_min = 6
49 SnapshotPoint_max = 6
50
51 PredSuccPoint_min = 3
52 PredSuccPoint_max = 3
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
```

```
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.5 Drei Städte, zwei Bewegungen

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 3
13 Snapshot_max = 3
14
15 Filmstrip_min = 2
16 Filmstrip_max = 2
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 2
25 move_TruckOpC_max = 2
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
```

```
32
33 Truck_min = 3
34 Truck_max = 3
35
36 Truck_debt_min = 0
37 Truck_debt_min = 2
38
39 SnapshotTruck_min = 3
40 SnapshotTruck_max = 3
41
42 PredSuccTruck_min = 2
43 PredSuccTruck_max = 2
44
45 Point_min = 9
46 Point_max = 9
47
48 SnapshotPoint_min = 9
49 SnapshotPoint_max = 9
50
51 PredSuccPoint_min = 6
52 PredSuccPoint_max = 6
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.6 Drei Städte, drei Bewegungen

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
```

```
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 4
13 Snapshot_max = 4
14
15 Filmstrip_min = 3
16 Filmstrip_max = 3
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 3
25 move_TruckOpC_max = 3
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 4
34 Truck_max = 4
35
36 Truck_debt_min = 0
37 Truck_debt_min = 3
38
39 SnapshotTruck_min = 4
40 SnapshotTruck_max = 4
41
42 PredSuccTruck_min = 3
43 PredSuccTruck_max = 3
44
45 Point_min = 12
46 Point_max = 12
47
48 SnapshotPoint_min = 12
49 SnapshotPoint_max = 12
50
```

```
51 PredSuccPoint_min = 9
52 PredSuccPoint_max = 9
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.7 Fünf Städte, eine Bewegung

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 2
13 Snapshot_max = 2
14
15 Filmstrip_min = 1
16 Filmstrip_max = 1
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 1
```

```
25 move_TruckOpC_max = 1
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 2
34 Truck_max = 2
35
36 Truck_debt_min = 0
37 Truck_debt_min = 1
38
39 SnapshotTruck_min = 2
40 SnapshotTruck_max = 2
41
42 PredSuccTruck_min = 1
43 PredSuccTruck_max = 1
44
45 Point_min = 10
46 Point_max = 10
47
48 SnapshotPoint_min = 10
49 SnapshotPoint_max = 10
50
51 PredSuccPoint_min = 5
52 PredSuccPoint_max = 5
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.8 Fünf Städte, zwei Bewegungen

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------
11
12 Snapshot_min = 3
13 Snapshot_max = 3
14
15 Filmstrip_min = 2
16 Filmstrip_max = 2
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 2
25 move_TruckOpC_max = 2
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 3
34 Truck_max = 3
35
36 Truck_debt_min = 0
37 Truck_debt_min = 2
38
39 SnapshotTruck_min = 3
40 SnapshotTruck_max = 3
41
42 PredSuccTruck_min = 2
43 PredSuccTruck_max = 2
```

```
44
45 Point_min = 15
46 Point_max = 15
47
48 SnapshotPoint_min = 15
49 SnapshotPoint_max = 15
50
51 PredSuccPoint_min = 10
52 PredSuccPoint_max = 10
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

III.2.9 Fünf Städte, drei Bewegungen

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 4
13 Snapshot_max = 4
14
15 Filmstrip_min = 3
16 Filmstrip_max = 3
17
```

```
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 3
25 move_TruckOpC_max = 3
26
27 pay_TruckOpC_min = 0
28 pay_TruckOpC_max = 0
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 4
34 Truck_max = 4
35
36 Truck_debt_min = 0
37 Truck_debt_min = 3
38
39 SnapshotTruck_min = 4
40 SnapshotTruck_max = 4
41
42 PredSuccTruck_min = 3
43 PredSuccTruck_max = 3
44
45 Point_min = 20
46 Point_max = 20
47
48 SnapshotPoint_min = 20
49 SnapshotPoint_max = 20
50
51 PredSuccPoint_min = 15
52 PredSuccPoint_max = 15
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
```

```
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_max = -1
```

IV Negativer Testfall

```
1
2 [default]
3
4 Integer_min = 0
5 Integer_max = 31
6
7 String_min = 0
8 String_max = 31
9
10 # ------------------------------------------------------------------------
11
12 Snapshot_min = 5
13 Snapshot_max = 5
14
15 Filmstrip_min = 4
16 Filmstrip_max = 4
17
18 init_TruckOpC_min = 0
19 init_TruckOpC_max = 0
20
21 enter_TruckOpC_min = 0
22 enter_TruckOpC_max = 0
23
24 move_TruckOpC_min = 1
25 move_TruckOpC_max = 1
26
27 pay_TruckOpC_min = 3
28 pay_TruckOpC_max = 3
29
30 bye_TruckOpC_min = 0
31 bye_TruckOpC_max = 0
32
33 Truck_min = 5
34 Truck_max = 5
```

```
35
36 Truck_debt_min = 0
37 Truck_debt_max = 4
38
39 SnapshotTruck_min = 5
40 SnapshotTruck_max = 5
41
42 PredSuccTruck_min = 4
43 PredSuccTruck_max = 4
44
45 Point_min = 10
46 Point_max = 10
47
48 SnapshotPoint_min = 10
49 SnapshotPoint_max = 10
50
51 PredSuccPoint_min = 8
52 PredSuccPoint_max = 8
53
54 init_PointOpC_min = 0
55 init_PointOpC_max = 0
56
57 southConnect_PointOpC_min = 0
58 southConnect_PointOpC_max = 0
59
60 northConnect_PointOpC_min = 0
61 northConnect_PointOpC_max = 0
62
63 Connection_min = 0
64 Connection_max = -1
65
66 Current_min = 0
67 Current_min = -1
```

V Kommandozeilenausgaben der Testfälle

V.1 Initialisierung eines Trucks

```
use> mv -validate tollcoll_marcel_FS_mv_Truck_InitOps.properties
21:44:17 INFO  ModelTransformator: Start model transformation for 'TollCollect'

21:44:17 INFO  InvariantTransformator: Invariant transformation successful

21:44:17 INFO  ModelTransformator: Model transformation successful

21:44:17 INFO  ModelTransformator: Translation time: 184 ms

21:44:17 INFO  Model: Reset successful

21:44:17 INFO  ConfigurablePlugin: Model configuration successful

21:44:17 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

21:44:18 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
   12...

21:44:18 INFO  KodkodModelValidator: SATISFIABLE

21:44:18 INFO  KodkodModelValidator: Translation time: 290 ms; Solving time: 21 ms

21:44:18 INFO  UseKodkodModelValidator: Create object diagram
```

V.2 Initialisierung zweier Punkte

```
use> mv -validate tollcoll_marcel_FS_mv_Point_InitOps.properties
21:45:52 INFO  Model: Reset successful

21:45:52 INFO  ConfigurablePlugin: Model configuration successful

21:45:52 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

21:45:52 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
   12...

```

```
21:45:53 INFO  KodkodModelValidator: SATISFIABLE

21:45:53 INFO  KodkodModelValidator: Translation time: 227 ms; Solving time: 913 ms

21:45:53 INFO  UseKodkodModelValidator: Create object diagram
```

V.3 Alle Initialisierungsoperationen

```
use> mv -validate tollcoll_marcel_FS_mv_InitOps.properties
21:47:12 INFO  Model: Reset successful

21:47:12 INFO  ConfigurablePlugin: Model configuration successful

21:47:12 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

21:47:12 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
  12...

21:50:37 INFO  KodkodModelValidator: SATISFIABLE

21:50:37 INFO  KodkodModelValidator: Translation time: 750 ms; Solving time: 204406 ms

21:50:37 INFO  UseKodkodModelValidator: Create object diagram
```

V.4 Alle Operationen des Trucks

```
use> mv -validate tollcoll_marcel_FS_mv_AllTruckOps.properties
15:37:18 INFO  ModelTransformator: Start model transformation for 'TollCollect'

15:37:19 INFO  KodkodModelValidatorConfiguration: Set SatSolver to 'MiniSat'.

15:37:19 INFO  KodkodModelValidatorConfiguration: Set bitwidth to 12.

15:37:19 INFO  KodkodModelValidatorConfiguration: Enable automatic extraction of an
  available object diagram.

15:37:19 INFO  InvariantTransformator: Invariant transformation successful

15:37:19 INFO  ModelTransformator: Model transformation successful

15:37:19 INFO  ModelTransformator: Translation time: 423 ms

```

```
15:37:19 INFO  Model: Reset successful

15:37:19 INFO  ConfigurablePlugin: Model configuration successful

15:37:19 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

15:37:19 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
    12...

15:40:17 INFO  KodkodModelValidator: SATISFIABLE

15:40:17 INFO  KodkodModelValidator: Translation time: 874 ms; Solving time: 176753 ms

15:40:17 INFO  UseKodkodModelValidator: Create object diagram
```

V.5 Alle Operationen von Truck und Point

```
use> mv -validate tollcoll_marcel_FS_mv_All_Ops.properties
15:50:55 INFO  ModelTransformator: Start model transformation for 'TollCollect'

15:50:55 INFO  InvariantTransformator: Invariant transformation successful

15:50:55 INFO  ModelTransformator: Model transformation successful

15:50:55 INFO  ModelTransformator: Translation time: 167 ms

15:50:55 INFO  Model: Reset successful

15:50:55 INFO  ConfigurablePlugin: Model configuration successful

15:50:55 INFO  KodkodSolver: Searching solution with SatSolver 'LightSAT4J' and bitwidth
    12...

Exception in thread "Thread-3" java.lang.OutOfMemoryError: Java heap space
        at org.sat4j.minisat.constraints.cnf.WLClause.<init>(Unknown Source)
        at org.sat4j.minisat.constraints.cnf.LearntWLClause.<init>(Unknown Source)
        at org.sat4j.minisat.constraints.MixedDataStructureDanielWL.
          createUnregisteredClause(Unknown Source)
        at org.sat4j.minisat.core.Solver.analyze(Unknown Source)
        at org.sat4j.minisat.core.Solver.search(Unknown Source)
        at org.sat4j.minisat.core.Solver.isSatisfiable(Unknown Source)
        at org.sat4j.minisat.core.Solver.isSatisfiable(Unknown Source)
        at org.sat4j.minisat.core.Solver.isSatisfiable(Unknown Source)
        at kodkod.engine.satlab.SAT4J.solve(SAT4J.java:116)
```

```
26          at kodkod.engine.Solver.solve(Solver.java:188)
27          at org.tzi.kodkod.KodkodSolver.solve(KodkodSolver.java:51)
28          at org.tzi.kodkod.KodkodModelValidator.validate(KodkodModelValidator.java:40)
29          at org.tzi.use.kodkod.plugin.KodkodValidateCmd.validate(KodkodValidateCmd.java
              :100)
30          at org.tzi.use.kodkod.plugin.KodkodValidateCmd.extractConfigureAndValidate(
              KodkodValidateCmd.java:82)
31          at org.tzi.use.kodkod.plugin.KodkodValidateCmd.handleArguments(KodkodValidateCmd.
              java:66)
32          at org.tzi.use.kodkod.plugin.KodkodValidateCmd.performCommand(KodkodValidateCmd.
              java:36)
33          at org.tzi.use.runtime.shell.impl.PluginShellCmd.executeCmd(Unknown Source)
34          at org.tzi.use.main.shell.Shell.processLine(Unknown Source)
35          at org.tzi.use.main.shell.Shell.processLineSafely(Unknown Source)
36          at org.tzi.use.main.shell.Shell.run(Unknown Source)
37          at java.lang.Thread.run(Unknown Source)
```

VI Kommandozeilenausgaben für Streckennetze

VI.1 Zwei Städte, eine Bewegung

```
1 use> mv -validate partialSolution_2Point_1Connect_1Move.properties
2 13:48:15 INFO  ModelTransformator: Start model transformation for 'TollCollect'
3
4 13:48:15 INFO  KodkodModelValidatorConfiguration: Set SatSolver to 'MiniSat'.
5
6 13:48:15 INFO  KodkodModelValidatorConfiguration: Set bitwidth to 12.
7
8 13:48:15 INFO  KodkodModelValidatorConfiguration: Enable automatic extraction of an
    available object diagram.
9
10 13:48:16 INFO  InvariantTransformator: Invariant transformation successful
11
12 13:48:16 INFO  ModelTransformator: Model transformation successful
13
14 13:48:16 INFO  ModelTransformator: Translation time: 465 ms
15
16 13:48:16 INFO  Model: Reset successful
17
18 13:48:16 INFO  ConfigurablePlugin: Model configuration successful
```

```
19
20 13:48:16 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.
21
22 13:48:16 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
     12...
23
24 13:48:16 INFO  KodkodModelValidator: SATISFIABLE
25
26 13:48:16 INFO  KodkodModelValidator: Translation time: 152 ms; Solving time: 2 ms
27
28 13:48:16 INFO  UseKodkodModelValidator: Create object diagram
```

VI.2 Zwei Städte, zwei Bewegung

```
1 use> mv -validate partialSolution_2Point_1Connect_2Move.properties
2 14:00:25 INFO  Model: Reset successful
3
4 14:00:25 INFO  ConfigurablePlugin: Model configuration successful
5
6 14:00:25 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.
7
8 14:00:25 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
    12...
9
10 14:00:25 INFO  KodkodModelValidator: SATISFIABLE
11
12 14:00:25 INFO  KodkodModelValidator: Translation time: 133 ms; Solving time: 17 ms
13
14 14:00:25 INFO  UseKodkodModelValidator: Create object diagram
```

VI.3 Zwei Städte, drei Bewegung

```
1 use> mv -validate partialSolution_2Point_1Connect_3Move.properties
2 14:04:19 INFO  Model: Reset successful
3
4 14:04:19 INFO  ConfigurablePlugin: Model configuration successful
5
6 14:04:19 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.
7
8 14:04:19 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
    12...
9
```

```
14:04:20 INFO  KodkodModelValidator: SATISFIABLE

14:04:20 INFO  KodkodModelValidator: Translation time: 281 ms; Solving time: 137 ms

14:04:20 INFO  UseKodkodModelValidator: Create object diagram
```

VI.4 Drei Städte, eine Bewegung

```
use> mv -validate partialSolution_3Point_3Conect_1Move.properties
14:10:38 INFO  Model: Reset successful

14:10:38 INFO  ConfigurablePlugin: Model configuration successful

14:10:38 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:10:38 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
    12...

14:10:38 INFO  KodkodModelValidator: SATISFIABLE

14:10:38 INFO  KodkodModelValidator: Translation time: 66 ms; Solving time: 5 ms

14:10:38 INFO  UseKodkodModelValidator: Create object diagram
```

VI.5 Drei Städte, zwei Bewegungen

```
use> mv -validate partialSolution_3Point_3Conect_2Move.properties
14:13:50 INFO  Model: Reset successful

14:13:50 INFO  ConfigurablePlugin: Model configuration successful

14:13:50 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:13:50 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
    12...

14:13:51 INFO  KodkodModelValidator: SATISFIABLE

14:13:51 INFO  KodkodModelValidator: Translation time: 195 ms; Solving time: 236 ms

14:13:51 INFO  UseKodkodModelValidator: Create object diagram
```

VI.6 Drei Städte, drei Bewegungen

```
use> mv -validate partialSolution_3Point_3Conect_3Move.properties
14:17:11 INFO  Model: Reset successful

14:17:11 INFO  ConfigurablePlugin: Model configuration successful

14:17:11 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:17:11 INFO  KodkodSolver: Searching solution with SatSolver ‘MiniSat’ and bitwidth
    12...

14:17:13 INFO  KodkodModelValidator: SATISFIABLE

14:17:13 INFO  KodkodModelValidator: Translation time: 448 ms; Solving time: 1916 ms

14:17:13 INFO  UseKodkodModelValidator: Create object diagram
```

VI.7 Fünf Städte, eine Bewegung

```
use> mv -validate partialSolution_5Point_6Conect_1Move.properties
14:17:55 INFO  Model: Reset successful

14:17:55 INFO  ConfigurablePlugin: Model configuration successful

14:17:55 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:17:55 INFO  KodkodSolver: Searching solution with SatSolver ‘MiniSat’ and bitwidth
    12...

14:17:55 INFO  KodkodModelValidator: SATISFIABLE

14:17:55 INFO  KodkodModelValidator: Translation time: 163 ms; Solving time: 27 ms

14:17:55 INFO  UseKodkodModelValidator: Create object diagram
```

VI.8 Fünf Städte, zwei Bewegungen

```
use> mv -validate partialSolution_5Point_6Conect_2Move.properties
14:18:47 INFO  Model: Reset successful

```

```
14:18:47 INFO  ConfigurablePlugin: Model configuration successful

14:18:47 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:18:47 INFO  KodkodSolver: Searching solution with SatSolver ‘MiniSat’ and bitwidth
   12...

14:18:50 INFO  KodkodModelValidator: SATISFIABLE

14:18:50 INFO  KodkodModelValidator: Translation time: 670 ms; Solving time: 1996 ms

14:18:50 INFO  UseKodkodModelValidator: Create object diagram
```

VI.9 Fünf Städte, drei Bewegungen

```
use> mv -validate partialSolution_5Point_6Conect_3Move.properties
14:19:13 INFO  Model: Reset successful

14:19:13 INFO  ConfigurablePlugin: Model configuration successful

14:19:13 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

14:19:13 INFO  KodkodSolver: Searching solution with SatSolver ‘MiniSat’ and bitwidth
   12...

14:19:31 INFO  KodkodModelValidator: SATISFIABLE

14:19:31 INFO  KodkodModelValidator: Translation time: 2036 ms; Solving time: 15897 ms

14:19:31 INFO  UseKodkodModelValidator: Create object diagram
```

VII Kommandozeilenausgaben für negativen Testfall

```
use> mv -config automaticDiagramExtraction:=On

18:32:57 INFO  KodkodModelValidatorConfiguration: Set SatSolver to ‘MiniSat’.

18:32:57 INFO  KodkodModelValidatorConfiguration: Set bitwidth to 12.

```

```
18:32:57 INFO  KodkodModelValidatorConfiguration: Enable automatic extraction of an
  available object diagram.

18:32:57 INFO  KodkodModelValidatorConfiguration: Enable automatic extraction of an
  available object diagram.

use> open PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> -- Script generated by USE 4.2.0
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil>
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !new Point('point1')
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !new Point('point2')
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !new Truck('truck1')
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !new Snapshot('snapshot1')
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !point1.name := 'HB'
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !point2.name := 'HH'
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !truck1.num := 'trk'
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !truck1.debt := 1
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !insert (truck1,point1) into
  Current
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !insert (point2,point1) into
  Connection
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !insert (truck1,snapshot1) into
  SnapshotTruck
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !insert (point1,snapshot1) into
  SnapshotPoint
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil> !insert (point2,snapshot1) into
  SnapshotPoint
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil>
PartialSolution_2Pont_1Connect_1Snap_negativeTest.soil>
use> mv -validate partialSolution_2Point_1Connect_1Move2Pay_negativeTest.properties
18:33:32 INFO  ModelTransformator: Start model transformation for 'TollCollect'

18:33:32 INFO  InvariantTransformator: Invariant transformation successful

18:33:32 INFO  ModelTransformator: Model transformation successful

18:33:32 INFO  ModelTransformator: Translation time: 164 ms

18:33:32 INFO  Model: Reset successful

18:33:32 INFO  ConfigurablePlugin: Model configuration successful

18:33:32 INFO  ObjectDiagramModelEnricher: Object diagram extraction successful.

18:33:32 INFO  KodkodSolver: Searching solution with SatSolver 'MiniSat' and bitwidth
```

```
    12...
45
46 18:33:34 INFO  KodkodModelValidator: UNSATISFIABLE
47
48 18:33:34 INFO  KodkodModelValidator: Translation time: 486 ms; Solving time: 2036 ms
```

Eidesstattliche Erklärung

Hiermit versichere ich eidesstattlich, dass die vorliegende Arbeit selbstständig und ausschließlich unter Verwendung der angegebenen Quellen und Hilfsmittel angefertigt wurde. Wörtlich oder sinngemäß aus anderen Werken entnommene Stellen wurden unter Angabe der Quelle kenntlich gemacht. Diese Arbeit wurde weder in gleicher noch in ähnlicher Form einem anderen Prüfungsamt vorgelegt oder veröffentlicht.

Bremen, den 7. Oktober 2016 Unterschrift: _____